

The Mothership Has Landed

Adding to the Library

Contents

1. [Introduction](#)
2. [Known behavioral changes](#)
 - 2.1. [Was well-formed, now ill-formed](#)
 - 2.2. [Was ill-formed, now well-formed](#)
3. [Acknowledgments](#)
4. [Wording](#)
 - 4.1. [Clause 16: Library Introduction](#)
 - 4.2. [Clause 17: Language support library](#)
 - 4.3. [Clause 18: Concepts Library](#)
 - 4.4. [Clause 19: Diagnostics Library](#)
 - 4.5. [Clause 20: General utilities library](#)
 - 4.6. [Clause 21: Strings library](#)
 - 4.7. [Clause 22: Containers library](#)
 - 4.8. [Clause 23: Iterators library](#)
 - 4.9. [Clause 24: Ranges library](#)
 - 4.10. [Clause 25: Algorithms library](#)
 - 4.11. [Clause 26: Numerics library](#)
 - 4.12. [Clause 27: Time library](#)
 - 4.13. [Clause 28: Localization library](#)
 - 4.14. [Clause 29: Input/output library](#)
 - 4.15. [Clause 30: Regular expressions library](#)
 - 4.16. [Clause 31: Atomic operations library](#)
 - 4.17. [Clause 32: Thread support library](#)
5. [References](#)

§ 1. Introduction

The work of integrating  into the library has been performed by multiple different papers, each addressing a different aspect of the integration. In the interest of streamlining review by the Library Working Group, the wording has been combined into a single paper. This is that paper.

In San Diego and Kona, several papers were approved by LEWG adding functionality to the library related to comparisons. What follows is the list of those papers, in alphabetical order, with a brief description of what those papers are. The complete motivation and design rationale for each can be found within the papers themselves.

- [P0790R2](#) - adding `operator<=>` to the standard library types whose behavior is not dependent on a template parameter.
- [P0891R2](#) - making the `XXX_order` algorithms customization points and introducing `compare_XXX_order_fallback` algorithms that preferentially invoke the former algorithm and fallback to synthesizing an ordering from `==` and `<` (using the rules from [P1186R1](#)).
- [P1154R1](#) - adding the type trait `has_strong_structural_equality<T>` (useful to check if a type can be used as a non-type template parameter).
- [P1188R0](#) - adding the type trait `compare_three_way_result<T>`, the concepts `ThreeWayComparable<T>` and `ThreeWayComparableWith<T,U>`, removing the algorithm `compare_3way` and replacing it with a function comparison object `compare_three_way` (i.e. the `<=>` version of `std::ranges::less`).
- [P1189R0](#) - adding `operator<=>` to the standard library types whose behavior is dependent on a template parameter, removing those equality operators made redundant by [P1185R1](#) and defaulting `operator==` where appropriate.
- [P1191R0](#) - adding equality to several previously incomparable standard library types.
- [P1295R0](#) - adding equality and `common_type` for the comparison categories.
- [P1380R1](#) - extending the floating point customization points for `strong_order` and `weak_order`.

LEWG's unanimous preference was that `operator<=>` s be declared as hidden friends.

§ 2. Known behavioral changes

There are a few things that will change behavior as a result of all these papers and the chosen direction for declaring operators as hidden friends.

§ 2.1. Was well-formed, now ill-formed

For the preexisting non-member, non-template comparison operators, any comparison that relies on finding the operator in `std` with regular unqualified lookup will fail:

```
using namespace std;
struct X { operator error_code() const; };
X{} == X{};           // ok in C++17, ill-formed with this change
X{} == error_code{}; // ok
```

Here is a more subtle example, reproduced from the LLVM codebase:

```
structStringRef {
    StringRef(std::string const&); // NB: non-explicit
    operator std::string() const; // NB: non-explicit
};
bool operator==(StringRef, StringRef);

bool f(StringRef a, std::string b) {
    8 return a == b; // (*)
}
```

In C++17, the marked line is well-formed. The `operator==` for `basic_string` is a non-member function template, and so would not be considered a candidate; the only viable candidate is the `operator==` taking two `StringRef` s. With the proposed changes, the `operator==` for `basic_string` becomes a non-member hidden friend, *non-*

template, which makes it a candidate (converting `a` to a `string`). That candidate is ambiguous with the `operator==` (`StringRef`, `StringRef`) candidate - each requires a conversion in one argument, so the call becomes ill-formed.

§ 2.2. Was ill-formed, now well-formed

```
bool is42(std::variant<int, std::string> const& v) {  
2   return v == 42; // (*)  
}
```

In C++17, the `operator==` for `variant` is a non-member function template and is thus not a viable candidate for the marked line. That check is ill-formed. With the proposed changes, the `operator==` for `variant` becomes a non-member hidden friend, *non-template*, which makes it a candidate (converting `42` to a `variant<int, string>`). This is arguably a fix, since both `variant<int, string> v = 42;` and `v = 42;` are already well-formed, so it is surely reasonable that `v == 42` is as well.

§ 3. Acknowledgments

Thank you to Casey Carter for the tremendous wording review.

§ 4. Wording

§ 4.1. Clause 16: Library Introduction

Change 16.4.2.1/2 [expos.only.func]:

The following ~~function is~~ are defined for exposition only to aid in the specification of the library:

and append:

```
constexpr auto synth-3way =  
    [<class T, class U>(const T& t, const U& u)  
    requires requires {  
        { t < u } -> bool;  
        { u < t } -> bool;  
    }  
{  
    if constexpr (ThreeWayComparableWith<T, U>) {  
        return t <=> u;  
    } else {  
        if (t < u) return weak_ordering::less;  
        if (u < t) return weak_ordering::greater;  
        return weak_ordering::equivalent;  
    }  
};  
  
template<class T, class U=T>  
using synth-3way-result = decltype(synth-3way(declval<T>(), declval<U>()));
```

Remove 16.4.2.3 [operators], which begins:

In this library, whenever a declaration is provided for an `operator!=`, `operator>`, `operator<=`, or `operator>=` for a type `T`, its requirements and semantics are as follows, unless explicitly specified otherwise.

Add a clause to 16.5.5 [conforming], probably after 16.5.5.4 [global.functions]. Not strictly related to `<=>` as a whole, but it's a requirement that's currently missing and needs to be added somewhere. See also P1601.

16.5.5.x Hidden friend functions [conforming.hidden.friend]

An implementation shall not provide any additional out-of-class declarations or redeclarations for any non-member function specified as a non-member `friend` and defined within the body of a class. [*Note: The intent is that such functions are to be found via argument-dependent lookup only. -end note*]

§ 4.2. Clause 17: Language support library

Added: `compare_three_way_result`, concepts `ThreeWayComparable` and `ThreeWayComparableWith`, `compare_three_way` and `compare_XXX_order_fallback`

Changed operators for: `type_info`

Respecified: `strong_order()`, `weak_order()`, and `partial_order()`

Removed: `compare_3way()`, `strong_equal()`, and `weak_equal()`

In 17.7.2 [type.info], remove `operator!=`:

```
namespace std {
    class type_info {
    public:
        virtual ~type_info();
        bool operator==(const type_info& rhs) const noexcept;
bool operator!=(const type_info& rhs) const noexcept;
        bool before(const type_info& rhs) const noexcept;
        size_t hash_code() const noexcept;
        const char* name() const noexcept;
        type_info(const type_info& rhs) = delete; // cannot be copied
        type_info& operator=(const type_info& rhs) = delete; // cannot be copied
    };
}
```

and

```
bool operator==(const type_info& rhs) const noexcept;
```

Effects: Compares the current object with rhs.

Returns: `true` if the two values describe the same type.

```
bool operator!=(const type_info& rhs) const noexcept;
```

Returns: ~~`!(*this == rhs)`~~.

Add into 17.11.1 [compare.syn]:

```

namespace std {
    // [cmp.categories], comparison category types
    class weak_equality;
    class strong_equality;
    class partial_ordering;
    class weak_ordering;
    class strong_ordering;

    // named comparison functions
    constexpr bool is_eq (weak_equality cmp) noexcept { return cmp == 0; }
    constexpr bool is_neq (weak_equality cmp) noexcept { return cmp != 0; }
    constexpr bool is_lt (partial_ordering cmp) noexcept { return cmp < 0; }
    constexpr bool is_lteq (partial_ordering cmp) noexcept { return cmp <= 0; }
    constexpr bool is_gt (partial_ordering cmp) noexcept { return cmp > 0; }
    constexpr bool is_gteq (partial_ordering cmp) noexcept { return cmp >= 0; }

    // common type specializations
    template<> struct common_type<strong_equality, partial_ordering>
    { using type = weak_equality; };
    template<> struct common_type<partial_ordering, strong_equality>
    { using type = weak_equality; };
    template<> struct common_type<strong_equality, weak_ordering>
    { using type = weak_equality; };
    template<> struct common_type<weak_ordering, strong_equality>
    { using type = weak_equality; };

    // [cmp.common], common comparison category type
    template<class... Ts>
    struct common_comparison_category {
        using type = see below;
    };
    template<class... Ts>
        using common_comparison_category_t = typename common_comparison_category<Ts...>::type;

    // [cmp.concept], concept ThreeWayComparable
    template<class T, class Cat = partial_ordering>
        concept ThreeWayComparable = see below;
    template<class T, class U, class Cat = partial_ordering>
        concept ThreeWayComparableWith = see below;

    // [cmp.result], spaceship invocation result
    template<class T, class U = T> struct compare_three_way_result;

    template<class T, class U = T>
        using compare_three_way_result_t = typename compare_three_way_result<T, U>::type;

    // [cmp.object], spaceship object
    struct compare_three_way;

    // [cmp.alg], comparison algorithms
    template<class T> constexpr strong_ordering strong_order(const T& a, const T& b);
    template<class T> constexpr weak_ordering weak_order(const T& a, const T& b);
    template<class T> constexpr partial_ordering partial_order(const T& a, const T& b);
    template<class T> constexpr strong_equality strong_equal(const T& a, const T& b);
    template<class T> constexpr weak_equality weak_equal(const T& a, const T& b);
    inline namespace unspecified {

```

```

inline constexpr unspecified strong_order = unspecified;
inline constexpr unspecified weak_order = unspecified;
inline constexpr unspecified partial_order = unspecified;
inline constexpr unspecified compare_strong_order_fallback = unspecified;
inline constexpr unspecified compare_weak_order_fallback = unspecified;
inline constexpr unspecified compare_partial_order_fallback = unspecified;
}
}

```

Change 17.11.2.2 [cmp.weak_eq]:

```

namespace std {
class weak_equality {
    int value; // exposition only
    [...]

    // comparisons
    friend constexpr bool operator==(weak_equality v, unspecified) noexcept;
    friend constexpr bool operator!=(weak_equality v, unspecified) noexcept;
    friend constexpr bool operator==(unspecified, weak_equality v) noexcept;
    friend constexpr bool operator!=(unspecified, weak_equality v) noexcept;
    friend constexpr bool operator==(weak_equality v, weak_equality w) noexcept = default;
    friend constexpr weak_equality operator<==(weak_equality v, unspecified) noexcept;
    friend constexpr weak_equality operator<==(unspecified, weak_equality v) noexcept;
};

// valid values' definitions
inline constexpr weak_equality weak_equality::equivalent(eq::equivalent);
inline constexpr weak_equality weak_equality::nonequivalent(eq::nonequivalent);
}

```

Remove the rest of the clause (now defined inline):

```

constexpr bool operator==(weak_equality v, unspecified) noexcept;
constexpr bool operator==(unspecified, weak_equality v) noexcept;

```

~~Returns: v.value == 0.~~

```

constexpr bool operator!=(weak_equality v, unspecified) noexcept;
constexpr bool operator!=(unspecified, weak_equality v) noexcept;

```

~~Returns: v.value != 0.~~

```

constexpr weak_equality operator<==(weak_equality v, unspecified) noexcept;
constexpr weak_equality operator<==(unspecified, weak_equality v) noexcept;

```

~~Returns: v.~~

Change 17.11.2.3 [cmp.strong_eq]:

```

namespace std {
    class strong_equality {
        int value; // exposition only
        [...]

        // comparisons
        friend constexpr bool operator==(strong_equality v, unspecified) noexcept;
        friend constexpr bool operator!=(strong_equality v, unspecified) noexcept;
        friend constexpr bool operator==(unspecified, strong_equality v) noexcept;
        friend constexpr bool operator!=(unspecified, strong_equality v) noexcept;
        friend constexpr bool operator==(strong_equality v, strong_equality w) noexcept = default;
        friend constexpr strong_equality operator<==(strong_equality v, unspecified) noexcept;
        friend constexpr strong_equality operator<==(unspecified, strong_equality v) noexcept;
        friend constexpr strong_equality operator<!=(strong_equality v, unspecified) noexcept;
        friend constexpr strong_equality operator<!=(unspecified, strong_equality v) noexcept;
    };

    // valid values' definitions
    inline constexpr strong_equality strong_equality::equal(eq::equal);
    inline constexpr strong_equality strong_equality::nonequal(eq::nonequal);
    inline constexpr strong_equality strong_equality::equivalent(eq::equivalent);
    inline constexpr strong_equality strong_equality::nonequivalent(eq::nonequivalent);
}

```

Remove most of the rest of the clause:

```
constexpr operator weak_equality() const noexcept;
```

Returns: `value == 0 ? weak_equality::equivalent : weak_equality::nonequivalent`.

```
constexpr bool operator==(strong_equality v, unspecified) noexcept;
constexpr bool operator==(unspecified, strong_equality v) noexcept;
```

Returns: ~~`v.value == 0`~~.

```
constexpr bool operator!=(strong_equality v, unspecified) noexcept;
constexpr bool operator!=(unspecified, strong_equality v) noexcept;
```

Returns: ~~`v.value != 0`~~.

```
constexpr strong_equality operator<==(strong_equality v, unspecified) noexcept;
constexpr strong_equality operator<==(unspecified, strong_equality v) noexcept;
```

Returns: ~~`v`~~.

Change 17.11.2.4 [cmp.partialord]:

```

namespace std {
    class partial_ordering {
        int value; // exposition only
        bool is_ordered; // exposition only

        [...]
    };
}

```

```

// conversion
constexpr operator weak_equality() const noexcept;

// comparisons
friend constexpr bool operator==(partial_ordering v, unspecified) noexcept;
friend constexpr bool operator!=(partial_ordering v, unspecified) noexcept;
friend constexpr bool operator==(partial_ordering v, partial_ordering w) noexcept = default;
friend constexpr bool operator< (partial_ordering v, unspecified) noexcept;
friend constexpr bool operator> (partial_ordering v, unspecified) noexcept;
friend constexpr bool operator<=(partial_ordering v, unspecified) noexcept;
friend constexpr bool operator>=(partial_ordering v, unspecified) noexcept;
friend constexpr bool operator==(unspecified, partial_ordering v) noexcept;
friend constexpr bool operator!=(unspecified, partial_ordering v) noexcept;
friend constexpr bool operator< (unspecified, partial_ordering v) noexcept;
friend constexpr bool operator> (unspecified, partial_ordering v) noexcept;
friend constexpr bool operator<=(unspecified, partial_ordering v) noexcept;
friend constexpr bool operator>=(unspecified, partial_ordering v) noexcept;
friend constexpr partial_ordering operator<=>(partial_ordering v, unspecified) noexcept;
friend constexpr partial_ordering operator<=>(unspecified, partial_ordering v) noexcept;
};

[...]
```

Remove just the extra `==` and `!=` operators in 17.11.2.4 [cmp.partialord]/3 and 4:

```

constexpr bool operator==(partial_ordering v, unspecified) noexcept;
constexpr bool operator< (partial_ordering v, unspecified) noexcept;
constexpr bool operator> (partial_ordering v, unspecified) noexcept;
constexpr bool operator<=(partial_ordering v, unspecified) noexcept;
constexpr bool operator>=(partial_ordering v, unspecified) noexcept;
```

Returns: For `operator@`, `v.is_ordered && v.value @ 0`.

```

constexpr bool operator==(unspecified, partial_ordering v) noexcept;
constexpr bool operator< (unspecified, partial_ordering v) noexcept;
constexpr bool operator> (unspecified, partial_ordering v) noexcept;
constexpr bool operator<=(unspecified, partial_ordering v) noexcept;
constexpr bool operator>=(unspecified, partial_ordering v) noexcept;
```

Returns: For `operator@`, `v.is_ordered && 0 @ v.value`.

```

constexpr bool operator!=(partial_ordering v, unspecified) noexcept;
constexpr bool operator!=(unspecified, partial_ordering v) noexcept;
```

Returns: For `operator@`, `+v.is_ordered || v.value != 0`.

Change 17.11.2.5 [cmp.weakord]:

```

namespace std {
    class weak_ordering {
        int value; // exposition only

        [...]
        // comparisons
```



```

friend constexpr bool operator==(weak_ordering v, unspecified) noexcept;
friend constexpr bool operator==(weak_ordering v, weak_ordering w) noexcept = default;
friend constexpr bool operator!=(weak_ordering v, unspecified) noexcept;
friend constexpr bool operator< (weak_ordering v, unspecified) noexcept;
friend constexpr bool operator> (weak_ordering v, unspecified) noexcept;
friend constexpr bool operator<=(weak_ordering v, unspecified) noexcept;
friend constexpr bool operator>=(weak_ordering v, unspecified) noexcept;
friend constexpr bool operator==(unspecified, weak_ordering v) noexcept;
friend constexpr bool operator!=(unspecified, weak_ordering v) noexcept;
friend constexpr bool operator< (unspecified, weak_ordering v) noexcept;
friend constexpr bool operator> (unspecified, weak_ordering v) noexcept;
friend constexpr bool operator<=(unspecified, weak_ordering v) noexcept;
friend constexpr bool operator>=(unspecified, weak_ordering v) noexcept;
friend constexpr weak_ordering operator<=>(weak_ordering v, unspecified) noexcept;
friend constexpr weak_ordering operator<=>(unspecified, weak_ordering v) noexcept;
};

[...]
};

```

Remove just the extra `==` and `!=` operators from 17.11.2.5 [cmp.weakord]/4 and /5:

```

constexpr bool operator==(weak_ordering v, unspecified) noexcept;
constexpr bool operator!=(weak_ordering v, unspecified) noexcept;
constexpr bool operator< (weak_ordering v, unspecified) noexcept;
constexpr bool operator> (weak_ordering v, unspecified) noexcept;
constexpr bool operator<=(weak_ordering v, unspecified) noexcept;
constexpr bool operator>=(weak_ordering v, unspecified) noexcept;

```

Returns: `v.value @ 0` for `operator@`.

```

constexpr bool operator==(unspecified, weak_ordering v) noexcept;
constexpr bool operator!=(unspecified, weak_ordering v) noexcept;
constexpr bool operator< (unspecified, weak_ordering v) noexcept;
constexpr bool operator> (unspecified, weak_ordering v) noexcept;
constexpr bool operator<=(unspecified, weak_ordering v) noexcept;
constexpr bool operator>=(unspecified, weak_ordering v) noexcept;

```

Returns: `0 @ v.value` for `operator@`.

Change 17.11.2.6 [cmp.strongord]:

```

namespace std {
class strong_ordering {
    int value; // exposition only

    [...]

    // comparisons
    friend constexpr bool operator==(strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator==(strong_ordering v, strong_ordering w) noexcept = default;
friend constexpr bool operator!=(strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator< (strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator> (strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator<=(strong_ordering v, unspecified) noexcept;

```

```

friend constexpr bool operator>=(strong_ordering v, unspecified) noexcept;
friend constexpr bool operator==(unspecified, strong_ordering v) noexcept;
friend constexpr bool operator!=(unspecified, strong_ordering v) noexcept;
friend constexpr bool operator< (unspecified, strong_ordering v) noexcept;
friend constexpr bool operator> (unspecified, strong_ordering v) noexcept;
friend constexpr bool operator<=(unspecified, strong_ordering v) noexcept;
friend constexpr bool operator>=(unspecified, strong_ordering v) noexcept;
friend constexpr strong_ordering operator<=>(strong_ordering v, unspecified) noexcept;
friend constexpr strong_ordering operator<=>(unspecified, strong_ordering v) noexcept;
};

[...]
```

Remove just the extra `==` and `!=` operators from 17.11.2.6 [cmp.strongord]/6 and /7:

```

constexpr bool operator==(strong_ordering v, unspecified) noexcept;
constexpr bool operator!=(strong_ordering v, unspecified) noexcept;
constexpr bool operator< (strong_ordering v, unspecified) noexcept;
constexpr bool operator> (strong_ordering v, unspecified) noexcept;
constexpr bool operator<=(strong_ordering v, unspecified) noexcept;
constexpr bool operator>=(strong_ordering v, unspecified) noexcept;
```

Returns: `v.value @ 0` for `operator@`.

```

constexpr bool operator==(unspecified, strong_ordering v) noexcept;
constexpr bool operator!=(unspecified, strong_ordering v) noexcept;
constexpr bool operator< (unspecified, strong_ordering v) noexcept;
constexpr bool operator> (unspecified, strong_ordering v) noexcept;
constexpr bool operator<=(unspecified, strong_ordering v) noexcept;
constexpr bool operator>=(unspecified, strong_ordering v) noexcept;
```

Returns: `0 @ v.value` for `operator@`.

Add a new subclause [cmp.concept] "concept `ThreeWayComparable`":

```

template <typename T, typename Cat>
concept compares-as = // exposition only
    Same<common_comparison_category_t<T, Cat>, Cat>;
```

```

template<class T, class U>
concept partially-ordered-with = // exposition only
    requires(const remove_reference_t<T>& t,
        const remove_reference_t<U>& u) {
        { t < u } -> Boolean;
        { t > u } -> Boolean;
        { t <= u } -> Boolean;
        { t >= u } -> Boolean;
        { u < t } -> Boolean;
        { u > t } -> Boolean;
        { u <= t } -> Boolean;
```

```
{ u >= t } -> Boolean;
};
```

Let `t` and `u` be lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>` respectively. `partially-ordered-with<T, U>` is satisfied only if:

- `t < u`, `t <= u`, `t > u`, `t >= u`, `u < t`, `u <= t`, `u > t`, and `u >= t` have the same domain.
- `bool(t < u) == bool(u > t)`
- `bool(u < t) == bool(t > u)`
- `bool(t <= u) == bool(u >= t)`
- `bool(u <= t) == bool(t >= u)`

```
template <typename T, typename Cat = partial_ordering>
concept ThreeWayComparable =
    weakly-equality-comparable-with<T, T> &&
    (!ConvertibleTo<Cat, partial_ordering> || partially-ordered-with<T, T>) &&
    requires(const remove_reference_t<T>& a,
              const remove_reference_t<T>& b) {
        { a <=> b } -> compares-as<Cat>;
    };
```

Let `a` and `b` be lvalues of type `const remove_reference_t<T>`. `T` and `Cat` model `ThreeWayComparable<T, Cat>` only if:

- `(a <=> b == 0) == bool(a == b)`.
- `(a <=> b != 0) == bool(a != b)`.
- `((a <=> b) <=> 0)` and `(0 <=> (b <=> a))` are equal
- If `Cat` is convertible to `strong_equality`, `T` models `EqualityComparable` ([concept.equalitycomparable]).
- If `Cat` is convertible to `partial_ordering`:
 - `(a <=> b < 0) == bool(a < b)`.
 - `(a <=> b > 0) == bool(a > b)`.
 - `(a <=> b <= 0) == bool(a <= b)`.
 - `(a <=> b >= 0) == bool(a >= b)`.
- If `Cat` is convertible to `strong_ordering`, `T` models `StrictTotallyOrdered` ([concept.stricttotallyordered]).

```
template <typename T, typename U,
          typename Cat = partial_ordering>
concept ThreeWayComparableWith =
    weakly-equality-comparable-with<T, U> &&
    (!ConvertibleTo<Cat, partial_ordering> || partially-ordered-with<T, U>) &&
    ThreeWayComparable<T, Cat> &&
    ThreeWayComparable<U, Cat> &&
    CommonReference<const remove_reference_t<T>&, const remove_reference_t<U>&> &&
    ThreeWayComparable<
        common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>,
        Cat> &&
    requires(const remove_reference_t<T>& t,
              const remove_reference_t<U>& u) {
```

```

{ t <=> u } -> compares-as<Cat>;
{ u <=> t } -> compares-as<Cat>;
};

```

Let `t` and `u` be lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`, respectively. Let `C` be `common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>`. `T`, `U`, and `Cat` model `ThreeWayComparableWith<T, U, Cat>` only if:

- `t <=> u` and `u <=> t` have the same domain.
- `((t <=> u) <=> 0)` and `(0 <=> (u <=> t))` are equal
- `(t <=> u == 0) == bool(t == u)`.
- `(t <=> u != 0) == bool(t != u)`.
- `Cat(t <=> u) == Cat(C(t) <=> C(u))`.
- If `Cat` is convertible to `strong_equality`, `T` and `U` model `EqualityComparableWith<T, U>` ([`concepts.equalitycomparable`]).
- If `Cat` is convertible to `partial_ordering`:
 - `(t <=> u < 0) == bool(t < u)`
 - `(t <=> u > 0) == bool(t > u)`
 - `(t <=> u <= 0) == bool(t <= u)`
 - `(t <=> u >= 0) == bool(t >= u)`
- If `Cat` is convertible to `strong_ordering`, `T` and `U` model `StrictTotallyOrderedWith<T, U>` ([`concepts.stricttotallyordered`]).

Add a new subclause [cmp.result] "spaceship invocation result":

The behavior of a program that adds specializations for the `compare_three_way_result` template defined in this subclause is undefined.

For the `compare_three_way_result` type trait applied to the types `T` and `U`, let `t` and `u` denote lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`, respectively. If the expression `t <=> u` is well-formed, the member *typedef-name* `type` denotes the type `decltype(t <=> u)`. Otherwise, there is no member `type`.

Add a new subclause [cmp.object] "spaceship object":

In this subclause, `BUILTIN_PTR_3WAY(T, U)` for types `T` and `U` is a boolean constant expression.

`BUILTIN_PTR_3WAY(T, U)` is `true` if and only if `<=>` in the expression `declval<T>() <=> declval<U>()` resolves to a built-in operator comparing pointers.

```

struct compare_three_way {
    template<class T, class U>
        requires ThreeWayComparableWith<T,U> || BUILTIN_PTR_3WAY(T, U)
    constexpr auto operator()(T&& t, U&& u) const;

    using is_transparent = unspecified;
};

```

Expects: If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`, the conversion sequences from both `T` and `U` to `P` are equality-preserving ([`concepts.equality`]).

Effects:

- If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`: returns `strong_ordering::less` if (the converted value of) `t` precedes `u` in the implementation-defined strict total order (`[range.cmp]`) over pointers of type `P`, `strong_ordering::greater` if `u` precedes `t`, and otherwise `strong_ordering::equal`.
- Otherwise, equivalent to: `return std::forward<T>(t) <=> std::forward<U>(u);`

In addition to being available via inclusion of the `<compare>` header, the class `compare_three_way` is available when the header `<functional>` is included.

Replace the entirety of 17.11.4 [cmp.alg]. This wording relies on the specification-only function `3WAY<R>` defined in [P1186R1](#).

```
template<class T> constexpr strong_ordering strong_order(const T& a, const T& b);
```

~~Effects: Compares two values and produces a result of type `strong_ordering`:~~

- ~~• If `numeric_limits::is_iec559` is true, returns a result of type `strong_ordering` that is consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.~~
- ~~• Otherwise, returns `a <=> b` if that expression is well-formed and convertible to `strong_ordering`.~~
- ~~• Otherwise, if the expression `a <=> b` is well-formed, then the function is defined as deleted.~~
- ~~• Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then
 - ~~◦ if `a == b` is true, returns `strong_ordering::equal`;~~
 - ~~◦ otherwise, if `a < b` is true, returns `strong_ordering::less`;~~
 - ~~◦ otherwise, returns `strong_ordering::greater`.~~~~
- ~~• Otherwise, the function is defined as deleted.~~

```
template<class T> constexpr weak_ordering weak_order(const T& a, const T& b);
```

~~Effects: Compares two values and produces a result of type `weak_ordering`:~~

- ~~• Returns `a <=> b` if that expression is well-formed and convertible to `weak_ordering`.~~
- ~~• Otherwise, if the expression `a <=> b` is well-formed, then the function is defined as deleted.~~
- ~~• Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then
 - ~~◦ if `a == b` is true, returns `weak_ordering::equivalent`;~~
 - ~~◦ otherwise, if `a < b` is true, returns `weak_ordering::less`;~~
 - ~~◦ otherwise, returns `weak_ordering::greater`.~~~~
- ~~• Otherwise, the function is defined as deleted.~~

```
template<class T> constexpr partial_ordering partial_order(const T& a, const T& b);
```

~~Effects: Compares two values and produces a result of type `partial_ordering`:~~

- ~~• Returns `a <=> b` if that expression is well-formed and convertible to `partial_ordering`.~~
- ~~• Otherwise, if the expression `a <=> b` is well-formed, then the function is defined as deleted.~~
- ~~• Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then
 - ~~◦ if `a == b` is true, returns `partial_ordering::equivalent`;~~
 - ~~◦ otherwise, if `a < b` is true, returns `partial_ordering::less`;~~
 - ~~◦ otherwise, returns `partial_ordering::greater`.~~~~
- ~~• Otherwise, the function is defined as deleted.~~

```
template<class T> constexpr strong_equality strong_equal(const T& a, const T& b);
```

Effects: Compares two values and produces a result of type `strong_equality`:

- Returns `a <=> b` if that expression is well-formed and convertible to `strong_equality`.
- Otherwise, if the expression `a <=> b` is well-formed, then the function is defined as deleted.
- Otherwise, if the expression `a == b` is well-formed and convertible to `bool`, then
 - if `a == b` is true, returns `strong_equality::equal`;
 - otherwise, returns `strong_equality::nonequal`.
- Otherwise, the function is defined as deleted.

```
template<class T> constexpr weak_equality weak_equal(const T& a, const T& b);
```

Effects: Compares two values and produces a result of type `weak_equality`:

- Returns `a <=> b` if that expression is well-formed and convertible to `weak_equality`.
- Otherwise, if the expression `a <=> b` is well-formed, then the function is defined as deleted.
- Otherwise, if the expression `a == b` is well-formed and convertible to `bool`, then
 - if `a == b` is true, returns `weak_equality::equivalent`;
 - otherwise, returns `weak_equality::nonequivalent`.
- Otherwise, the function is defined as deleted.

The name `strong_order` denotes a customization point object ([customization.point.object]). The expression `strong_order(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to the following:

- If the decayed types of `E` and `F` differ, `strong_order(E, F)` is ill-formed.
- Otherwise, `strong_ordering(strong_order(E, F))` if it is a well-formed expression with overload resolution performed in a context that does not include a declaration of `std::strong_order`.
- Otherwise, if the decayed type `T` of `E` and `F` is a floating point type, yields a value of type `strong_ordering` that is consistent with the ordering observed by `T`'s comparison operators, and if `numeric_limits<T>::is_iec559` is `true` is additionally consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60599.
- Otherwise, `strong_ordering(E <=> F)` if it is a well-formed expression.
- Otherwise, `strong_order(E, F)` is ill-formed. [Note: This case can result in substitution failure when `strong_order(E, F)` appears in the immediate context of a template instantiation. —end note]

The name `weak_order` denotes a customization point object ([customization.point.object]). The expression `weak_order(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to the following:

- If the decayed types of `E` and `F` differ, `weak_order(E, F)` is ill-formed.
- Otherwise, `weak_ordering(weak_order(E, F))` if it is a well-formed expression with overload resolution performed in a context that does not include a declaration of `std::weak_order`.
- Otherwise, if the decayed type `T` of `E` and `F` is a floating point type, yields a value of type `weak_ordering` that is consistent with the ordering observed by `T`'s comparison operators and `strong_order`, and if `numeric_limits<T>::is_iec559` is `true` is additionally consistent with the following equivalence classes, ordered from lesser to greater:
 - Together, all negative NaN values
 - Negative infinity

- Each normal negative value
- Each subnormal negative value
- Together, both zero values
- Each subnormal positive value
- Each normal positive value
- Positive infinity
- Together, all positive NaN values
- Otherwise, `weak_ordering(strong_order(E, F))` if it is a well-formed expression.
- Otherwise, `weak_ordering(E <=> F)` if it is a well-formed expression.
- Otherwise, `weak_order(E, F)` is ill-formed. [Note: This case can result in substitution failure when `std::weak_order(E, F)` appears in the immediate context of a template instantiation. —end note]

The name `partial_order` denotes a customization point object ([customization.point.object]). The expression `partial_order(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to the following:

- If the decayed types of `E` and `F` differ, `partial_order(E, F)` is ill-formed.
- Otherwise, `partial_ordering(partial_order(E, F))` if it is a well-formed expression with overload resolution performed in a context that does not include a declaration of `std::partial_order`.
- Otherwise, `partial_ordering(weak_order(E, F))` if it is a well-formed expression.
- Otherwise, `partial_ordering(E <=> F)` if it is a well-formed expression.
- Otherwise, `partial_order(E, F)` is ill-formed. [Note: This case can result in substitution failure when `std::partial_order(E, F)` appears in the immediate context of a template instantiation. —end note]

The name `compare_strong_order_fallback` denotes a comparison customization point ([customization.point.object]) object. The expression `compare_strong_order_fallback(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to:

- If the decayed types of `E` and `F` differ, `compare_strong_order_fallback(E, F)` is ill-formed.
- Otherwise, `strong_order(E, F)` if it is a well-formed expression.
- Otherwise, `3WAY<strong_ordering>(E, F)` ([class.spaceship]) if it is a well-formed expression.
- Otherwise, `compare_strong_order_fallback(E, F)` is ill-formed.

The name `compare_weak_order_fallback` denotes a customization point object ([customization.point.object]). The expression `compare_weak_order_fallback(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to:

- If the decayed types of `E` and `F` differ, `compare_weak_order_fallback(E, F)` is ill-formed.
- Otherwise, `weak_order(E, F)` if it is a well-formed expression.
- Otherwise, `3WAY<weak_ordering>(E, F)` ([class.spaceship]) if it is a well-formed expression.
- Otherwise, `compare_weak_order_fallback(E, F)` is ill-formed.

The name `compare_partial_order_fallback` denotes a customization point object ([customization.point.object]). The expression `compare_partial_order_fallback(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to:

- If the decayed types of `E` and `F` differ, `compare_partial_order_fallback(E, F)` is ill-formed.
- Otherwise, `partial_order(E, F)` if it is a well-formed expression.
- Otherwise, `3WAY<partial_ordering>(E, F)` ([class.spaceship]) if it is a well-formed expression.
- Otherwise, `compare_partial_order_fallback(E, F)` is ill-formed.

Change 17.13.1 [coroutine.syn]:

```
namespace std {
    [...]
    // 17.13.5 noop coroutine
    noop_coroutine_handle noop_coroutine() noexcept;

    // 17.13.3.6 comparison operators:
    constexpr bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    constexpr bool operator!=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    constexpr bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    constexpr bool operator>(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    constexpr bool operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    constexpr bool operator>=(coroutine_handle<> x, coroutine_handle<> y) noexcept;

    // 17.13.6 trivial awaitables
    [...]
}
```

Change 17.13.3 [coroutine.handle]:

```
namespace std {
    template <>
    struct coroutine_handle<void>
    {
        [...]
        // 17.13.3.4 resumption
        void operator()() const;
        void resume() const;
        void destroy() const;

        // comparison operators
        friend constexpr bool operator==(coroutine_handle x, coroutine_handle y) noexcept
        { return x.address() == y.address(); }
        friend constexpr strong_ordering operator<=>(coroutine_handle x, coroutine_handle y) r
        { return compare_three_way()(x.address(), y.address()); }
    private:
        void* ptr; // exposition only
    };
    [...]
}
```

Remove 17.13.3.6 [coroutine.handle.compare] (as it's now all defined in the header):

```
constexpr bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

~~Returns: `x.address() == y.address()`.~~

```
constexpr bool operator!=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

~~Returns: `!(x == y)`.~~

```
<constexpr bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```


~~Returns: `less<>()(x.address(), y.address())`:~~

```
constexpr bool operator>(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

~~Returns: `(y < x)`:~~

```
constexpr bool operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

~~Returns: `!(x > y)`:~~

```
constexpr bool operator>=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

~~Returns: `!(x < y)`:~~

§ 4.3. Clause 18: Concepts Library

No changes.

§ 4.4. Clause 19: Diagnostics Library

Changed operators for: `error_category`, `error_code`, and `error_condition`

Change 19.5.1 [system_error.syn]

```
namespace std {
    [...]
    // [syserr.errcondition.nonmembers], non-member functions
    error_condition make_error_condition(errc e) noexcept;

    // [syserr.compare], comparison functions
    bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
    bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
    bool operator==(const error_condition& lhs, const error_code& rhs) noexcept;
    bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
    bool operator!=(const error_code& lhs, const error_code& rhs) noexcept;
    bool operator!=(const error_code& lhs, const error_condition& rhs) noexcept;
    bool operator!=(const error_condition& lhs, const error_code& rhs) noexcept;
    bool operator!=(const error_condition& lhs, const error_condition& rhs) noexcept;
    bool operator<(const error_code& lhs, const error_code& rhs) noexcept;
    bool operator<(const error_condition& lhs, const error_condition& rhs) noexcept;

    // [syserr.hash], hash support
    [...]
}
```

Change 19.5.2.1 [syserr.errcat.overview]:

```
namespace std {
    class error_category {
        [...]
        bool operator==(const error_category& rhs) const noexcept;
        bool operator!=(const error_category& rhs) const noexcept;
        bool operator<(const error_category& rhs) const noexcept;
    };
}
```

```

    strong_ordering_operator<=>(const error_category& rhs) const noexcept;
};
[...]
```

Change 19.5.2.3 [syserr.errcat.nonvirtuals]:

```
bool operator==(const error_category& rhs) const noexcept;
```

Returns: `this == &rhs`.

```
bool operator!=(const error_category& rhs) const noexcept;
```

~~Returns: `!(*this == rhs)`.~~

```
bool operator<(const error_category& rhs) const noexcept;
```

~~Returns: `less<const error_category*>()(this, &rhs)`.~~

~~[Note: `less` (19.14.7) provides a total ordering for pointers. —end note]~~

```
strong_ordering_operator<=>(const error_category& rhs) const noexcept;
```

Returns: `compare_three_way()(this, &rhs)`.

[Note: `compare_three_way` (cmp.object) provides a total ordering for pointers. —end note]

Change 19.5.3.1 [syserr.errcode.overview]:

```

namespace std {
    class error_code {
    [...]
        // [syserr.compare], comparison functions
        friend bool operator==(const error_code&, const error_code&) { see below; }
        friend strong_ordering_operator<=>(const error_code&, const error_code&) { see below; }
        friend bool operator==(const error_code&, const error_condition&) { see below; }
    private:
        int val_; // exposition only
        const error_category* cat_; // exposition only
    };

    [...]
}
```

Change 19.5.4.1 [syserr.errcondition.overview]:

```

namespace std {
    class error_condition {
    public:
    [...]
        // [syserr.compare], comparison functions
        friend bool operator==(const error_condition&, const error_condition&) { see below; }
        friend strong_ordering_operator<=>(const error_condition&, const error_condition&) { see below; }
        friend bool operator==(const error_condition&, const error_code&) { see below; }
    private:
```

```

    int val_; // exposition only
    const error_category* cat_; // exposition only
};
}

```

Change 19.5.5 [syserr.compare]

```
friend bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
```

Returns: `lhs.category() == rhs.category() && lhs.value() == rhs.value()`

Remarks: This function is to be found via argument-dependent lookup only.

```
friend bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
```

Returns: `lhs.category().equivalent(lhs.value(), rhs) || rhs.category().equivalent(lhs, rhs.value())`

Remarks: This function is to be found via argument-dependent lookup only.

```
friend bool operator==(const error_condition& lhs, const error_code& rhs) noexcept;
```

Returns: `rhs.category().equivalent(rhs.value(), lhs) || lhs.category().equivalent(rhs, lhs.value())`

Remarks: This function is to be found via argument-dependent lookup only.

```
friend bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
```

Returns: `lhs.category() == rhs.category() && lhs.value() == rhs.value()`

Remarks: This function is to be found via argument-dependent lookup only.

```

bool operator!=(const error_code& lhs, const error_code& rhs) noexcept;
bool operator!=(const error_code& lhs, const error_condition& rhs) noexcept;
bool operator!=(const error_condition& lhs, const error_code& rhs) noexcept;
bool operator!=(const error_condition& lhs, const error_condition& rhs) noexcept;

```

~~Returns: `!(lhs == rhs)`.~~

```
bool operator<(const error_code& lhs, const error_code& rhs) noexcept;
```

~~Returns: `lhs.category() < rhs.category() || (lhs.category() == rhs.category() && lhs.value() < rhs.value())`~~

```
bool operator<(const error_condition& lhs, const error_condition& rhs) noexcept;
```

~~Returns: `lhs.category() < rhs.category() || (lhs.category() == rhs.category() && lhs.value() < rhs.value())`~~

```
friend strong_ordering operator<=>(const error_code& lhs, const error_code& rhs) noexcept;
```

Effects: Equivalent to:

```
if (auto c = lhs.category() <=> rhs.category(); c != 0) return c;
return lhs.value() <=> rhs.value();
```

Remarks: This function is to be found via argument-dependent lookup only.

```
friend strong_ordering_operator<=>(const error_condition& lhs, const error_condition& rhs)
```

Effects: Equivalent to:

```
if (auto c = lhs.category() <=> rhs.category(); c != 0) return c;
return lhs.value() <=> rhs.value();
```

Remarks: This function is to be found via argument-dependent lookup only.

§ 4.5. Clause 20: General utilities library

Changed operators for: `pair`, `tuple`, `optional`, `variant`, `monostate`, `bitset`, `allocator`, `unique_ptr`, `shared_ptr`, `memory_resource`, `polymorphic_allocator`, `scoped_allocator_adaptor`, `function`, `type_index`

Change 20.2.1 [utility.syn]

```
#include // see 16.10.1

namespace std {
    [...]
    // 20.4, class template pair
    template<class T1, class T2>
    struct pair;

    // 20.4.3, pair specialized algorithms
    template<class T1, class T2>
    constexpr bool operator==(const pair<T1, T2>&, const pair<T1, T2>&);
    template<class T1, class T2>
    constexpr bool operator!=(const pair<T1, T2>&, const pair<T1, T2>&);
    template<class T1, class T2>
    constexpr bool operator<(const pair<T1, T2>&, const pair<T1, T2>&);
    template<class T1, class T2>
    constexpr bool operator>(const pair<T1, T2>&, const pair<T1, T2>&);
    template<class T1, class T2>
    constexpr bool operator<=(const pair<T1, T2>&, const pair<T1, T2>&);
    template<class T1, class T2>
    constexpr bool operator>=(const pair<T1, T2>&, const pair<T1, T2>&);

    [...]
}
```

Change 20.4.2 [pairs.pair]:

```
namespace std {
    template<class T1, class T2>
```

```

struct pair {
    [...]
    constexpr void swap(pair& p) noexcept(see below);

    friend constexpr bool operator==(const pair&, const pair&) = default;
    friend constexpr common comparison category t<synth-3way-result<T1>, synth-3way-result<T2>>
    operator<=>(const pair&, const pair&).
    { see below }.
};

```

[...]

```
constexpr void swap(pair& p) noexcept(see below);
```

Requires: `first` shall be swappable with (15.5.3.2) `p.first` and `second` shall be swappable with `p.second`.

Effects: Swaps `first` with `p.first` and `second` with `p.second`.

Remarks: The expression inside `noexcept` is equivalent to: `is_nothrow_swappable_v<first_type> && is_nothrow_swappable_v<second_type>`

```

friend constexpr common comparison category t<synth-3way-result<T1>, synth-3way-result<T2>>
operator<=>(const pair& lhs, const pair& rhs);

```

Effects: Equivalent to:

```

if (auto c = synth-3way(lhs.first, rhs.first); c != 0) return c;
return synth-3way(lhs.second, rhs.second);

```

Remarks: This function is to be found via argument-dependent lookup only.

Change 20.4.3 [pairs.spec].

```

template<class T1, class T2>
constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);

```

~~**Returns:** `x.first == y.first && x.second == y.second` :~~

```

template<class T1, class T2>
constexpr bool operator!=(const pair<T1, T2>& x, const pair<T1, T2>& y);

```

~~**Returns:** `!(x == y)` :~~

```

template<class T1, class T2>
constexpr bool operator<(const pair<T1, T2>& x, const pair<T1, T2>& y);

```

~~**Returns:** `x.first < y.first || (!(y.first < x.first) && x.second < y.second)` :~~

```

template<class T1, class T2>
constexpr bool operator>(const pair<T1, T2>& x, const pair<T1, T2>& y);

```

~~**Returns:** `y < x` .~~

```
template<class T1, class T2>
constexpr bool operator<=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

~~Returns: $!(y < x)$.~~

```
template<class T1, class T2>
constexpr bool operator>=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

~~Returns: $!(x < y)$.~~

```
template<class T1, class T2>
constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y) noexcept(noexcept(x.swap(y)));
```

[...]

Change 20.5.2 [tuple.syn]:

```
namespace std {
    // 20.5.3, class template tuple
    template<class... Types>
        class tuple;

    [...]

    template<class T, class... Types>
        constexpr const T& get(const tuple<Types...>&& t) noexcept;

    // 20.5.3.8, relational operators
    template<class... TTypes, class... UTypes>
        constexpr bool operator==(const tuple<TTypes...>&, const tuple<UTypes...>&);
    template<class... TTypes, class... UTypes>
        constexpr bool operator!=(const tuple<TTypes...>&, const tuple<UTypes...>&);
    template<class... TTypes, class... UTypes>
        constexpr bool operator<(const tuple<TTypes...>&, const tuple<UTypes...>&);
    template<class... TTypes, class... UTypes>
        constexpr bool operator>(const tuple<TTypes...>&, const tuple<UTypes...>&);
    template<class... TTypes, class... UTypes>
        constexpr bool operator<=(const tuple<TTypes...>&, const tuple<UTypes...>&);
    template<class... TTypes, class... UTypes>
        constexpr bool operator>=(const tuple<TTypes...>&, const tuple<UTypes...>&);

    // 20.5.3.9, allocator-related traits
    template<class... Types, class Alloc>
        struct uses_allocator<tuple<Types...>, Alloc>;

    [...]
}
```

Change 20.5.3 [tuple.tuple]:

```
namespace std {
    template
    class tuple {
    public:
```

```
[...]

// 20.5.3.3, tuple swap
constexpr void swap(tuple&) noexcept(see below );

// 20.5.3.8, tuple relational operators
template<class... UTypes>
    friend constexpr bool operator==(const tuple&, const tuple<UTypes...>&).
    { see below }.
template<class... UTypes>
    friend constexpr common comparison category t<synth-3way-result<Types, UTypes>...>
    operator<=>(const tuple&, const tuple<UTypes...>&).
    { see below }.
};
```

Change 20.5.3.8 [tuple.rel]:

```
template<class... TTypes, class... UTypes>
constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

```
template<class... UTypes>
    friend constexpr bool operator==(const tuple&, const tuple<UTypes...>&).
```

Requires: For all i , where $0 \leq i$ and $i < \text{sizeof} \dots (\text{TTypes } \text{Types})$, $\text{get}<i>(t) == \text{get}<i>(u)$ is a well-formed expression returning a type that is convertible to `bool`.

`sizeof... (TTypes Types) == sizeof... (UTypes)`.

Returns: `true` if $\text{get}<i>(t) == \text{get}<i>(u)$ for all i , otherwise `false`. For any two zero-length tuples e and f , $e == f$ returns `true`.

Effects: The elementary comparisons are performed in order from the zeroth index upwards. No comparisons or element accesses are performed after the first equality comparison that evaluates to `false`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class... TTypes, class... UTypes>
constexpr bool operator!=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

Returns: $!(t == u)$.

```
template<class... TTypes, class... UTypes>
constexpr bool operator<(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

```
template<class... UTypes>
    friend constexpr common comparison category t<synth-3way-result<Types, UTypes>...>
    operator<=>(const tuple& t, const tuple<UTypes...>& u);.
```

Requires: For all i , where $0 \leq i$ and $i < \text{sizeof} \dots (\text{Types})$, ~~both $\text{get}<i>(t) \leftarrow \text{get}<i>(u)$ and $\text{get}<i>(u) \leftarrow \text{get}<i>(t)$ are well-formed expressions returning types that are convertible to `bool`~~ `synth-3way(get<i>(t), get<i>(u))` is a well-formed expression.

`sizeof... (TTypes Types) == sizeof... (UTypes)`.

Returns: ~~The result of a lexicographical comparison between t and u . The result is defined as: `(bool)` $(\text{get}<0>(t) \leftarrow \text{get}<0>(u)) \mid \mid (!(\text{bool})(\text{get}<0>(u) \leftarrow \text{get}<0>(t)) \ \&\& \ t_{\text{tail}} \leftarrow u_{\text{tail}})$, where r_{tail} for some tuple r is a tuple containing all but the first element of r . For any two zero-length tuples e and f , e~~

~~<f returns false:~~

Effects: Performs a lexicographical comparison between `t` and `u`. For any two zero-length tuples `t` and `u`, `t <=> u` returns `strong_ordering::equal`. Otherwise, equivalent to:

```
auto c = synth-3way(get<0>(t), get<0>(u));
return (c != 0) ? c : (ttail <=> utail);
```

where `rtail` for some tuple `r` is a tuple containing all but the first element of `r`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class... TTypes, class... UTypes>
constexpr bool operator<(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

~~Returns: `u < t`.~~

```
template<class... TTypes, class... UTypes>
constexpr bool operator<=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

~~Returns: `!(u < t)`.~~

```
template<class... TTypes, class... UTypes>
constexpr bool operator>(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

~~Returns: `!(t < u)`.~~

[Note: The above definitions for comparison functions do not require `ttail` (or `utail`) to be constructed. It may not even be possible, as `t` and `u` are not required to be copy constructible. Also, all comparison functions are short circuited; they do not perform element accesses beyond what is required to determine the result of the comparison. —end note]

Change 20.6.2 [optional.syn]:

```
namespace std {
    [...]
    // [optional.bad.access], class bad_optional_access
    class bad_optional_access;

    // [optional.relops], relational operators
    template<class T, class U>
    constexpr bool operator==(const optional<T>&, const optional<U>&);
    template<class T, class U>
    constexpr bool operator!=(const optional<T>&, const optional<U>&);
    template<class T, class U>
    constexpr bool operator<(const optional<T>&, const optional<U>&);
    template<class T, class U>
    constexpr bool operator>(const optional<T>&, const optional<U>&);
    template<class T, class U>
    constexpr bool operator<=(const optional<T>&, const optional<U>&);
    template<class T, class U>
    constexpr bool operator>=(const optional<T>&, const optional<U>&);

    // [optional.nullopts], comparison with nullopt
    template<class T> constexpr bool operator==(const optional<T>&, nullopt_t) noexcept;
```



```

template<class T> constexpr bool operator==(nullopt_t, const optional<T>&) noexcept;
template<class T> constexpr bool operator!=(const optional<T>&, nullopt_t) noexcept;
template<class T> constexpr bool operator!=(nullopt_t, const optional<T>&) noexcept;
template<class T> constexpr bool operator<(const optional<T>&, nullopt_t) noexcept;
template<class T> constexpr bool operator<(nullopt_t, const optional<T>&) noexcept;
template<class T> constexpr bool operator>(const optional<T>&, nullopt_t) noexcept;
template<class T> constexpr bool operator>(nullopt_t, const optional<T>&) noexcept;
template<class T> constexpr bool operator<=(const optional<T>&, nullopt_t) noexcept;
template<class T> constexpr bool operator<=(nullopt_t, const optional<T>&) noexcept;
template<class T> constexpr bool operator>=(const optional<T>&, nullopt_t) noexcept;
template<class T> constexpr bool operator>=(nullopt_t, const optional<T>&) noexcept;

// [optional.comp_with_t], comparison with T
template<class T, class U> constexpr bool operator==(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator==(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator!=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator!=(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator<(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator<(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator>(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator>(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator<=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator<=(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);

// [optional.specalg], specialized algorithms
template<class T>
    void swap(optional<T>&, optional<T>&) noexcept(see below);
[...]
```

Change 20.6.3 [optional.optional]:

```

namespace std {
    template<class T>
    class optional {
    public:
        [...]

        // [optional.mod], modifiers
        void reset() noexcept;

        // [optional.relops], relational operators
        template<class U>
        friend constexpr bool operator==(const optional&, const optional<U>&) { see below }
        template<class U>
        friend constexpr bool operator!=(const optional&, const optional<U>&) { see below }
        template<class U>
        friend constexpr bool operator<(const optional&, const optional<U>&) { see below }
        template<class U>
        friend constexpr bool operator>(const optional&, const optional<U>&) { see below }
        template<class U>
        friend constexpr bool operator<=(const optional&, const optional<U>&) { see below }
        template<class U>
        friend constexpr bool operator>=(const optional&, const optional<U>&) { see below }

```

```

template<ThreeWayComparableWith<T> U>
    friend constexpr compare_three_way_result t<T,U>
    operator<=>(const optional&, const optional<U>&).
    { see below }.

// comparison with nullopt
friend constexpr bool operator==(const optional& x, nullopt t) { return !x; }
friend constexpr strong_ordering operator<=>(const optional& x, nullopt t) { return bo

// [optional.comp with t], comparison with T
template<class U> friend constexpr bool operator==(const optional&, const U&) { see be
template<class U> friend constexpr bool operator==(const U&, const optional&) { see be
template<class U> friend constexpr bool operator!=(const optional&, const U&) { see be
template<class U> friend constexpr bool operator!=(const U&, const optional&) { see be
template<class U> friend constexpr bool operator<(const optional&, const U&) { see be
template<class U> friend constexpr bool operator<(const U&, const optional&) { see be
template<class U> friend constexpr bool operator>(const optional&, const U&) { see be
template<class U> friend constexpr bool operator>(const U&, const optional&) { see be
template<class U> friend constexpr bool operator<=(const optional&, const U&) { see be
template<class U> friend constexpr bool operator<=(const U&, const optional&) { see be
template<class U> friend constexpr bool operator>=(const optional&, const U&) { see be
template<class U> friend constexpr bool operator>=(const U&, const optional&) { see be
template<ThreeWayComparableWith<T> U>
    friend constexpr compare_three_way_result t<T,U>
    operator<=>(const optional&, const U&).
    { see below }.

private:
    T *val;          // exposition only
};
}

```

Change 20.6.6 [optional.relops]:

```

template<class T, class U> friend constexpr bool operator==(const optional<T>& x, const op

```

RequiresMandates: The expression `*x == *y` shall be well-formed and its result shall be convertible to `bool`.

[Note: `T` need not be `Cpp17EqualityComparable`. —end note]

Returns: If `bool(x) != bool(y)`, `false`; otherwise if `bool(x) == false`, `true`; otherwise `*x == *y`.

Remarks: Specializations of this function template for which `*x == *y` is a core constant expression shall be constexpr functions. This function is to be found via argument-dependent lookup only.

```

template<class T, class U> friend constexpr bool operator!=(const optional<T>& x, const op

```

RequiresMandates: The expression `*x != *y` shall be well-formed and its result shall be convertible to `bool`.

Returns: If `bool(x) != bool(y)`, `true`; otherwise, if `bool(x) == false`, `false`; otherwise `*x != *y`.

Remarks: Specializations of this function template for which `*x != *y` is a core constant expression shall be constexpr functions. This function is to be found via argument-dependent lookup only.

```

template<class T, class U> friend constexpr bool operator<(const optional<T>& x, const op

```

RequiresMandates: `*x < *y` shall be well-formed and its result shall be convertible to `bool`.

Returns: If `!y`, `false`; otherwise, if `!x`, `true`; otherwise `*x < *y`.

Remarks: Specializations of this function template for which `*x < *y` is a core constant expression shall be constexpr functions. [This function is to be found via argument-dependent lookup only.](#)

```
template<class T, class U> friend constexpr bool operator>(const optional<T>& x, const optional<U>& y);
```

Requires/Mandates: The expression `*x > *y` shall be well-formed and its result shall be convertible to `bool`.

Returns: If `!x`, `false`; otherwise, if `!y`, `true`; otherwise `*x > *y`.

Remarks: Specializations of this function template for which `*x > *y` is a core constant expression shall be constexpr functions. [This function is to be found via argument-dependent lookup only.](#)

```
template<class T, class U> friend constexpr bool operator<=(const optional<T>& x, const optional<U>& y);
```

Requires/Mandates: The expression `*x <= *y` shall be well-formed and its result shall be convertible to `bool`.

Returns: If `!x`, `true`; otherwise, if `!y`, `false`; otherwise `*x <= *y`.

Remarks: Specializations of this function template for which `*x <= *y` is a core constant expression shall be constexpr functions. [This function is to be found via argument-dependent lookup only.](#)

```
template<class T, class U> friend constexpr bool operator>=(const optional<T>& x, const optional<U>& y);
```

Requires/Mandates: The expression `*x >= *y` shall be well-formed and its result shall be convertible to `bool`.

Returns: If `!y`, `true`; otherwise, if `!x`, `false`; otherwise `*x >= *y`.

Remarks: Specializations of this function template for which `*x >= *y` is a core constant expression shall be constexpr functions. This function is to be found via argument-dependent lookup only.

```
template<ThreeWayComparableWith<T> U>
friend constexpr compare_three_way_result_t<T,U>
operator<=>(const optional& x, const optional<U>& y);
```

Returns: If `x && y`, `*x <=> *y`; otherwise `bool(x) <=> bool(y)`.

Remarks: Specializations of this function template for which `*x <=> *y` is a core constant expression shall be `constexpr` functions. [This function is to be found via argument-dependent lookup only.](#)

Remove 20.6.7 [optional.nullopts] (it is now fully expressed by the two hidden friends defined in the header):

```
template<class T> constexpr bool operator==(const optional<T>& x, nullopt_t) noexcept;
template<class T> constexpr bool operator==(nullopt_t, const optional<T>& x) noexcept;
```

Returns: ~~`!x`~~.

```
template<class T> constexpr bool operator!=(const optional<T>& x, nullopt_t) noexcept;
template<class T> constexpr bool operator!=(nullopt_t, const optional<T>& x) noexcept;
```

Returns: ~~`bool(x)`~~ : [...]

Change 20.6.8 [optional.comp_with_t]:

```
template<class T, class U> friend constexpr bool operator==(const optional<T>& x, const optional<U>& y);
```

Requires/Mandates: The expression `*x == v` shall be well-formed and its result shall be convertible to `bool`.

[Note: `T` need not be `Cpp17EqualityComparable`. —end note]

Effects: Equivalent to: `return bool(x) ? *x == v : false;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator==(const T& v, const optional<U>& x);
```

Requires/Mandates: The expression `v == *x` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? v == *x : false;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator!=(const optional<T>& x, const U& v);
```

Requires/Mandates: The expression `*x != v` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? *x != v : true;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator!=(const T& v, const optional<U>& x);
```

Requires/Mandates: The expression `v != *x` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? v != *x : true;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator<(const optional<T>& x, const U& v);
```

Requires/Mandates: The expression `*x < v` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? *x < v : true;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator<(const T& v, const optional<U>& x);
```

Requires/Mandates: The expression `v < *x` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? v < *x : false;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator>(const optional<T>& x, const U& v);
```

Requires/Mandates: The expression `*x > v` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? *x > v : false;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator>(const T& v, const optional<U>& x);
```

Requires/Mandates: The expression `v > *x` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? v > *x : true;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator<=(const optional<T>& x, const U& v);
```

Requires/Mandates: The expression `*x <= v` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? *x <= v : true;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator<=(const TU& v, const optional<U>
```

Requires/Mandates: The expression `v <= *x` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? v <= *x : false;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator>=(const optional<T>& x, const U&
```

Requires/Mandates: The expression `*x >= v` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? *x >= v : false;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T, class U> friend constexpr bool operator>=(const TU& v, const optional<U>
```

Requires/Mandates: The expression `v >= *x` shall be well-formed and its result shall be convertible to `bool`.

Effects: Equivalent to: `return bool(x) ? v >= *x : true;`

Remarks: This function is to be found via argument-dependent lookup only.

```
template<ThreeWayComparableWith<T> U>  
friend constexpr compare_three_way_result t<T,U>  
operator<=>(const optional& x, const U& v);
```

Effects: Equivalent to: `return bool(x) ? *x <=> v : strong_ordering::less;`

Remarks: This function is to be found via argument-dependent lookup only.

Change 20.7.2 [variant.syn]:

```
namespace std {  
    [...]  
    template<class T, class... Types>  
        constexpr add_pointer_t<T>  
            get_if(variant<Types...>*) noexcept;  
    template<class T, class... Types>  
        constexpr add_pointer_t<const T>  
            get_if(const variant<Types...>*) noexcept;  
  
    // [variant.relops], relational operators  
template<class... Types>  
constexpr bool operator==(const variant<Types...>&, const variant<Types...>&);  
template<class... Types>  
constexpr bool operator!=(const variant<Types...>&, const variant<Types...>&);  
template<class... Types>  
constexpr bool operator<(const variant<Types...>&, const variant<Types...>&);  
template<class... Types>  
constexpr bool operator>(const variant<Types...>&, const variant<Types...>&);  
template<class... Types>  
constexpr bool operator<=(const variant<Types...>&, const variant<Types...>&);  
template<class... Types>  
constexpr bool operator>=(const variant<Types...>&, const variant<Types...>&);  
  
    // [variant.visit], visitation
```

```

[...]
```

```

// [variant.monostate], class monostate
struct monostate;

// [variant.monostate.relops], monostate relational operators
constexpr bool operator==(monostate, monostate) noexcept;
constexpr bool operator!=(monostate, monostate) noexcept;
constexpr bool operator<(monostate, monostate) noexcept;
constexpr bool operator>(monostate, monostate) noexcept;
constexpr bool operator<=(monostate, monostate) noexcept;
constexpr bool operator>=(monostate, monostate) noexcept;

// [variant.specalg], specialized algorithms
template<class... Types>
    void swap(variant<Types...>&, variant<Types...>&) noexcept(see below);
[...]
```

Change 20.7.3 [variant.variant]:

```

namespace std {
    template<class... Types>
    class variant {
    public:
        [...]
```

```

        // [variant.swap], swap
        void swap(variant&) noexcept(see below);

        // [variant.relops], relational operators
        friend constexpr bool operator==(const variant&, const variant&) { see below }
        friend constexpr bool operator!=(const variant&, const variant&) { see below }
        friend constexpr bool operator<(const variant&, const variant&) { see below }
        friend constexpr bool operator>(const variant&, const variant&) { see below }
        friend constexpr bool operator<=(const variant&, const variant&) { see below }
        friend constexpr bool operator>=(const variant&, const variant&) { see below }
        friend constexpr common comparison category t<compare three way result t<Types>...>
        operator<=>(const variant&, const variant&)
        requires (ThreeWayComparable<Types> && ...)
        { see below }
    };
}
```

Change 20.7.6 [variant.relops]:

```

template<class... Types>
friend constexpr bool operator==(const variant<Types...>& v, const variant<Types...>& w);
```

Requires/Mandates: `get< i >(v) == get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Returns: If `v.index() != w.index()`, `false`; otherwise if `v.valueless_by_exception()`, `true`; otherwise `get< i >(v) == get< i >(w)` with `i` being `v.index()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class... Types>
friend constexpr bool operator!=(const variant<Types...>& v, const variant<Types...>& w);
```

Requires/Mandates: `get< i >(v) != get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`. **Returns:** If `v.index() != w.index()`, `true`; otherwise if `v.valueless_by_exception()`, `false`; otherwise `get< i >(v) != get< i >(w)` with `i` being `v.index()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class... Types>
friend constexpr bool operator<(const variant<Types...>& v, const variant<Types...>& w);
```

Requires/Mandates: `get< i >(v) < get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Returns: If `w.valueless_by_exception()`, `false`; otherwise if `v.valueless_by_exception()`, `true`; otherwise, if `v.index() < w.index()`, `true`; otherwise if `v.index() > w.index()`, `false`; otherwise `get< i >(v) < get< i >(w)` with `i` being `v.index()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class... Types>
friend constexpr bool operator>(const variant<Types...>& v, const variant<Types...>& w);
```

Requires/Mandates: `get< i >(v) > get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Returns: If `v.valueless_by_exception()`, `false`; otherwise if `w.valueless_by_exception()`, `true`; otherwise, if `v.index() > w.index()`, `true`; otherwise if `v.index() < w.index()`, `false`; otherwise `get< i >(v) > get< i >(w)` with `i` being `v.index()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class... Types>
friend constexpr bool operator<=(const variant<Types...>& v, const variant<Types...>& w);
```

Requires/Mandates: `get< i >(v) <= get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Returns: If `v.valueless_by_exception()`, `true`; otherwise if `w.valueless_by_exception()`, `false`; otherwise, if `v.index() < w.index()`, `true`; otherwise if `v.index() > w.index()`, `false`; otherwise `get< i >(v) <= get< i >(w)` with `i` being `v.index()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class... Types>
friend constexpr bool operator>=(const variant<Types...>& v, const variant<Types...>& w);
```

Requires/Mandates: `get< i >(v) >= get< i >(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

Returns: If `w.valueless_by_exception()`, `true`; otherwise if `v.valueless_by_exception()`, `false`; otherwise, if `v.index() > w.index()`, `true`; otherwise if `v.index() < w.index()`, `false`; otherwise `get< i >(v) >= get< i >(w)` with `i` being `v.index()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
constexpr common_comparison_category_t<compare_three_way_result t<Types>...>
friend operator<=>(const variant& v, const variant& w).
```

```
requires (ThreeWayComparable<Types> && ...);
```

Returns: Let `c` be `(v.index() + 1) <=> (w.index() + 1)`. If `c != 0`, `c`. Otherwise, `get<i>(v) <=> get<i>(w)` with `i` being `v.index()`.

Remarks: This function is to be found via argument-dependent lookup only.

Change 20.7.8 [variant.monostate]:

```
struct monostate{};  
struct monostate {  
    friend constexpr bool operator==(monostate, monostate) noexcept = default;  
    friend constexpr strong_ordering operator<=(monostate, monostate) noexcept = default;  
};
```

[Note: monostate objects have only a single state; they thus always compare equal. —end note]

Remove 20.7.9 [variant.monostate.relops]:

```
constexpr bool operator==(monostate, monostate) noexcept { return true; }  
constexpr bool operator!=(monostate, monostate) noexcept { return false; }  
constexpr bool operator<(monostate, monostate) noexcept { return false; }  
constexpr bool operator>(monostate, monostate) noexcept { return false; }  
constexpr bool operator<=(monostate, monostate) noexcept { return true; }  
constexpr bool operator>=(monostate, monostate) noexcept { return true; }
```

~~[Note: monostate objects have only a single state; they thus always compare equal. —end note]~~

Change 20.9.2 [template.bitset]:

```
namespace std {  
    template<size_t N> class bitset {  
    public:  
        [...]  
        constexpr size_t size() const noexcept;  
        bool operator==(const bitset<N>& rhs) const noexcept;  
        bool operator!=(const bitset<N>& rhs) const noexcept;  
        bool test(size_t pos) const;  
        [...]  
    };  
}
```

Change 20.9.2.2 [bitset.members]:

```
bool operator==(const bitset<N>& rhs) const noexcept;
```

Returns: `true` if the value of each bit in `*this` equals the value of the corresponding bit in `rhs`.

```
bool operator!=(const bitset<N>& rhs) const noexcept;
```

~~Returns: `true` if `!(*this == rhs)`.~~

Change 20.10.2 [memory.syn]:


```

namespace std {
    [...]
    // [default allocator], the default allocator
    template<class T> class allocator;
template<class T, class U>
bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
template<class T, class U>
bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;
    [...]
    template<class T, class D>
        void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;

template<class T1, class D1, class T2, class D2>
bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template<class T1, class D1, class T2, class D2>
bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

template<class T, class D>
bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D>
bool operator==(nullptr_t, const unique_ptr<T, D>& y) noexcept;
template<class T, class D>
bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D>
bool operator!=(nullptr_t, const unique_ptr<T, D>& y) noexcept;
template<class T, class D>
bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
bool operator<(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D>
bool operator>(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
bool operator>(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D>
bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
bool operator<=(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D>
bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
bool operator>=(nullptr_t, const unique_ptr<T, D>& y);

    template<class E, class T, class Y, class D>
        basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);
    [...]
    // [util.smartptr.shared.emp], shared_ptr comparisons
template<class T, class U>
bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

```

```

template<class T, class U>
bool operator!=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
bool operator>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
bool operator<=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
bool operator>=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

template<class T>
bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
bool operator==(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
bool operator!=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
bool operator!=(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
bool operator<(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
bool operator<(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
bool operator>(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
bool operator>(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
bool operator<=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
bool operator<=(nullptr_t, const shared_ptr<T>& y) noexcept;
template<class T>
bool operator>=(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
bool operator>=(nullptr_t, const shared_ptr<T>& y) noexcept;

// [util.smartptr.shared.spec], shared_ptr specialized algorithms
template<class T>
void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;
[...]
```

```

}
```

Change 20.10.10 [default allocator]:

```

namespace std {
    template<class T> class allocator {
    public:
        using value_type      = T;
        using size_type       = size_t;
        using difference_type = ptrdiff_t;
        using propagate_on_container_move_assignment = true_type;
        using is_always_equal = true_type;

        constexpr allocator() noexcept;
        constexpr allocator(const allocator&) noexcept;
        template<class U> constexpr allocator(const allocator<U>&) noexcept;
```

```

~allocator();
allocator& operator=(const allocator&) = default;

[[nodiscard]] T* allocate(size_t n);
void deallocate(T* p, size_t n);

template<class U>
    friend bool operator==(const allocator&, const allocator<U>&) { return true; }
};
}

```

Remove 20.10.10.2 [allocator.globals]:

```

template<class T, class U>
    bool operator==(const allocator<T>&, const allocator<U>&) noexcept;

```

~~Returns: true :~~

```

template<class T, class U>
    bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;

```

~~Returns: false :~~

Change 20.11.1.2 [unique.ptr.single]:

```

namespace std {
    template<class T, class D = default_delete<T>> class unique_ptr {
    public:
        using pointer      = see below;
        using element_type = T;
        using deleter_type = D;
        [...]
        // disable copy from lvalue
        unique_ptr(const unique_ptr&) = delete;
        unique_ptr& operator=(const unique_ptr&) = delete;

        // [unique.ptr.special] Specialized algorithms
        template<class T2, class D2>
            friend bool operator==(const unique_ptr& x, const unique_ptr<T2, D2>& y) { return x.
        template<class T2, class D2>
            friend bool operator<(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            friend bool operator>(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            friend bool operator<=(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            friend bool operator>=(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            requires ThreeWayComparableWith<pointer, typename unique_ptr<T2, D2>::pointer>
            friend auto operator<=>(const unique_ptr& x, const unique_ptr<T2, D2>& y)
            { return compare_three_way().(x.get()., y.get().); }.

        friend bool operator==(const unique_ptr& x, nullptr t) noexcept { return !x; }
        friend auto operator<=>(const unique_ptr& x, nullptr t)
            requires ThreeWayComparableWith<pointer, nullptr t>

```

```

        { return compare_three_way()(x.get(), nullptr); }
    };
}

```

Change 20.11.1.3 [unique.ptr.runtime]:

```

namespace std {
    template class unique_ptr {
    public:
        [...]
        // disable copy from lvalue
        unique_ptr(const unique_ptr&) = delete;
        unique_ptr& operator=(const unique_ptr&) = delete;

        // [unique.ptr.special] Specialized algorithms
        template<class T2, class D2>
            friend bool operator==(const unique_ptr& x, const unique_ptr<T2, D2>& y) { return x
        template<class T2, class D2>
            friend bool operator<(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            friend bool operator>(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            friend bool operator<=(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            friend bool operator>=(const unique_ptr& x, const unique_ptr<T2, D2>& y) { see below
        template<class T2, class D2>
            requires ThreeWayComparableWith<pointer, typename unique_ptr<T2, D2>::pointer>
            friend auto operator<=>(const unique_ptr& x, const unique_ptr<T2, D2>& y)
            { return compare_three_way()(x.get(), y.get()); }

            friend bool operator==(const unique_ptr& x, nullptr t) noexcept { return !x; }
            friend auto operator<=>(const unique_ptr& x, nullptr t)
            requires ThreeWayComparableWith<pointer, nullptr t>
            { return compare_three_way()(x.get(), nullptr); }
    };
}

```

Change 20.11.1.5 [unique.ptr.special]:

```

template<class T, class D> void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;

```

Remarks: This function shall not participate in overload resolution unless `is_swappable_v<D>` is `true`. Effects: Calls `x.swap(y)`.

```

template<class T1, class D1, class T2, class D2>
    bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

```

Returns: ~~`x.get() == y.get()`;~~

```

template<class T1, class D1, class T2, class D2>
    bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

```

Returns: ~~`x.get() != y.get()`;~~

```
template<class T1, class D1, class T2, class D2>
    friend bool operator<(const unique_ptr<T1, D1& x, const unique_ptr<T2, D2>& y);
```

Requires: Let `CT` denote

```
common_type_t<typename unique_ptr<T1, D1>::pointer, typename unique_ptr<T2, D2>::pointer>
```

Then the specialization `less<CT>` shall be a function object type that induces a strict weak ordering on the pointer values.

Returns: `less<CT>()(x.get(), y.get())`.

~~Remarks: If `unique_ptr<T1, D1>::pointer` is not implicitly convertible to `CT` or `unique_ptr<T2, D2>::pointer` is not implicitly convertible to `CT`, the program is ill-formed.~~

Mandates: `pointer` and `unique_ptr<T2, D2>::pointer` are implicitly convertible to `CT`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T1, class D1, class T2, class D2>
    friend bool operator>(const unique_ptr<T1, D1& x, const unique_ptr<T2, D2>& y);
```

Returns: `y < x`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T1, class D1, class T2, class D2>
    friend bool operator<=(const unique_ptr<T1, D1& x, const unique_ptr<T2, D2>& y);
```

Returns: `!(y < x)`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T1, class D1, class T2, class D2>
    friend bool operator>=(const unique_ptr<T1, D1& x, const unique_ptr<T2, D2>& y);
```

Returns: `!(x < y)`.

Remarks: This function is to be found via argument-dependent lookup only.

Change 20.11.3, [util.smartptr.shared]:

```
namespace std {
    template<class T> class shared_ptr {
    [...]
        // [util.smartptr.shared.obs], observers
        element_type* get() const noexcept;
        T& operator*() const noexcept;
        T* operator->() const noexcept;
        element_type& operator[](ptrdiff_t i) const;
        long use_count() const noexcept;
        explicit operator bool() const noexcept;
        template<class U>
            bool owner_before(const shared_ptr<U>& b) const noexcept;
        template<class U>
            bool owner_before(const weak_ptr<U>& b) const noexcept;

        // [util.smartptr.shared.cmp], shared_ptr comparisons
        template<class U>
            friend bool operator==(const shared_ptr& a, const shared_ptr<U>& b) noexcept
            { return a.get() == b.get(); }.
    }
```

```

template<class U>
    friend strong_ordering operator<=(const shared_ptr<U>& a, const shared_ptr<U>& b) noexcept
    { return compare_three_way()(a.get(), b.get()); }

    friend bool operator==(const shared_ptr<U>& a, nullptr_t) noexcept { return !a; }
    friend strong_ordering operator<=(const shared_ptr<U>& a, nullptr_t) noexcept
    { return compare_three_way()(a.get(), nullptr); }
};
}

```

Remove all of 20.11.3.7 [util.smartptr.shared.cmp]:

```

template<class T, class U>
    bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

```

~~Returns: `a.get() == b.get()` :~~

```

template<class T, class U>
    bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

```

~~Returns: `less<>()(a.get(), b.get())` :~~

[...]

Change 20.12.1 [mem.res.syn]:

```

namespace std::pmr {
    // [mem.res.class], class memory_resource
    class memory_resource;

    bool operator==(const memory_resource& a, const memory_resource& b) noexcept;
    bool operator!=(const memory_resource& a, const memory_resource& b) noexcept;

    // [mem.poly.allocator.class], class template polymorphic_allocator
    template<class Tp> class polymorphic_allocator;

    template<class T1, class T2>
        bool operator==(const polymorphic_allocator<T1>& a,
                        const polymorphic_allocator<T2>& b) noexcept;
    template<class T1, class T2>
        bool operator!=(const polymorphic_allocator<T1>& a,
                        const polymorphic_allocator<T2>& b) noexcept;

    // [mem.res.global], global memory resources
    memory_resource* new_delete_resource() noexcept;
    [...]
}

```

Change 20.12.2 [mem.res.class]:

```

namespace std::pmr {
    class memory_resource {
        static constexpr size_t max_align = alignof(max_align_t);    // exposition only

    public:
        memory_resource() = default;
    };
}

```

```

memory_resource(const memory_resource&) = default;
virtual ~memory_resource();

memory_resource& operator=(const memory_resource&) = default;

[[nodiscard]] void* allocate(size_t bytes, size_t alignment = max_align);
void deallocate(void* p, size_t bytes, size_t alignment = max_align);

bool is_equal(const memory_resource& other) const noexcept;

friend bool operator==(const memory_resource&, const memory_resource&) noexcept { see

private:
    virtual void* do_allocate(size_t bytes, size_t alignment) = 0;
    virtual void do_deallocate(void* p, size_t bytes, size_t alignment) = 0;

    virtual bool do_is_equal(const memory_resource& other) const noexcept = 0;
};
}

```

Change 20.12.2.3 [mem.res.eq]:

```
friend bool operator==(const memory_resource& a, const memory_resource& b) noexcept;
```

Returns: `&a == &b || a.is_equal(b)`.

```
bool operator!=(const memory_resource& a, const memory_resource& b) noexcept;
```

~~Returns: `!(a == b)`.~~

Change 20.12.3 [mem.poly allocator.class]:

```

namespace std::pmr {
    template<class Tp> class polymorphic_allocator {
        memory_resource* memory_rsrc;    // exposition only

    public:
        [...]

        memory_resource* resource() const;

        template<class T2>
        friend bool operator==(const polymorphic_allocator& a, const polymorphic_allocator<
        { see below }.
    };
}

```

Change 20.12.3.3 [mem.poly allocator.eq]:

```

template<class T1, class T2>
friend bool operator==(const polymorphic_allocator<T1>& a,
    const polymorphic_allocator<T2>& b) noexcept;

```

Returns: `*a.resource() == *b.resource()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class T1, class T2>
bool operator!=(const polymorphic_allocator<T1>& a,
               const polymorphic_allocator<T2>& b) noexcept;
```

Returns: `!(a == b)`.

Change 20.13.1 [allocator.adaptor.syn]:

```
namespace std {
    // class template scoped allocator adaptor
    template<class OuterAlloc, class... InnerAlloc>
        class scoped_allocator_adaptor;

    // [scoped.adaptor.operators], scoped allocator operators
    template<class OuterA1, class OuterA2, class... InnerAllocs>
    bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                   const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
    template<class OuterA1, class OuterA2, class... InnerAllocs>
    bool operator!=(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                   const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
}
```

```
namespace std {
    template<class OuterAlloc, class... InnerAllocs>
    class scoped_allocator_adaptor : public OuterAlloc {
    [...]
        scoped_allocator_adaptor select_on_container_copy_construction() const;

    // [scoped.adaptor.operators], scoped allocator operators
    template<class Outer2>
    friend bool operator==(const scoped_allocator_adaptor& a,
                          const scoped_allocator_adaptor<Outer2, InnerAllocs...>& b) noexcept;
    { see below }
    };

    template<class OuterAlloc, class... InnerAllocs>
    scoped_allocator_adaptor(OuterAlloc, InnerAllocs...)
        -> scoped_allocator_adaptor<OuterAlloc, InnerAllocs...>;
}
```

Change 20.13.5 [scoped.adaptor.operators]:

```
template<class OuterA1, class OuterA2, class... InnerAllocs>
    friend bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                          const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
```

Returns: If `sizeof...(InnerAllocs)` is zero,

`a.outer_allocator() == b.outer_allocator()`

otherwise

`a.outer_allocator() == b.outer_allocator() && a.inner_allocator() == b.inner_allocator()`

Remarks: This function is to be found via argument-dependent lookup only.


```
template<class OuterA1, class OuterA2, class... InnerAllocs>
bool operator!=(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
               const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
```

~~Returns: `!(a == b)`.~~

Change 20.14.1 [functional.syn]

```
namespace std {
    [...]
    template<class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
    template<class R, class... ArgTypes>
    bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
    template<class R, class... ArgTypes>
    bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
    template<class R, class... ArgTypes>
    bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

    // [func.search], searchers
    [...]
}
```

Change 20.14.8 [range.cmp]/2 to add `<=>`:

There is an implementation-defined strict total ordering over all pointer values of a given type. This total ordering is consistent with the partial order imposed by the builtin operators `<`, `>`, `<=`, **and** `>=`, **and** `<=>`.

Change 20.14.16.2 [func.wrap.func]:

```
namespace std {
    template<class> class function; // not defined

    template<class R, class... ArgTypes>
    class function<R(ArgTypes...)> {
    public:
        using result_type = R;
        [...]

        // [func.wrap.func.targ], function target access
        const type_info& target_type() const noexcept;
        template<class T> T* target() noexcept;
        template<class T> const T* target() const noexcept;

        friend bool operator==(const function& f, nullptr_t) noexcept { return !f; }
    };
    [...]
    // [func.wrap.func.nullptr], Null pointer comparisons
    template<class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

    template<class R, class... ArgTypes>
    bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

    template<class R, class... ArgTypes>
```

```

bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

template<class R, class... ArgTypes>
bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
[...]
```

Remove 20.14.16.2.6 [func.wrap.func.nullptr]:

```

template<class R, class... ArgTypes>
bool operator==(const function<R(ArgTypes...)>& f, nullptr_t) noexcept;
template<class R, class... ArgTypes>
bool operator==(nullptr_t, const function<R(ArgTypes...)>& f) noexcept;
```

Returns: ~~!f~~.

```

template<class R, class... ArgTypes>
bool operator!=(const function<R(ArgTypes...)>& f, nullptr_t) noexcept;
template<class R, class... ArgTypes>
bool operator!=(nullptr_t, const function<R(ArgTypes...)>& f) noexcept;
```

Returns: ~~(bool)f~~.

Add a new row to 20.15.4.3 [meta.unary.prop], the "Type property predicates" table:

Template	Condition	Preconditions
...		
<u>template<class T></u> <u>struct has_strong_structural_equality;</u>	<u>The type <code>T</code> has strong structural equality</u> <u>([class.compare.default]).</u>	<u><code>T</code> shall be a complete type,</u> <u>cv <code>void</code>, or an array of</u> <u>unknown bound.</u>

Change 20.17.2 [type.index.overview]. Note that the relational operators on `type_index` are based on `type_info::before` (effectively `<`). `type_info` *could* provide a three-way ordering function, but does not. Since an important motivation for the existence of `type_index` is to be used as a key in an associative container, we do not want to pessimize `<` - but do want to provide `<=>`.

```

namespace std {
    class type_index {
    public:
        type_index(const type_info& rhs) noexcept;
        bool operator==(const type_index& rhs) const noexcept;
bool operator!=(const type_index& rhs) const noexcept;
        bool operator< (const type_index& rhs) const noexcept;
        bool operator> (const type_index& rhs) const noexcept;
        bool operator<= (const type_index& rhs) const noexcept;
        bool operator>= (const type_index& rhs) const noexcept;
strong_ordering_operator<=>(const type_index& rhs) const noexcept;
        size_t hash_code() const noexcept;
        const char* name() const noexcept;

    private:
```

```

    const type_info* target;    // exposition only
    // Note that the use of a pointer here, rather than a reference,
    // means that the default copy/move constructor and assignment
    // operators will be provided and work as expected.
};
}

```

Change 20.17.3 [type.index.members]:

```

type_index(const type_info& rhs) noexcept;

```

Effects: Constructs a `type_index` object, the equivalent of `target = &rhs`.

```

bool operator==(const type_index& rhs) const noexcept;

```

Returns: `*target == *rhs.target`.

```

bool operator!=(const type_index& rhs) const noexcept;

```

~~*Returns:* `*target != *rhs.target`.~~

```

bool operator<(const type_index& rhs) const noexcept;

```

Returns: `target->before(*rhs.target)`.

```

bool operator>(const type_index& rhs) const noexcept;

```

Returns: `rhs.target->before(*target)`.

```

bool operator<=(const type_index& rhs) const noexcept;

```

Returns: `!rhs.target->before(*target)`.

```

bool operator>=(const type_index& rhs) const noexcept;

```

Returns: `!target->before(*rhs.target)`.

```

strong_ordering_operator<=>(const type_index& rhs) const noexcept;

```

Effects: Equivalent to

```

if (*target == *rhs.target) return strong_ordering::equal;
if (target->before(*rhs.target)) return strong_ordering::less;
return strong_ordering::greater;

```

```

size_t hash_code() const noexcept;

```

Returns: `target->hash_code()`. [...]

Change 20.19.1 [charconv.syn]:

```

namespace std {
    [...]

    // [charconv.to.chars], primitive numerical output conversion
    struct to_chars_result {
        char* ptr;
        errc ec;

        friend bool operator==(const to_chars_result&, const to_chars_result&) = default;
    };

    [...]

    // [charconv.from.chars], primitive numerical input conversion
    struct from_chars_result {
        const char* ptr;
        errc ec;

        friend bool operator==(const from_chars_result&, const from_chars_result&) = default;
    };

    [...]
}

```

§ 4.6. Clause 21: Strings library

Changing the operators for `basic_string` and `basic_string_view` and adding extra type aliases to the `char_traits` specializations provided by the standard.

Change 21.2.3.1 [char.traits.specializations.char]:

```

namespace std {
    template<> struct char_traits<char> {
        using char_type = char;
        using int_type = int;
        using off_type = streamoff;
        using pos_type = streampos;
        using state_type = mbstate_t;
        using comparison_category = strong_ordering;
        [...]
    };
}

```

Change 21.2.3.2 [char.traits.specializations.char8_t]:

```

namespace std {
    template<> struct char_traits<char8_t> {
        using char_type = char8_t;
        using int_type = unsigned int;
        using off_type = streamoff;
        using pos_type = u8streampos;
        using state_type = mbstate_t;
        using comparison_category = strong_ordering;
        [...]
    };
}

```

```
};
}
```

Change 21.2.3.3 [char.traits.specializations.char16_t]:

```
namespace std {
    template<> struct char_traits<char16_t> {
        using char_type    = char16_t;
        using int_type      = uint_least16_t;
        using off_type     = streamoff;
        using pos_type     = u16streampos;
        using state_type   = mbstate_t;
        using comparison_category = strong_ordering;
        [...]
    };
}
```

Change 21.2.3.4 [char.traits.specializations.char32_t]

```
namespace std {
    template<> struct char_traits<char32_t> {
        using char_type    = char32_t;
        using int_type      = uint_least32_t;
        using off_type     = streamoff;
        using pos_type     = u32streampos;
        using state_type   = mbstate_t;
        using comparison_category = strong_ordering;
        [...]
    };
}
```

Change 21.2.3.5 [char.traits.specializations.wchar_t]

```
namespace std {
    template<> struct char_traits<wchar_t> {
        using char_type    = wchar_t;
        using int_type      = wint_t;
        using off_type     = streamoff;
        using pos_type     = wstreampos;
        using state_type   = mbstate_t;
        using comparison_category = strong_ordering;
        [...]
    };
}
```

Change 21.3.1 [string.syn]:

```
#include <initializer_list>

namespace std {
    [...]
    template<class charT, class traits, class Allocator>
    bool operator==(const basic_string<charT, traits, Allocator>& lhs,
                   const basic_string<charT, traits, Allocator>& rhs) noexcept;
    template<class charT, class traits, class Allocator>
```

```

bool operator==(const charT* lhs,
const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator==(const basic_string<charT, traits, Allocator>& lhs,
const charT* rhs);
template<class charT, class traits, class Allocator>
bool operator!=(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator!=(const charT* lhs,
const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator!=(const basic_string<charT, traits, Allocator>& lhs,
const charT* rhs);

template<class charT, class traits, class Allocator>
bool operator< (const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator< (const basic_string<charT, traits, Allocator>& lhs,
const charT* rhs);
template<class charT, class traits, class Allocator>
bool operator< (const charT* lhs,
const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator> (const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator> (const basic_string<charT, traits, Allocator>& lhs,
const charT* rhs);
template<class charT, class traits, class Allocator>
bool operator> (const charT* lhs,
const basic_string<charT, traits, Allocator>& rhs);

template<class charT, class traits, class Allocator>
bool operator<=(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator<=(const basic_string<charT, traits, Allocator>& lhs,
const charT* rhs);
template<class charT, class traits, class Allocator>
bool operator<=(const charT* lhs,
const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator>=(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator>=(const basic_string<charT, traits, Allocator>& lhs,
const charT* rhs);
template<class charT, class traits, class Allocator>
bool operator>=(const charT* lhs,
const basic_string<charT, traits, Allocator>& rhs);

[...]
```

```

}
```

Change 21.3.2 [basic.string]/3. Insert wherever the editor deems appropriate:

```

namespace std {
    template<class charT, class traits = char_traits<charT>,
            class Allocator = allocator<charT>>
    class basic_string {
    [...]
        friend bool operator==(const basic_string& lhs, const basic_string& rhs) { see below }
        friend bool operator==(const basic_string& lhs, const charT* rhs) { see below }
        friend see below operator<=>(const basic_string& lhs, const basic_string& rhs) { see below }
        friend see below operator<=>(const basic_string& lhs, const charT* rhs) { see below }
    [...]
    };
}

```

Change 21.3.3.2 [string.cmp].

```

template<class charT, class traits, class Allocator>
bool operator==(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator==(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator==(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

friend bool operator==(const basic_string& lhs, const basic_string& rhs);
friend bool operator==(const basic_string& lhs, const charT* rhs);

```

```

template<class charT, class traits, class Allocator>
bool operator!=(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator!=(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator!=(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

```

Effects: Equivalent to `return basic_string_view<charT, traits>(lhs) == basic_string_view<charT, traits>(rhs);`

Remarks: This function is to be found via argument-dependent lookup only.

```

template<class charT, class traits, class Allocator>
bool operator<(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator<(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator<(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

template<class charT, class traits, class Allocator>
bool operator>(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator>(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator>(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

template<class charT, class traits, class Allocator>

```

```

bool operator<=(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator<=(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator<=(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

template<class charT, class traits, class Allocator>
bool operator>=(const basic_string<charT, traits, Allocator>& lhs,
const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
bool operator>=(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
bool operator>=(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

friend see below operator<==(const basic_string& lhs, const basic_string& rhs);.
friend see below operator<==(const basic_string& lhs, const charT* rhs);.

```

Effects: ~~Let~~ **op** ~~be the operator~~. Equivalent to:

```
return basic_string_view<charT, traits>(lhs) op <==> basic_string_view<charT, traits>(rhs);
```

Remarks: This function is to be found via argument-dependent lookup only.

Change 21.4.1 [string.view.synop]:

```

namespace std {
    // [string.view.template], class template basic_string_view
    template<class charT, class traits = char_traits<charT>>
    class basic_string_view;

    // [string.view.comparison], non-member comparison functions
template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> x,
basic_string_view<charT, traits> y) noexcept;
template<class charT, class traits>
constexpr bool operator!=(basic_string_view<charT, traits> x,
basic_string_view<charT, traits> y) noexcept;
template<class charT, class traits>
constexpr bool operator< (basic_string_view<charT, traits> x,
basic_string_view<charT, traits> y) noexcept;
template<class charT, class traits>
constexpr bool operator> (basic_string_view<charT, traits> x,
basic_string_view<charT, traits> y) noexcept;
template<class charT, class traits>
constexpr bool operator<=(basic_string_view<charT, traits> x,
basic_string_view<charT, traits> y) noexcept;
template<class charT, class traits>
constexpr bool operator>=(basic_string_view<charT, traits> x,
basic_string_view<charT, traits> y) noexcept;
// see [string.view.comparison], sufficient additional overloads of comparison functions
    [...]
}

```

Change 21.4.2 [string.view.template], insert wherever the editor deems appropriate


```
template<class charT, class traits = char_traits<charT>>
class basic_string_view {
    [...]
    friend constexpr bool operator==(basic_string_view, basic_string_view) noexcept { see below }
    friend constexpr see below operator<=>(basic_string_view, basic_string_view) noexcept {
    [...]
};
```

Remove the entirety of 21.4.3 [string.view.comparison]. The proposed two hidden friend declarations satisfy the requirements without needing extra wording. Replace it with the following:

```
friend constexpr bool operator==(basic_string_view lhs, basic_string_view rhs) noexcept;
```

Returns: `lhs.compare(rhs) == 0`.

Remarks: This function is to be found via argument-dependent lookup only.

```
friend constexpr see below operator<=>(basic_string_view, basic_string_view) noexcept;
```

Let `R` denote the type `traits::comparison_category` if it exists, otherwise `R` is `weak_ordering`.

Returns: `static_cast<R>(lhs.compare(rhs) <=> 0)`.

Remarks: This function is to be found via argument-dependent lookup only.

§ 4.7. Clause 22: Containers library

Change 22.3.2 [array.syn]:

```
#include <initializer_list>

namespace std {
    // [array], class template array
    template<class T, size_t N> struct array;

template<class T, size_t N>
constexpr bool operator==(const array<T, N>& x, const array<T, N>& y);
template<class T, size_t N>
constexpr bool operator!=(const array<T, N>& x, const array<T, N>& y);
template<class T, size_t N>
constexpr bool operator<(const array<T, N>& x, const array<T, N>& y);
template<class T, size_t N>
constexpr bool operator>(const array<T, N>& x, const array<T, N>& y);
template<class T, size_t N>
constexpr bool operator<=(const array<T, N>& x, const array<T, N>& y);
template<class T, size_t N>
constexpr bool operator>=(const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N>
        constexpr void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));
    [...]
}
```

Change 22.3.3 [deque.syn]:

```
#include <initializer_list>
```

```

namespace std {
    // [deque], class template deque
    template<class T, class Allocator = allocator<T>> class deque;

template<class T, class Allocator>
bool operator==(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
template<class T, class Allocator>
bool operator!=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
template<class T, class Allocator>
bool operator<(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
template<class T, class Allocator>
bool operator>(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
template<class T, class Allocator>
bool operator<=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
template<class T, class Allocator>
bool operator>=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);

    template<class T, class Allocator>
    void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
        noexcept(noexcept(x.swap(y)));

    [...]
}

```

Change 22.3.4 [forward_list.syn]:

```

#include <initializer_list>

namespace std {
    // [forwardlist], class template forwardlist
    template<class T, class Allocator = allocator<T>> class forward_list;

template<class T, class Allocator>
bool operator==(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
template<class T, class Allocator>
bool operator!=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
template<class T, class Allocator>
bool operator<(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
template<class T, class Allocator>
bool operator>(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
template<class T, class Allocator>
bool operator<=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
template<class T, class Allocator>
bool operator>=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);

    template<class T, class Allocator>
    void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
        noexcept(noexcept(x.swap(y)));

    [...]
}

```

Change 22.3.5 [list.syn]:

```

#include <initializer_list>

```

```

namespace std {
    // [list], class template list
    template<class T, class Allocator = allocator<T>> class list;

template<class T, class Allocator>
bool operator==(const list<T, Allocator>& x, const list<T, Allocator>& y);
template<class T, class Allocator>
bool operator!=(const list<T, Allocator>& x, const list<T, Allocator>& y);
template<class T, class Allocator>
bool operator<(const list<T, Allocator>& x, const list<T, Allocator>& y);
template<class T, class Allocator>
bool operator>(const list<T, Allocator>& x, const list<T, Allocator>& y);
template<class T, class Allocator>
bool operator<=(const list<T, Allocator>& x, const list<T, Allocator>& y);
template<class T, class Allocator>
bool operator>=(const list<T, Allocator>& x, const list<T, Allocator>& y);

    template<class T, class Allocator>
    void swap(list<T, Allocator>& x, list<T, Allocator>& y)
        noexcept(noexcept(x.swap(y)));

    [...]
}

```

Change 22.3.6 [vector.syn]:

```

#include <initializer_list>

namespace std {
    // [vector], class template vector
    template<class T, class Allocator = allocator<T>> class vector;

template<class T, class Allocator>
bool operator==(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
template<class T, class Allocator>
bool operator!=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
template<class T, class Allocator>
bool operator<(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
template<class T, class Allocator>
bool operator>(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
template<class T, class Allocator>
bool operator<=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
template<class T, class Allocator>
bool operator>=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);

    template<class T, class Allocator>
    void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
        noexcept(noexcept(x.swap(y)));

    [...]
}

```

Add to 22.2.1 [container.requirements.general], paragraph 4:

In Tables 62, 63, and 64 **X** denotes a container class containing objects of type **T**, **a** and **b** denote values of type **X**, **i** and **j** denote values of type (possibly-const) **X::iterator**, **u** denotes an identifier, **r** denotes a

non-const value of type `X`, and `rv` denotes a non-const rvalue of type `X`.

Add a row to Table 62 — Container requirements:

Expression	Return type	Operational semantics	Assertion/note pre/post-condition	Complexity
<code>i <=> j</code>	<code>strong_ordering</code> if <code>X::iterator</code> meets the random access iterator requirements, otherwise <code>strong_equality</code>			<code>constant</code>

Add to 22.2.1 [container.requirements.general], paragraph 7:

In the expressions

```
i == j
i != j
i < j
i <= j
i >= j
i > j
i <=> j
i - j
```

where `i` and `j` denote objects of a container's `iterator` type, either or both may be replaced by an object of the container's `const_iterator` type referring to the same element with no change in semantics.

Remove 22.2.1 [container.requirements.general] table 64 - the optional container operations are now just `<=>` instead of the four relational operators, and will be defined inline following the LWG guidance for `flat_map`.

~~Table 64 lists operations that are provided for some types of containers but not others. Those containers for which the listed operations are provided shall implement the semantics described in Table 64 unless otherwise stated. If the iterators passed to `lexicographical_compare` satisfy the `constexpr` iterator requirements ([iterator.requirements.general]) then the operations described in Table 64 are implemented by `constexpr` functions:~~

Expression	Return type	Operational semantics	Assertion/note pre/post-condition	Complexity
<code>a < b</code>	convertible to <code>bool</code>	<code>lexicographical_compare(</code> <code>—a.begin(), a.end(),</code> <code>—b.begin(), b.end())</code>	<i>Requires:</i> <code><</code> is defined for values of <code>T</code>. <code><</code> is a total ordering relationship.	linear
<code>a > b</code>	convertible to <code>bool</code>	<code>b < a</code>		linear
<code>a <= b</code>	convertible to <code>bool</code>	<code>!(a > b)</code>		linear

$a \geq b$	convertible to <code>bool</code>	$!(a < b)$		linear
------------	-------------------------------------	------------	--	--------

[Note: The algorithm `lexicographical_compare()` is defined in `[algorithms]`. —end note]

Change 22.3.7.1, paragraph 4 [array.overview]:

```
namespace std {
    template<class T, size_t N>
    struct array {
        [...]

        constexpr T *      data() noexcept;
        constexpr const T * data() const noexcept;

        friend constexpr bool operator==(const array&, const array&) = default;
        friend constexpr synth-3way-result<value_type> operator<==(const array& x, const array
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), s
    };

    template<class T, class... U>
        array(T, U...) -> array<T, 1 + sizeof...(U)>;
}
```

Change 22.3.8.1, paragraph 2 [deque.overview]:

```
namespace std {
    template<class T, class Allocator = allocator<T>>
    class deque {
    public:
        [...]
        void swap(deque&)
            noexcept(allocator_traits<Allocator>::is_always_equal::value);
        void clear() noexcept;

        friend bool operator==(const deque& x, const deque& y).
        { return ranges::equal(x, y); }
        friend synth-3way-result<value_type> operator<==(const deque& x, const deque& y).
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), s
    };

    [...]
}
```

Change 22.3.9.1, paragraph 3 [forwardlist.overview]

```
namespace std {
    template<class T, class Allocator = allocator<T>>
    class forward_list {
    public:
        [...]
        void sort();
        template<class Compare> void sort(Compare comp);

        void reverse() noexcept;
```

```

    friend bool operator==(const forward_list& x, const forward_list& y)
    { return ranges::equal(x, y); }
    friend synth-3way-result<value_type> operator<=(const forward_list& x, const forward_list& y)
    { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), s); }
};

[...]
```

Change 22.3.10.1, paragraph 2 [list.overview]

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class list {
    public:
        [...]
        void sort();
        template<class Compare> void sort(Compare comp);

        void reverse() noexcept;

        friend bool operator==(const list& x, const list& y)
        { return ranges::equal(x, y); }
        friend synth-3way-result<value_type> operator<=(const list& x, const list& y)
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), s); }
    };

    [...]
}
```

Change 22.3.11.1, paragraph 2 [vector.overview]

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class vector {
    public:
        [...]
        void swap(vector&)
            noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
                    allocator_traits<Allocator>::is_always_equal::value);
        void clear() noexcept;

        friend bool operator==(const vector& x, const vector& y)
        { return ranges::equal(x, y); }
        friend synth-3way-result<value_type> operator<=(const vector& x, const vector& y)
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), s); }
    };

    [...]
}
```

Change 22.3.12 [vector.bool]:

```

namespace std {
    template<class Allocator>
```

```

class vector<bool, Allocator> {
public:
    [...]
    static void swap(reference x, reference y) noexcept;
    void flip() noexcept;          // flips all bits
    void clear() noexcept;

    friend bool operator==(const vector& x, const vector& y)
    { return ranges::equal(x, y); }
    friend strong_ordering operator<=>(const vector& x, const vector& y)
    { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), c
};
}

```

Change 22.4.2 [associative.map.syn]:

```

#include <initializer_list>

namespace std {
    // [map], class template map
    template<class Key, class T, class Compare = less<Key>,
             class Allocator = allocator<pair<const Key, T>>>
    class map;

template<class Key, class T, class Compare, class Allocator>
bool operator==(const map<Key, T, Compare, Allocator>& x,
                const map<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator!=(const map<Key, T, Compare, Allocator>& x,
                const map<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator< (const map<Key, T, Compare, Allocator>& x,
                const map<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator> (const map<Key, T, Compare, Allocator>& x,
                const map<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator<= (const map<Key, T, Compare, Allocator>& x,
                 const map<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator>= (const map<Key, T, Compare, Allocator>& x,
                 const map<Key, T, Compare, Allocator>& y);

    template<class Key, class T, class Compare, class Allocator>
    void swap(map<Key, T, Compare, Allocator>& x,
              map<Key, T, Compare, Allocator>& y)
        noexcept(noexcept(x.swap(y)));

    template <class Key, class T, class Compare, class Allocator, class Predicate>
    void erase_if(map<Key, T, Compare, Allocator>& c, Predicate pred);

    // [multimap], class template multimap
    template<class Key, class T, class Compare = less<Key>,
             class Allocator = allocator<pair<const Key, T>>>
    class multimap;

```

```

template<class Key, class T, class Compare, class Allocator>
bool operator==(const multimap<Key, T, Compare, Allocator>& x,
const multimap<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator!=(const multimap<Key, T, Compare, Allocator>& x,
const multimap<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator< (const multimap<Key, T, Compare, Allocator>& x,
const multimap<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator> (const multimap<Key, T, Compare, Allocator>& x,
const multimap<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator<= (const multimap<Key, T, Compare, Allocator>& x,
const multimap<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
bool operator>= (const multimap<Key, T, Compare, Allocator>& x,
const multimap<Key, T, Compare, Allocator>& y);

template<class Key, class T, class Compare, class Allocator>
void swap(multimap<Key, T, Compare, Allocator>& x,
multimap<Key, T, Compare, Allocator>& y)
noexcept(noexcept(x.swap(y)));

template <class Key, class T, class Compare, class Allocator, class Predicate>
void erase_if(multimap<Key, T, Compare, Allocator>& c, Predicate pred);

namespace pmr {
    template<class Key, class T, class Compare = less<Key>>
    using map = std::map<Key, T, Compare,
        polymorphic_allocator<pair<const Key, T>>>;

    template<class Key, class T, class Compare = less<Key>>
    using multimap = std::multimap<Key, T, Compare,
        polymorphic_allocator<pair<const Key, T>>>;
}
}

```

Change 22.4.3 [associative.set.syn]:

```

#include <initializer_list>

namespace std {
    // [set], class template set
    template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
    class set;

template<class Key, class Compare, class Allocator>
bool operator==(const set<Key, Compare, Allocator>& x,
const set<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator!=(const set<Key, Compare, Allocator>& x,
const set<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator< (const set<Key, Compare, Allocator>& x,
const set<Key, Compare, Allocator>& y);

```



```

template<class Key, class Compare, class Allocator>
bool operator> (const set<Key, Compare, Allocator>& x,
const set<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator<=(const set<Key, Compare, Allocator>& x,
const set<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator>=(const set<Key, Compare, Allocator>& x,
const set<Key, Compare, Allocator>& y);

template<class Key, class Compare, class Allocator>
void swap(set<Key, Compare, Allocator>& x,
          set<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));

template <class Key, class Compare, class Allocator, class Predicate>
void erase_if(set<Key, Compare, Allocator>& c, Predicate pred);

// [multiset], class template multiset
template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
class multiset;

template<class Key, class Compare, class Allocator>
bool operator==(const multiset<Key, Compare, Allocator>& x,
const multiset<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator!=(const multiset<Key, Compare, Allocator>& x,
const multiset<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator< (const multiset<Key, Compare, Allocator>& x,
const multiset<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator> (const multiset<Key, Compare, Allocator>& x,
const multiset<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator<=(const multiset<Key, Compare, Allocator>& x,
const multiset<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
bool operator>=(const multiset<Key, Compare, Allocator>& x,
const multiset<Key, Compare, Allocator>& y);

template<class Key, class Compare, class Allocator>
void swap(multiset<Key, Compare, Allocator>& x,
          multiset<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));

template <class Key, class Compare, class Allocator, class Predicate>
void erase_if(multiset<Key, Compare, Allocator>& c, Predicate pred);

namespace pmr {
    template<class Key, class Compare = less<Key>>
        using set = std::set<Key, Compare, polymorphic_allocator<Key>>;

    template<class Key, class Compare = less<Key>>
        using multiset = std::multiset<Key, Compare, polymorphic_allocator<Key>>;

```

```

}
}

```

Change 22.4.4.1 [map.overview]:

```

namespace std {
    template<class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
    class map {
    public:
        [...]
        pair<iterator, iterator>          equal_range(const key_type& x);
        pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
        template<class K>
            pair<iterator, iterator>          equal_range(const K& x);
        template<class K>
            pair<const_iterator, const_iterator> equal_range(const K& x) const;

        friend bool operator==(const map& x, const map& y)
        { return ranges::equal(x, y); }
        friend synth-3way-result<value type> operator<==(const map& x, const map& y)
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(),
    };

    [...]
}

```

Change 22.4.5.1 [multimap.overview]:

```

namespace std {
    template<class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
    class multimap {
    public:
        [...]
        pair<iterator, iterator>          equal_range(const key_type& x);
        pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
        template<class K>
            pair<iterator, iterator>          equal_range(const K& x);
        template<class K>
            pair<const_iterator, const_iterator> equal_range(const K& x) const;

        friend bool operator==(const multimap& x, const multimap& y)
        { return ranges::equal(x, y); }
        friend synth-3way-result<value type> operator<==(const multimap& x, const multimap& y)
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(),
    };

    [...]
}

```

Change 22.4.6.1 [set.overview]:

```

namespace std {
    template<class Key, class Compare = less<Key>,
            class Allocator = allocator<Key>>
    class set {
    public:
        [...]
        pair<iterator, iterator>          equal_range(const key_type& x);
        pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
        template<class K>
            pair<iterator, iterator>          equal_range(const K& x);
        template<class K>
            pair<const_iterator, const_iterator> equal_range(const K& x) const;

        friend bool operator==(const set& x, const set& y).
        { return ranges::equal(x, y); }
        friend synth-3way-result<value type> operator<=(const set& x, const set& y).
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), s
    };

    [...]
}

```

Change 22.4.7.1 [multiset.overview]:

```

namespace std {
    template<class Key, class Compare = less<Key>,
            class Allocator = allocator<Key>>
    class multiset {
    public:
        [...]

        pair<iterator, iterator>          equal_range(const key_type& x);
        pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
        template<class K>
            pair<iterator, iterator>          equal_range(const K& x);
        template<class K>
            pair<const_iterator, const_iterator> equal_range(const K& x) const;

        friend bool operator==(const multiset& x, const multiset& y).
        { return ranges::equal(x, y); }
        friend synth-3way-result<value type> operator<=(const multiset& x, const multiset& y).
        { return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(), s
    };

    [...]
}

```

Change 22.5.2 [unord.map.syn]:

```

#include <initializer_list>

namespace std {
    // [unord.map], class template unordered_map
    template<class Key,
            class T,

```

```

        class Hash = hash<Key>,
        class Pred = equal_to<Key>,
        class Alloc = allocator<pair<const Key, T>>>
class unordered_map;

// [unord.multimap], class template unordered_multimap
template<class Key,
        class T,
        class Hash = hash<Key>,
        class Pred = equal_to<Key>,
        class Alloc = allocator<pair<const Key, T>>>
class unordered_multimap;

template<class Key, class T, class Hash, class Pred, class Alloc>
bool operator==(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
                const unordered_map<Key, T, Hash, Pred, Alloc>& b);
template<class Key, class T, class Hash, class Pred, class Alloc>
bool operator!=(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
                const unordered_map<Key, T, Hash, Pred, Alloc>& b);

template<class Key, class T, class Hash, class Pred, class Alloc>
bool operator==(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
                const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);
template<class Key, class T, class Hash, class Pred, class Alloc>
bool operator!=(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
                const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);

template<class Key, class T, class Hash, class Pred, class Alloc>
void swap(unordered_map<Key, T, Hash, Pred, Alloc>& x,
         unordered_map<Key, T, Hash, Pred, Alloc>& y)
    noexcept(noexcept(x.swap(y)));

[...]
```

```

}
```

Change 22.5.3 [unord.set.syn]:

```

#include <initializer_list>

namespace std {
    // [unord.set], class template unordered_set
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Alloc = allocator<Key>>
    class unordered_set;

    // [unord.multiset], class template unordered_multiset
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Alloc = allocator<Key>>
    class unordered_multiset;

template<class Key, class Hash, class Pred, class Alloc>
bool operator==(const unordered_set<Key, Hash, Pred, Alloc>& a,
```

```

template<class Key, class Hash, class Pred, class Alloc>
bool operator!=(const unordered_set<Key, Hash, Pred, Alloc>& a,
const unordered_set<Key, Hash, Pred, Alloc>& b);

template<class Key, class Hash, class Pred, class Alloc>
bool operator==(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
const unordered_multiset<Key, Hash, Pred, Alloc>& b);

template<class Key, class Hash, class Pred, class Alloc>
bool operator!=(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
const unordered_multiset<Key, Hash, Pred, Alloc>& b);

template<class Key, class Hash, class Pred, class Alloc>
void swap(unordered_set<Key, Hash, Pred, Alloc>& x,
          unordered_set<Key, Hash, Pred, Alloc>& y)
    noexcept(noexcept(x.swap(y)));

[...]
```

Change 22.5.4.1 [unord.map.overview]:

```

namespace std {
    template<class Key,
              class T,
              class Hash = hash<Key>,
              class Pred = equal_to<Key>,
              class Allocator = allocator<pair<const Key, T>>>
    class unordered_map {
    public:
        [...]
        // hash policy
        float load_factor() const noexcept;
        float max_load_factor() const noexcept;
        void max_load_factor(float z);
        void rehash(size_type n);
        void reserve(size_type n);

        friend bool operator==(const unordered_map& x, const unordered_map& y);
    };

    [...]
}
```

Change 22.5.5.1 [unord.multimap.overview]:

```

namespace std {
    template<class Key,
              class T,
              class Hash = hash<Key>,
              class Pred = equal_to<Key>,
              class Allocator = allocator<pair<const Key, T>>>
    class unordered_multimap {
    public:
        [...]
        // hash policy
```

```

float load_factor() const noexcept;
float max_load_factor() const noexcept;
void max_load_factor(float z);
void rehash(size_type n);
void reserve(size_type n);

    friend bool operator==(const unordered_multimap& x, const unordered_multimap& y);
};

[...]
```

Change 22.5.6.1 [unord.set.overview]:

```

namespace std {
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<Key>>
    class unordered_set {
    public:
        [...]
        // hash policy
        float load_factor() const noexcept;
        float max_load_factor() const noexcept;
        void max_load_factor(float z);
        void rehash(size_type n);
        void reserve(size_type n);

        friend bool operator==(const unordered_set& x, const unordered_set& y);
    };

    [...]
}
```

Change 22.5.7.1 [unord.multiset.overview]:

```

namespace std {
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<Key>>
    class unordered_multiset {
    public:
        [...]
        // hash policy
        float load_factor() const noexcept;
        float max_load_factor() const noexcept;
        void max_load_factor(float z);
        void rehash(size_type n);
        void reserve(size_type n);

        friend bool operator==(const unordered_multiset& x, const unordered_multiset& y);
    };
};
```

```
[...]  
}
```

Change 22.6.2 [queue.syn]:

```
#include <initializer_list>  
  
namespace std {  
    template<class T, class Container = deque<T>> class queue;  
    template<class T, class Container = vector<T>,  
            class Compare = less<typename Container::value_type>>  
        class priority_queue;  
  
template<class T, class Container>  
bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);  
template<class T, class Container>  
bool operator!=(const queue<T, Container>& x, const queue<T, Container>& y);  
template<class T, class Container>  
bool operator< (const queue<T, Container>& x, const queue<T, Container>& y);  
template<class T, class Container>  
bool operator> (const queue<T, Container>& x, const queue<T, Container>& y);  
template<class T, class Container>  
bool operator<= (const queue<T, Container>& x, const queue<T, Container>& y);  
template<class T, class Container>  
bool operator>= (const queue<T, Container>& x, const queue<T, Container>& y);  
  
    template<class T, class Container>  
        void swap(queue<T, Container>& x, queue<T, Container>& y) noexcept(noexcept(x.swap(y)));  
    template<class T, class Container, class Compare>  
        void swap(priority_queue<T, Container, Compare>& x,  
                  priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));  
}
```

Change 22.6.3 [stack.syn]:

```
#include <initializer_list>  
  
namespace std {  
    template<class T, class Container = deque<T>> class stack;  
  
template<class T, class Container>  
bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);  
template<class T, class Container>  
bool operator!=(const stack<T, Container>& x, const stack<T, Container>& y);  
template<class T, class Container>  
bool operator< (const stack<T, Container>& x, const stack<T, Container>& y);  
template<class T, class Container>  
bool operator> (const stack<T, Container>& x, const stack<T, Container>& y);  
template<class T, class Container>  
bool operator<= (const stack<T, Container>& x, const stack<T, Container>& y);  
template<class T, class Container>  
bool operator>= (const stack<T, Container>& x, const stack<T, Container>& y);  
  
    template<class T, class Container>
```

```

    void swap(stack<T, Container>& x, stack<T, Container>& y) noexcept(noexcept(x.swap(y)))
}

```

Change 22.6.4.1 [queue.defn]:

```

namespace std {
    template<class T, class Container = deque<T>>
    class queue {
    public:
        using value_type      = typename Container::value_type;
        using reference        = typename Container::reference;
        using const_reference  = typename Container::const_reference;
        using size_type        = typename Container::size_type;
        using container_type   = Container;

    protected:
        Container c;

    public:
        queue() : queue(Container()) {}
        explicit queue(const Container&);
        explicit queue(Container&&);
        template<class Alloc> explicit queue(const Alloc&);
        template<class Alloc> queue(const Container&, const Alloc&);
        template<class Alloc> queue(Container&&, const Alloc&);
        template<class Alloc> queue(const queue&, const Alloc&);
        template<class Alloc> queue(queue&&, const Alloc&);

        [[nodiscard]] bool empty() const { return c.empty(); }
        size_type size() const { return c.size(); }
        reference front() { return c.front(); }
        const_reference front() const { return c.front(); }
        reference back() { return c.back(); }
        const_reference back() const { return c.back(); }
        void push(const value_type& x) { c.push_back(x); }
        void push(value_type&& x) { c.push_back(std::move(x)); }
        template<class... Args>
            decltype(auto) emplace(Args&&... args)
            { return c.emplace_back(std::forward<Args>(args)...); }
        void pop() { c.pop_front(); }
        void swap(queue& q) noexcept(is_nothrow_swappable_v<Container>)
            { using std::swap; swap(c, q.c); }

        friend bool operator==(const queue& x, const queue& y)
        { return x.c == y.c; }
        friend bool operator!=(const queue& x, const queue& y)
        { return x.c != y.c; }
        friend bool operator<(const queue& x, const queue& y)
        { return x.c < y.c; }
        friend bool operator>(const queue& x, const queue& y)
        { return x.c > y.c; }
        friend bool operator<=(const queue& x, const queue& y)
        { return x.c <= y.c; }
        friend bool operator>=(const queue& x, const queue& y)
        { return x.c >= y.c; }
        friend auto operator<=>(const queue& x, const queue& y)

```



```

    requires ThreeWayComparable<Container>
    { return x.c <=> y.c; }
};
[...]
```

Remove 22.6.4.4 [queue.ops] (as we've now defined them all inline in the header):

```

template<class T, class Container>
bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);

```

~~Returns:~~ `x.c == y.c` :

[...]

Change 22.6.6.1 [stack.defn]:

```

namespace std {
    template<class T, class Container = deque<T>>
    class stack {
    public:
        using value_type      = typename Container::value_type;
        using reference        = typename Container::reference;
        using const_reference  = typename Container::const_reference;
        using size_type        = typename Container::size_type;
        using container_type    = Container;

    protected:
        Container c;

    public:
        stack() : stack(Container()) {}
        explicit stack(const Container&);
        explicit stack(Container&&);
        template<class Alloc> explicit stack(const Alloc&);
        template<class Alloc> stack(const Container&, const Alloc&);
        template<class Alloc> stack(Container&&, const Alloc&);
        template<class Alloc> stack(const stack&, const Alloc&);
        template<class Alloc> stack(stack&&, const Alloc&);

        [[nodiscard]] bool empty() const    { return c.empty(); }
        size_type size() const              { return c.size(); }
        reference top()                     { return c.back(); }
        const_reference top() const         { return c.back(); }
        void push(const value_type& x)      { c.push_back(x); }
        void push(value_type&& x)           { c.push_back(std::move(x)); }
        template<class... Args>
            decltype(auto) emplace(Args&&... args)
            { return c.emplace_back(std::forward<Args>(args)...); }
        void pop()                          { c.pop_back(); }
        void swap(stack& s) noexcept(is_nothrow_swappable_v<Container>)
            { using std::swap; swap(c, s.c); }

        friend bool operator==(const stack& x, const stack& y).
        { return x.c == y.c; }
        friend bool operator!=(const stack& x, const stack& y).
        { return x.c != y.c; }
    };
}

```

```

    friend bool operator< (const stack& x, const stack& y).
    _{ return x.c < y.c; }.
    friend bool operator> (const stack& x, const stack& y).
    _{ return x.c > y.c; }.
    friend bool operator<=(const stack& x, const stack& y).
    _{ return x.c <= y.c; }.
    friend bool operator>=(const stack& x, const stack& y).
    _{ return x.c >= y.c; }.
    friend auto operator<=>(const stack& x, const stack& y).
    _requires ThreeWayComparable<Container>
    _{ return x.c <=> y.c; }.
};

[ ... ]
}

```

Remove 22.6.6.4 [stack.ops] (as we've now defined them all inline in the header):

```

template<class T, class Container>
bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);

```

Returns: `x.c == y.c` :

[...]

§ 4.8. Clause 23: Iterators library

Change 23.2 [iterator.synopsis]:

```

#include <concepts>

namespace std {
    [...]
    // [predef.iterators], predefined iterators and sentinels
    // [reverse.iterators], reverse iterators
    template<class Iterator> class reverse_iterator;

template<class Iterator1, class Iterator2>
constexpr bool operator==(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator!=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator<=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator>=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator<=>(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);

```

```

const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator==(
const reverse_iterator<Iterator1>& x,
const reverse_iterator<Iterator2>& y);
[...]
```

// [move.iterators], move iterators and sentinels

```

template<class Iterator> class move_iterator;

template<class Iterator1, class Iterator2>
constexpr bool operator==(
const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator!=(
const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator<(
const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator>(
const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator<=(
const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
constexpr bool operator>=(
const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
[...]
```

// [stream.iterators], stream iterators

```

template<class T, class charT = char, class traits = char_traits<charT>,
        class Distance = ptrdiff_t>
class istream_iterator;

template<class T, class charT, class traits, class Distance>
bool operator==(const istream_iterator<T, charT, traits, Distance>& x,
const istream_iterator<T, charT, traits, Distance>& y);
template<class T, class charT, class traits, class Distance>
bool operator!=(const istream_iterator<T, charT, traits, Distance>& x,
const istream_iterator<T, charT, traits, Distance>& y);

template<class T, class charT = char, class traits = char_traits<charT>>
class ostream_iterator;

template<class charT, class traits = char_traits<charT>>
class istreambuf_iterator;

template<class charT, class traits>
bool operator==(const istreambuf_iterator<charT, traits>& a,
const istreambuf_iterator<charT, traits>& b);
template<class charT, class traits>
bool operator!=(const istreambuf_iterator<charT, traits>& a,
const istreambuf_iterator<charT, traits>& b);

template<class charT, class traits = char_traits<charT>>
class ostreambuf_iterator;

```

```
[...]  
}
```

Change 23.5.1.1 [reverse.iterator]:

```
namespace std {  
    template<class Iterator>  
    class reverse_iterator {  
    public:  
        [...]  
        template<IndirectlySwappable<Iterator> Iterator2>  
        friend constexpr void  
            iter_swap(const reverse_iterator& x,  
                    const reverse_iterator<Iterator2>& y) noexcept(see below);  
  
        // [reverse.iter.cmp] Comparisons  
        template<class Iterator2>  
        friend constexpr bool operator==(const reverse_iterator& x,  
                                        const reverse_iterator<Iterator2>& y)  
        { see below }.  
        template<class Iterator2>  
        friend constexpr bool operator!=(const reverse_iterator& x,  
                                        const reverse_iterator<Iterator2>& y)  
        { see below }.  
        template<class Iterator2>  
        friend constexpr bool operator< (const reverse_iterator& x,  
                                        const reverse_iterator<Iterator2>& y)  
        { see below }.  
        template<class Iterator2>  
        friend constexpr bool operator> (const reverse_iterator& x,  
                                        const reverse_iterator<Iterator2>& y)  
        { see below }.  
        template<class Iterator2>  
        friend constexpr bool operator<=(const reverse_iterator& x,  
                                        const reverse_iterator<Iterator2>& y)  
        { see below }.  
        template<class Iterator2>  
        friend constexpr bool operator>=(const reverse_iterator& x,  
                                        const reverse_iterator<Iterator2>& y)  
        { see below }.  
        template<ThreeWayComparableWith<Iterator> Iterator2>  
        friend constexpr compare_three_way_result_t<Iterator, Iterator2>  
        operator<=>(const reverse_iterator& x,  
                      const reverse_iterator<Iterator2>& y)  
        { see below }.  
  
    protected:  
        Iterator current;  
    };  
}
```

Change 23.5.1.7 [reverse.iter.cmp]:

```
template<class Iterator1, class Iterator2>  
    friend constexpr bool operator==(
```

```
const reverse_iterator<Iterator1>& x,  
const reverse_iterator<Iterator2>& y);
```

Constraints: `x.current == y.current` is well-formed and convertible to `bool`.

Returns: `x.current == y.current`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>  
friend constexpr bool operator!=(  
    const reverse_iterator<Iterator1>& x,  
    const reverse_iterator<Iterator2>& y);
```

Constraints: `x.current != y.current` is well-formed and convertible to `bool`.

Returns: `x.current != y.current`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>  
friend constexpr bool operator<(  
    const reverse_iterator<Iterator1>& x,  
    const reverse_iterator<Iterator2>& y);
```

Constraints: `x.current > y.current` is well-formed and convertible to `bool`.

Returns: `x.current > y.current`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>  
friend constexpr bool operator>(  
    const reverse_iterator<Iterator1>& x,  
    const reverse_iterator<Iterator2>& y);
```

Constraints: `x.current < y.current` is well-formed and convertible to `bool`.

Returns: `x.current < y.current`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>  
friend constexpr bool operator<=(  
    const reverse_iterator<Iterator1>& x,  
    const reverse_iterator<Iterator2>& y);
```

Constraints: `x.current >= y.current` is well-formed and convertible to `bool`.

Returns: `x.current >= y.current`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>  
friend constexpr bool operator>=(  
    const reverse_iterator<Iterator1>& x,  
    const reverse_iterator<Iterator2>& y);
```

Constraints: `x.current <= y.current` is well-formed and convertible to `bool`.

Returns: `x.current <= y.current`.

Remarks: This function is to be found via argument-dependent lookup only.

```

template<ThreeWayComparableWith<Iterator> Iterator2>
    friend constexpr compare_three_way_result t<Iterator, Iterator2> operator<=>(
        const reverse_iterator& x,
        const reverse_iterator<Iterator2>& y);

```

Returns: `y.current <=> x.current`.

Remarks: This function is more constrained than ([temp.constr.order]) each of the other relational operator function templates. This function is to be found via argument-dependent lookup only.

Change 23.5.3.1 [move.iterator]:

```

namespace std {
    template<class Iterator>
    class move_iterator {
    [...]
    // [move.iter.op.comp] Comparisons
    template<class Iterator2>
        friend constexpr bool operator==(const move_iterator& x, const move_iterator<Iterator2>& y)
        { see below };
    template<class Iterator2>
        friend constexpr bool operator<(const move_iterator& x, const move_iterator<Iterator2>& y)
        { see below };
    template<class Iterator2>
        friend constexpr bool operator>(const move_iterator& x, const move_iterator<Iterator2>& y)
        { see below };
    template<class Iterator2>
        friend constexpr bool operator<=(const move_iterator& x, const move_iterator<Iterator2>& y)
        { see below };
    template<class Iterator2>
        friend constexpr bool operator>=(const move_iterator& x, const move_iterator<Iterator2>& y)
        { see below };
    template<ThreeWayComparableWith<Iterator> Iterator2>
        friend constexpr compare_three_way_result t<Iterator, Iterator2>
        operator<=>(const move_iterator& x, const move_iterator<Iterator2>& y)
        { see below };

    template<Sentinel<Iterator> S>
        friend constexpr bool
        operator==(const move_iterator& x, const move_sentinel<S>& y);
template<Sentinel<Iterator> S>
friend constexpr bool
operator==(const move_sentinel<S>& x, const move_iterator& y);
template<Sentinel<Iterator> S>
friend constexpr bool
operator!=(const move_iterator& x, const move_sentinel<S>& y);
template<Sentinel<Iterator> S>
friend constexpr bool
operator!=(const move_sentinel<S>& x, const move_iterator& y);
    template<SizedSentinel<Iterator> S>
        friend constexpr iter_difference_t<Iterator>
        operator-(const move_sentinel<S>& x, const move_iterator& y);
    [...]
private:
    Iterator current;    // exposition only

```

```
};
}
```

Change 23.5.3.7 [move.iter.op.comp]:

```
template<class Iterator1, class Iterator2>
    friend constexpr bool operator==(const move_iterator<Iterator1>& x,
                                     const move_iterator<Iterator2>& y);
template<Sentinel<Iterator> S>
    friend constexpr bool operator==(const move_iterator& x,
                                     const move_sentinel<S>& y);
template<Sentinel<Iterator> S>
    friend constexpr bool operator==(const move_sentinel<S>& x,
                                     const move_iterator& y);
```

Constraints: `x.base() == y.base()` is well-formed and convertible to `bool`.

Returns: `x.base() == y.base()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>
    constexpr bool operator!=(const move_iterator<Iterator1>& x,
                              const move_iterator<Iterator2>& y);
template<Sentinel<Iterator> S>
    friend constexpr bool operator!=(const move_iterator& x,
                                     const move_sentinel<S>& y);
template<Sentinel<Iterator> S>
    friend constexpr bool operator!=(const move_sentinel<S>& x,
                                     const move_iterator& y);
```

~~Constraints: `x.base() == y.base()` is well-formed and convertible to `bool`.~~

~~Returns: `!(x == y)`.~~

```
template<class Iterator1, class Iterator2>
    friend constexpr bool operator<(const move_iterator<Iterator1>& x, const move_iterator<It
```

Constraints: `x.base() < y.base()` is well-formed and convertible to `bool`.

Returns: `x.base() < y.base()`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>
    friend constexpr bool operator>(const move_iterator<Iterator1>& x, const move_iterator<It
```

Constraints: `y.base() < x.base()` is well-formed and convertible to `bool`.

Returns: `y < x`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>
    friend constexpr bool operator<=(const move_iterator<Iterator1>& x, const move_iterator<It
```

Constraints: `y.base() < x.base()` is well-formed and convertible to `bool`.

Returns: `!(y < x)`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<class Iterator1, class Iterator2>
friend constexpr bool operator==(const move_iterator<Iterator1>& x, const move_iterator<It
```

Constraints: `x.base() < y.base()` is well-formed and convertible to `bool`.

Returns: `!(x < y)`.

Remarks: This function is to be found via argument-dependent lookup only.

```
template<ThreeWayComparableWith<Iterator> Iterator2>
friend constexpr compare_three_way_result t<Iterator, Iterator2> operator<==>(.
const move_iterator& x,
const move_iterator<Iterator2>& y);
```

Returns: `x.base() <==> y.base()`.

Remarks: This function is more constrained than ([temp.constr.order]) each of the other relational operator function templates. This function is to be found via argument-dependent lookup only.

Remove the `operator!=` from 23.5.4.1 [common.iterator]:

```
namespace std {
template<Iterator I, Sentinel<I> S>
requires (!Same<I, S>)
class common_iterator {
public:
[...]
```

```
template<class I2, Sentinel<I> S2>
requires Sentinel<S, I2>
friend bool operator==(
const common_iterator& x, const common_iterator<I2, S2>& y);
template<class I2, Sentinel<I> S2>
requires Sentinel<S, I2> && EqualityComparableWith<I, I2>
friend bool operator==(
const common_iterator& x, const common_iterator<I2, S2>& y);
template<class I2, Sentinel<I> S2>
requires Sentinel<S, I2>
friend bool operator!=(
const common_iterator& x, const common_iterator<I2, S2>& y);
[...]
```

```
private:
variant<I, S> v_; // exposition only
};
[...]
```

Remove the `operator!=` from 23.5.4.6 [common.iter.cmp]:

```
template<class I2, Sentinel<I> S2>
requires Sentinel<S, I2>
friend bool operator!=(
const common_iterator& x, const common_iterator<I2, S2>& y);
```

Effects: Equivalent to: `return !(x == y);`

Change 23.5.6.1 [counted.iterator]:


```

namespace std {
    template<Iterator I>
    class counted_iterator {
    public:
        using iterator_type = I;
        [...]
        template<Common<I> I2>
            friend constexpr bool operator==(
                const counted_iterator& x, const counted_iterator<I2>& y);
        friend constexpr bool operator==(
            const counted_iterator& x, default_sentinel_t);
friend constexpr bool operator==(
    default_sentinel_t, const counted_iterator& x);

template<Common<I> I2>
friend constexpr bool operator!=(
    const counted_iterator& x, const counted_iterator<I2>& y);
friend constexpr bool operator!=(
    const counted_iterator& x, default_sentinel_t y);
friend constexpr bool operator!=(
    default_sentinel_t x, const counted_iterator& y);

        template<Common<I> I2>
            friend constexpr bool operator<(
                const counted_iterator& x, const counted_iterator<I2>& y);
        template<Common<I> I2>
            friend constexpr bool operator>(
                const counted_iterator& x, const counted_iterator<I2>& y);
        template<Common<I> I2>
            friend constexpr bool operator<=(
                const counted_iterator& x, const counted_iterator<I2>& y);
        template<Common<I> I2>
            friend constexpr bool operator>=(
                const counted_iterator& x, const counted_iterator<I2>& y);
template<Common<I> I2> requires ThreeWayComparableWith<I, I2>
friend constexpr compare_three_way_result t<I, I2> operator<==(
    const counted_iterator& x, const counted_iterator<I2>& y);

        friend constexpr iter_rvalue_reference_t<I> iter_move(const counted_iterator& i)
            noexcept(noexcept(ranges::iter_move(i.current)))
            requires InputIterator<I>;
        template<IndirectlySwappable<I> I2>
            friend constexpr void iter_swap(const counted_iterator& x, const counted_iterator<I2>& y)
            noexcept(noexcept(ranges::iter_swap(x.current, y.current)));

    private:
        I current = I(); // exposition only
        iter_difference_t<I> length = 0; // exposition only
    };
    [...]
}

```

Change 23.5.6.6 [counted.iter.cmp]:

```

template<Common<I> I2>
    friend constexpr bool operator==(

```

```
const counted_iterator& x, const counted_iterator<I2>& y);
```

Expects: `x` and `y` refer to elements of the same sequence ([counted.iterator]).

Effects: Equivalent to: `return x.length == y.length;`

```
friend constexpr bool operator==(
    const counted_iterator& x, default_sentinel_t);
friend constexpr bool operator==(
    default_sentinel_t, const counted_iterator& x);
```

Effects: Equivalent to: `return x.length == 0;`

```
template<Common<I> I2>
friend constexpr bool operator!=(
    const counted_iterator& x, const counted_iterator<I2>& y);
friend constexpr bool operator!=(
    const counted_iterator& x, default_sentinel_t y);
friend constexpr bool operator!=(
    default_sentinel_t x, const counted_iterator& y);
```

Effects: Equivalent to: ~~`return !(x == y);`~~

```
template<Common<I> I2>
friend constexpr bool operator<(
    const counted_iterator& x, const counted_iterator<I2>& y);
```

Expects: `x` and `y` refer to elements of the same sequence ([counted.iterator]).

Effects: Equivalent to: `return y.length < x.length;`

[Note: The argument order in the *Effects*: element is reversed because length counts down, not up. —end note]

```
template<Common<I> I2>
friend constexpr bool operator>(
    const counted_iterator& x, const counted_iterator<I2>& y);
```

Effects: Equivalent to: `return y < x;`

```
template<Common<I> I2>
friend constexpr bool operator<=(
    const counted_iterator& x, const counted_iterator<I2>& y);
```

Effects: Equivalent to: `return !(y < x);`

```
template<Common<I> I2>
friend constexpr bool operator>=(
    const counted_iterator& x, const counted_iterator<I2>& y);
```

Effects: Equivalent to: `return !(x < y);`

```
template<Common<I> I2> requires ThreeWayComparableWith<I, I2>
friend constexpr compare_three_way_result t<I, I2> operator<==(<
    const counted_iterator& x, const counted_iterator<I2>& y);
```

Effects: Equivalent to: `return y <=> x;`

Change 23.5.7.1 [unreachable.sentinel]:

```
namespace std {
    struct unreachable_sentinel_t {
        template<WeaklyIncrementable I>
            friend constexpr bool operator==(unreachable_sentinel_t, const I&) noexcept;
        { return true; };
        template<WeaklyIncrementable I>
            friend constexpr bool operator==(const I&, unreachable_sentinel_t) noexcept;
        template<WeaklyIncrementable I>
            friend constexpr bool operator!=(unreachable_sentinel_t, const I&) noexcept;
        template<WeaklyIncrementable I>
            friend constexpr bool operator!=(const I&, unreachable_sentinel_t) noexcept;
    };
}
```

Remove 23.5.7.2 [unreachable.sentinel.cmp] (as it's now entirely defined in the synopsis):

```
template<WeaklyIncrementable I>
    friend constexpr bool operator==(unreachable_sentinel_t, const I&) noexcept;
template<WeaklyIncrementable I>
    friend constexpr bool operator==(const I&, unreachable_sentinel_t) noexcept;
```

Returns: `false` :

```
template<WeaklyIncrementable I>
    friend constexpr bool operator!=(unreachable_sentinel_t, const I&) noexcept;
template<WeaklyIncrementable I>
    friend constexpr bool operator!=(const I&, unreachable_sentinel_t) noexcept;
```

Returns: `true` :

Change 23.6.1 [istream.iterator]:

```
namespace std {
    template<class T, class charT = char, class traits = char_traits<charT>,
            class Distance = ptrdiff_t>
    class istream_iterator {
    public:
        using iterator_category = input_iterator_tag;
        using value_type = T;
        using difference_type = Distance;
        using pointer = const T*;
        using reference = const T&;
        using char_type = charT;
        using traits_type = traits;
        using istream_type = basic_istream<charT, traits>;

        constexpr istream_iterator();
        constexpr istream_iterator(default_sentinel_t);
        istream_iterator(istream_type& s);
        istream_iterator(const istream_iterator& x) = default;
        ~istream_iterator() = default;
```

```

istream_iterator& operator=(const istream_iterator&) = default;

const T& operator*() const;
const T* operator->() const;
istream_iterator& operator++();
istream_iterator operator++(int);

friend bool operator==(const istream_iterator& i, default_sentinel_t)÷
{ return !i.in_stream; }
friend bool operator==(const istream_iterator& x, const istream_iterator& y).
{ return x.in_stream == y.in_stream; }
friend bool operator==(default_sentinel_t, const istream_iterator& i);
friend bool operator!=(const istream_iterator& x, default_sentinel_t y);
friend bool operator!=(default_sentinel_t x, const istream_iterator& y);

private:
    basic_istream<charT, traits>* in_stream; // exposition only
    T value;                                // exposition only
};
}

```

Remove the specifications for `operator==` and `operator!=` from 23.6.1.2 [istream.iterator.ops]:

```

template<class T, class charT, class traits, class Distance>
bool operator==(const istream_iterator<T, charT, traits, Distance>& x,
const istream_iterator<T, charT, traits, Distance>& y);

```

~~Returns:~~ `x.in_stream == y.in_stream`.

```

friend bool operator==(default_sentinel_t, const istream_iterator& i);
friend bool operator==(const istream_iterator& i, default_sentinel_t);

```

~~Returns:~~ `!i.in_stream`.

```

template<class T, class charT, class traits, class Distance>
bool operator!=(const istream_iterator<T, charT, traits, Distance>& x,
const istream_iterator<T, charT, traits, Distance>& y);
friend bool operator!=(default_sentinel_t x, const istream_iterator& y);
friend bool operator!=(const istream_iterator& x, default_sentinel_t y);

```

~~Returns:~~ `!(x == y)`

Change 23.6.3 [istreambuf.iterator]:

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class istreambuf_iterator {
    public:
        [...]
        bool equal(const istreambuf_iterator& b) const;

friend bool operator==(default_sentinel_t s, const istreambuf_iterator& i);
        friend bool operator==(const istreambuf_iterator& i, default_sentinel_t s)÷
        { return i.equal(s); }
friend bool operator!=(default_sentinel_t a, const istreambuf_iterator& b);
    };
}

```

```

friend bool operator!=(const istreambuf_iterator& a, default_sentinel_t b);
friend bool operator==(const istreambuf_iterator& a, const istreambuf_iterator& b)
{ return a.equal(b); }

private:
    streambuf_type* sbuf_;           // exposition only
};
}

```

Remove the specifications for `operator==` and `operator!=` from 23.6.3.3 [istreambuf.iterator.ops]:

```

template<class charT, class traits>
bool operator==(const istreambuf_iterator<charT,traits>& a,
const istreambuf_iterator<charT,traits>& b);

```

Returns: `a.equal(b)`.

```

friend bool operator==(default_sentinel_t s, const istreambuf_iterator& i);
friend bool operator==(const istreambuf_iterator& i, default_sentinel_t s);

```

Returns: `i.equal(s)`.

```

template<class charT, class traits>
bool operator!=(const istreambuf_iterator<charT,traits>& a,
const istreambuf_iterator<charT,traits>& b);
friend bool operator!=(default_sentinel_t a, const istreambuf_iterator& b);
friend bool operator!=(const istreambuf_iterator& a, default_sentinel_t b);

```

Returns: `!a.equal(b)`.

§ 4.9. Clause 24: Ranges library

Change 24.6.3.3 [range.iota.iterator]:

```

namespace std::ranges {
    template<class W, class Bound>
    struct iota_view<W, Bound>::iterator {
    private:
        W value_ = W();           // exposition only
    public:
        [...]

        friend constexpr bool operator==(const iterator& x, const iterator& y)
            requires EqualityComparable<W>;
friend constexpr bool operator!=(const iterator& x, const iterator& y)
requires EqualityComparable<W>;

        friend constexpr bool operator<(const iterator& x, const iterator& y)
            requires StrictTotallyOrdered<W>;
        friend constexpr bool operator>(const iterator& x, const iterator& y)
            requires StrictTotallyOrdered<W>;
        friend constexpr bool operator<=(const iterator& x, const iterator& y)
            requires StrictTotallyOrdered<W>;
        friend constexpr bool operator>=(const iterator& x, const iterator& y)

```

```

    requires StrictTotallyOrdered<W>;
    friend constexpr compare three way result t<W> operator<==(const iterator& x, const iterator& y)
    requires StrictTotallyOrdered<W> && ThreeWayComparable<W>

    [...]
};
}

```

Change 24.6.3.3 [range.iota.iterator], paragraphs 14-20:

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires EqualityComparable<W>;

```

Effects: Equivalent to: `return x.value_ == y.value_;`

```

friend constexpr bool operator!=(const iterator& x, const iterator& y)
    requires EqualityComparable<W>;

```

Effects: Equivalent to: ~~`return !(x == y);`~~

```

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires StrictTotallyOrdered<W>;

```

Effects: Equivalent to: `return x.value_ < y.value_;`

[...]

```

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires StrictTotallyOrdered<W>;

```

Effects: Equivalent to: `return !(x < y);`

```

friend constexpr compare three way result t<W> operator<==(const iterator& x, const iterator& y)
    requires StrictTotallyOrdered<W> && ThreeWayComparable<W>;

```

Effects: Equivalent to: `return x.value_ <== y.value_;`

Remove obsolete equality operators 24.6.3.4 [range.iota.sentinel]:

```

namespace std::ranges {
    template<class W, class Bound>
    struct iota_view<W, Bound>::sentinel {
    private:
        Bound bound_ = Bound();    // exposition only
    public:
        sentinel() = default;
        constexpr explicit sentinel(Bound bound);
        friend constexpr bool operator==(const iterator& x, const sentinel& y);
        friend constexpr bool operator==(const sentinel& x, const iterator& y);
        friend constexpr bool operator!=(const iterator& x, const sentinel& y);
        friend constexpr bool operator!=(const sentinel& x, const iterator& y);
    };
}

```

```
constexpr explicit sentinel(Bound bound);
```

Effects: Initializes `bound_` with `bound`.

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

Effects: Equivalent to: `return x.value_ == y.bound_;`

```
friend constexpr bool operator==(const sentinel& x, const iterator& y);
```

~~Effects: Equivalent to: return y == x;~~

```
friend constexpr bool operator!=(const iterator& x, const sentinel& y);
```

~~Effects: Equivalent to: return !(x == y);~~

```
friend constexpr bool operator!=(const sentinel& x, const iterator& y);
```

~~Effects: Equivalent to: return !(y == x);~~

Remove `operator!=` from 24.7.4.3 [range.filter.iterator]:

```
namespace std::ranges {  
    template<class V, class Pred>  
    class filter_view<V, Pred>::iterator {  
    private:  
        iterator_t<V> current_ = iterator_t<V>(); // exposition only  
        filter_view* parent_ = nullptr;           // exposition only  
    public:  
        [...]  
        friend constexpr bool operator==(const iterator& x, const iterator& y)  
            requires EqualityComparable<iterator_t<V>>;  
        friend constexpr bool operator!=(const iterator& x, const iterator& y)  
            requires EqualityComparable<iterator_t<V>>;  
        [...]  
    };  
}
```

and

```
friend constexpr bool operator==(const iterator& x, const iterator& y)  
    requires EqualityComparable<iterator_t<V>>;
```

Effects: Equivalent to: `return x.current_ == y.current_;`

```
friend constexpr bool operator!=(const iterator& x, const iterator& y)  
    requires EqualityComparable<iterator_t<V>>;
```

~~Effects: Equivalent to: return !(x == y);~~

Remove obsolete equality operators from 24.7.4.4 [range.filter.sentinel]:

```

namespace std::ranges {
    template<class V, class Pred>
    class filter_view<V, Pred>::sentinel {
    private:
        sentinel_t<V> end_ = sentinel_t<V>();           // exposition only
    public:
        sentinel() = default;
        constexpr explicit sentinel(filter_view& parent);
        constexpr sentinel_t<V> base() const;
        friend constexpr bool operator==(const iterator& x, const sentinel& y);
friend constexpr bool operator==(const sentinel& x, const iterator& y);
friend constexpr bool operator!=(const iterator& x, const sentinel& y);
friend constexpr bool operator!=(const sentinel& x, const iterator& y);
    };
}

```

```
constexpr explicit sentinel(filter_view& parent);
```

Effects: Initializes `end_` with `ranges::end(parent)`.

```
constexpr sentinel_t<V> base() const;
```

Effects: Equivalent to: `return end_;`

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

Effects: Equivalent to: `return x.current_ == y.end_;`

```
friend constexpr bool operator==(const sentinel& x, const iterator& y);
```

Effects: Equivalent to: ~~`return y == x;`~~

```
friend constexpr bool operator!=(const iterator& x, const sentinel& y);
```

Effects: Equivalent to: ~~`return !(x == y);`~~

```
friend constexpr bool operator!=(const sentinel& x, const iterator& y);
```

Effects: Equivalent to: ~~`return !(y == x);`~~

Change 24.7.5.3 [range.transform.iterator]:

```

namespace std::ranges {
    template<class V, class F>
    template<bool Const>
    class transform_view<V, F>::iterator {
    [...]
        friend constexpr bool operator==(const iterator& x, const iterator& y)
            requires EqualityComparable<iterator_t<Base>>;
friend constexpr bool operator!=(const iterator& x, const iterator& y)
    requires EqualityComparable<iterator_t<Base>>;

        friend constexpr bool operator<(const iterator& x, const iterator& y)

```



```

        requires RandomAccessRange<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
        requires RandomAccessRange<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
        requires RandomAccessRange<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
        requires RandomAccessRange<Base>;
    friend constexpr compare_three_way_result t<iterator t<Base>> operator<=>(const iterator& x, const iterator& y)
    requires RandomAccessRange<Base>; && ThreeWayComparable<iterator t<Base>>
    [...]
};
}

```

Change 24.7.5.3 [range.transform.iterator], paragraphs 13-18:

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires EqualityComparable<iterator_t<Base>>;

```

Effects: Equivalent to: `return x.current_ == y.current_;`

```

friend constexpr bool operator!=(const iterator& x, const iterator& y)
requires EqualityComparable<iterator_t<Base>>;

```

Effects: Equivalent to: ~~`return !(x == y);`~~

```

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires RandomAccessRange<Base>;

```

Effects: Equivalent to: `return x.current_ < y.current_;`

[...]

```

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires RandomAccessRange<Base>;

```

Effects: Equivalent to: `return !(x < y);`

```

friend constexpr compare_three_way_result t<iterator t<Base>> operator<=>(const iterator& x, const iterator& y)
requires RandomAccessRange<Base>; && ThreeWayComparable<iterator t<Base>>

```

Effects: Equivalent to: `return x.current_ <=> y.current_;`

Remove obsolete equality operators from 24.7.5.4 [range.transform.sentinel]:

```

namespace std::ranges {
    template<class V, class F>
    template<bool Const>
    class transform_view<V, F>::sentinel {
        [...]
        friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);
        friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);
        friend constexpr bool operator!=(const iterator<Const>& x, const sentinel& y);
        friend constexpr bool operator!=(const sentinel& x, const iterator<Const>& y);
        [...]
    };
}

```

```
};
}
```

and from 24.7.5.4 [range.transform.sentinel], paragraphs 4-7:

```
friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);
```

Effects: Equivalent to: `return x.current_ == y.end_;`

```
friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);
```

Effects: Equivalent to: ~~`return y == x;`~~

```
friend constexpr bool operator!=(const iterator<Const>& x, const sentinel& y);
```

Effects: Equivalent to: ~~`return !(x == y);`~~

```
friend constexpr bool operator!=(const sentinel& x, const iterator<Const>& y);
```

Effects: Equivalent to: ~~`return !(y == x);`~~

Remove obsolete equality operators from 24.7.6.3 [range.take.sentinel]:

```
namespace std::ranges {
    template<class V>
    template<bool Const>
    class take_view<V>::sentinel {
    private:
        using Base = conditional_t<Const, const V, V>;           // exposition only
        using CI = counted_iterator<iterator_t<Base>>;           // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>();           // exposition only
    public:
        [...]
        friend constexpr bool operator==(const sentinel& x, const CI& y);
        friend constexpr bool operator==(const CI& y, const sentinel& x);
        friend constexpr bool operator!=(const sentinel& x, const CI& y);
        friend constexpr bool operator!=(const CI& y, const sentinel& x);
    };
}
```

and from 24.7.6.3 [range.take.sentinel] paragraphs 4-5:

```
friend constexpr bool operator==(const sentinel& x, const CI& y);
friend constexpr bool operator==(const CI& y, const sentinel& x);
```

Effects: Equivalent to: `return y.count() == 0 || y.base() == x.end_;`

```
friend constexpr bool operator!=(const sentinel& x, const CI& y);
friend constexpr bool operator!=(const CI& y, const sentinel& x);
```

Effects: Equivalent to: `return !(x == y);`

Remove obsolete `operator!=` from 24.7.7.3 [range.join.iterator]:

```

namespace std::ranges {
template<class V>
template<bool Const>
struct join_view<V>::iterator {
    [...]
    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&
            EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;

    friend constexpr bool operator!=(const iterator& x, const iterator& y)
    requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&
    EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;

    [...]
};
}

```

and from 24.7.7.3 [range.join.iterator] paragraph 17:

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&
        EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;

```

Effects: Equivalent to: `return x.outer_ == y.outer_ && x.inner_ == y.inner_;`

```

friend constexpr bool operator!=(const iterator& x, const iterator& y)
requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&
EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;

```

Effects: Equivalent to: `return !(x == y);`

Remove obsolete equality operators from 24.7.7.4 [range.join.sentinel]:

```

namespace std::ranges {
template<class V>
template<bool Const>
struct join_view<V>::sentinel {
    [...]
    friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);
    friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);
    friend constexpr bool operator!=(const iterator<Const>& x, const sentinel& y);
    friend constexpr bool operator!=(const sentinel& x, const iterator<Const>& y);
};
}

```

and from 24.7.7.4 [range.join.sentinel] paragraphs 3-6:

```

friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);

```

Effects: Equivalent to: `return x.outer_ == y.end_;`

```

friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);

```

Effects: Equivalent to: `return y == x;`

```
friend constexpr bool operator!=(const_iterator<Const>& x, const_sentinel& y);
```

Effects: Equivalent to: `return !(x == y);`

```
friend constexpr bool operator!=(const_sentinel& x, const_iterator<Const>& y);
```

Effects: Equivalent to: `return !(y == x);`

Remove obsolete equality operators from 24.7.8.3 [range.split.outer]:

```
namespace std::ranges {  
    template<class V, class Pattern>  
    template<bool Const>  
    struct split_view<V, Pattern>::outer_iterator {  
        [...]  
        friend constexpr bool operator==(const outer_iterator& x, const outer_iterator& y)  
            requires ForwardRange<Base>;  
        friend constexpr bool operator!=(const outer_iterator& x, const outer_iterator& y)  
            requires ForwardRange<Base>;  
  
        friend constexpr bool operator==(const outer_iterator& x, default_sentinel_t);  
        friend constexpr bool operator==(default_sentinel_t, const outer_iterator& x);  
        friend constexpr bool operator!=(const outer_iterator& x, default_sentinel_t y);  
        friend constexpr bool operator!=(default_sentinel_t y, const outer_iterator& x);  
    };  
}
```

and from 24.7.8.3 [range.split.outer] paragraphs 7-10:

```
friend constexpr bool operator==(const outer_iterator& x, const outer_iterator& y)  
    requires ForwardRange<Base>;
```

Effects: Equivalent to: `return x.current_ == y.current_;`

```
friend constexpr bool operator!=(const outer_iterator& x, const outer_iterator& y)  
    requires ForwardRange<Base>;
```

Effects: Equivalent to: `return !(x == y);`

```
friend constexpr bool operator==(const outer_iterator& x, default_sentinel_t);  
friend constexpr bool operator==(default_sentinel_t, const outer_iterator& x);
```

Effects: Equivalent to: `return x.current_ == ranges::end(x.parent_ -> base_);`

```
friend constexpr bool operator!=(const outer_iterator& x, default_sentinel_t y);  
friend constexpr bool operator!=(default_sentinel_t y, const outer_iterator& x);
```

Effects: Equivalent to: `return !(x == y);`

Remove obsolete equality operators from 24.7.8.5 [range.split.inner]:

```
namespace std::ranges {  
    template<class V, class Pattern>
```

```

template<bool Const>
struct split_view<V, Pattern>::inner_iterator {
    [...]
    friend constexpr bool operator==(const inner_iterator& x, const inner_iterator& y)
        requires ForwardRange<Base>;
friend constexpr bool operator!=(const inner_iterator& x, const inner_iterator& y)
    —requires ForwardRange<Base>;

    friend constexpr bool operator==(const inner_iterator& x, default_sentinel_t);
friend constexpr bool operator==(default_sentinel_t, const inner_iterator& x);
friend constexpr bool operator!=(const inner_iterator& x, default_sentinel_t y);
friend constexpr bool operator!=(default_sentinel_t y, const inner_iterator& x);
    [...]
};
}

```

and from 24.7.8.5 [ranges.split.inner] paragraphs 4-7:

```

friend constexpr bool operator==(const inner_iterator& x, const inner_iterator& y)
    requires ForwardRange<Base>;

```

Effects: Equivalent to: `return x.i_.current_ == y.i_.current_;`

```

friend constexpr bool operator!=(const inner_iterator& x, const inner_iterator& y)
    —requires ForwardRange<Base>;

```

Effects: Equivalent to: `return !(x == y);`

```

friend constexpr bool operator==(const inner_iterator& x, default_sentinel_t);
friend constexpr bool operator==(default_sentinel_t, const inner_iterator& x);

```

Effects: Equivalent to:

```

auto cur = x.i_.current;
auto end = ranges::end(x.i_.parent_->base_);
if (cur == end) return true;
auto [pcur, pend] = subrange{x.i_.parent_->pattern_};
if (pcur == pend) return x.incremented_;
do {
    if (cur != pcur) return false;
    if (++pcur == pend) return true;
} while (++cur != end);
return false;

```

```

friend constexpr bool operator!=(const inner_iterator& x, default_sentinel_t y);
friend constexpr bool operator!=(default_sentinel_t y, const inner_iterator& x);

```

Effects: Equivalent to: `return !(x == y);`

§ 4.10. Clause 25: Algorithms library

Change 25.4 [algorithm.syn]:

```

namespace std {
    [...]
    // [alg.3way], three-way comparison algorithms
template<class T, class U>
constexpr auto compare_3way(const T& a, const U& b);
    template<class InputIterator1, class InputIterator2, class Cmp>
        constexpr auto
lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                InputIterator2 b2, InputIterator2 e2,
                                Cmp comp)
        -> common_comparison_category_t<decltype(comp(*b1, *b2)), strong_ordering>;
    template<class InputIterator1, class InputIterator2>
        constexpr auto
lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                InputIterator2 b2, InputIterator2 e2);
    [...]
}

```

Change 25.7.11 [alg.3way]:

```
template<class T, class U> constexpr auto compare_3way(const T& a, const U& b);
```

Effects: Compares two values and produces a result of the strongest applicable comparison category type:

- Returns $a \leq b$ if that expression is well-formed.
- Otherwise, if the expressions $a == b$ and $a < b$ are each well-formed and convertible to bool, returns `strong_ordering::equal` when $a == b$ is true, otherwise returns `strong_ordering::less` when $a < b$ is true, and otherwise returns `strong_ordering::greater`.
- Otherwise, if the expression $a == b$ is well-formed and convertible to bool, returns `strong_equality::equal` when $a == b$ is true, and otherwise returns `strong_equality::nonequal`.
- Otherwise, the function is defined as deleted.

Change 25.7.11 [alg.3way] paragraph 2:

```

template<class InputIterator1, class InputIterator2, class Cmp>
    constexpr auto
lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                InputIterator2 b2, InputIterator2 e2,
                                Cmp comp);

```

Change 25.7.11 [alg.3way] paragraph 4:

```

template<class InputIterator1, class InputIterator2>
    constexpr auto
lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                InputIterator2 b2, InputIterator2 e2);

```

Effects: Equivalent to:

```
return lexicographical_compare_3way(b1, e1, b2, e2,
    [](const auto& t, const auto& u) {
        return compare_3way(t, u);
    });
return lexicographical_compare_three_way(b1, e1, b2, e2, compare_three_way());
```

§ 4.11. Clause 26: Numerics library

Remove obsolete equality operators from 26.4.1 [complex.syn]:

```
namespace std {
    // [complex], class template complex
    template<class T> class complex;

    // [complex.special], specializations
    template<> class complex<float>;
    template<> class complex<double>;
    template<> class complex<long double>;

    [...]

    template<class T> constexpr bool operator==(const complex<T>&, const complex<T>&);
    template<class T> constexpr bool operator==(const complex<T>&, const T&);
template<class T> constexpr bool operator==(const T&, const complex<T>&);

template<class T> constexpr bool operator!=(const complex<T>&, const complex<T>&);
template<class T> constexpr bool operator!=(const complex<T>&, const T&);
template<class T> constexpr bool operator!=(const T&, const complex<T>&);

    [...]
}
```

and in 26.4.6 [complex.ops]:

```
template<class T> constexpr bool operator==(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr bool operator==(const complex<T>& lhs, const T& rhs);
template<class T> constexpr bool operator==(const T& lhs, const complex<T>& rhs);
```

Returns: `lhs.real() == rhs.real() && lhs.imag() == rhs.imag()`.

Remarks: The imaginary part is assumed to be `T()`, or `0.0`, for the `T` arguments.

```
template<class T> constexpr bool operator!=(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr bool operator!=(const complex<T>& lhs, const T& rhs);
template<class T> constexpr bool operator!=(const T& lhs, const complex<T>& rhs);
```

Returns: ~~`rhs.real() != lhs.real() || rhs.imag() != lhs.imag()`~~.

Add `operator==` to `std::slice` in 26.7.4.1 [class.slice.overview]:

```
namespace std {
    class slice {
    public:
        slice();
        slice(size_t, size_t, size_t);
```

```

    size_t start() const;
    size_t size() const;
    size_t stride() const;

    friend bool operator==(const slice& x, const slice& y);
};
}

```

and add a new subclause "Operators" after 26.7.4.3 [slice.access]:

```

friend bool operator==(const slice& x, const slice& y);

```

Effects: Equivalent to

```

return x.start() == y.start() &&
       x.size() == y.size() &&
       x.stride() == y.stride();

```

§ 4.12. Clause 27: Time library

Change 27.2 [time.syn]:

```

namespace std {
    namespace chrono {
        [...]

        // [time.duration.comparisons], duration comparisons
        template<class Rep1, class Period1, class Rep2, class Period2>
        constexpr bool operator==(const duration<Rep1, Period1>& lhs,
                                   const duration<Rep2, Period2>& rhs);
        template<class Rep1, class Period1, class Rep2, class Period2>
        constexpr bool operator!=(const duration<Rep1, Period1>& lhs,
                                   const duration<Rep2, Period2>& rhs);
        template<class Rep1, class Period1, class Rep2, class Period2>
        constexpr bool operator< (const duration<Rep1, Period1>& lhs,
                                   const duration<Rep2, Period2>& rhs);
        template<class Rep1, class Period1, class Rep2, class Period2>
        constexpr bool operator> (const duration<Rep1, Period1>& lhs,
                                   const duration<Rep2, Period2>& rhs);
        template<class Rep1, class Period1, class Rep2, class Period2>
        constexpr bool operator<= (const duration<Rep1, Period1>& lhs,
                                    const duration<Rep2, Period2>& rhs);
        template<class Rep1, class Period1, class Rep2, class Period2>
        constexpr bool operator>= (const duration<Rep1, Period1>& lhs,
                                    const duration<Rep2, Period2>& rhs);

        [...]


        // [time.point.comparisons], time_point comparisons
        template<class Clock, class Duration1, class Duration2>
        constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
                                   const time_point<Clock, Duration2>& rhs);
        template<class Clock, class Duration1, class Duration2>
        constexpr bool operator!=(const time_point<Clock, Duration1>& lhs,
                                   const time_point<Clock, Duration2>& rhs);


```



```

template<class Clock, class Duration1, class Duration2>
constexpr bool operator< (const time_point<Clock, Duration1>& lhs,
constexpr time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2>
constexpr bool operator> (const time_point<Clock, Duration1>& lhs,
constexpr time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2>
constexpr bool operator<= (const time_point<Clock, Duration1>& lhs,
constexpr time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2>
constexpr bool operator>= (const time_point<Clock, Duration1>& lhs,
constexpr time_point<Clock, Duration2>& rhs);

```

[...]

```

// [time.cal.day], class day
class day;

```

```

constexpr bool operator==(const day& x, const day& y) noexcept;
constexpr bool operator!=(const day& x, const day& y) noexcept;
constexpr bool operator< (const day& x, const day& y) noexcept;
constexpr bool operator> (const day& x, const day& y) noexcept;
constexpr bool operator<= (const day& x, const day& y) noexcept;
constexpr bool operator>= (const day& x, const day& y) noexcept;

```

[...]

```

// [time.cal.month], class month
class month;

```

```

constexpr bool operator==(const month& x, const month& y) noexcept;
constexpr bool operator!=(const month& x, const month& y) noexcept;
constexpr bool operator< (const month& x, const month& y) noexcept;
constexpr bool operator> (const month& x, const month& y) noexcept;
constexpr bool operator<= (const month& x, const month& y) noexcept;
constexpr bool operator>= (const month& x, const month& y) noexcept;

```

[...]

```

// [time.cal.year], class year
class year;

```

```

constexpr bool operator==(const year& x, const year& y) noexcept;
constexpr bool operator!=(const year& x, const year& y) noexcept;
constexpr bool operator< (const year& x, const year& y) noexcept;
constexpr bool operator> (const year& x, const year& y) noexcept;
constexpr bool operator<= (const year& x, const year& y) noexcept;
constexpr bool operator>= (const year& x, const year& y) noexcept;

```

[...]

```

// [time.cal.wd], class weekday
class weekday;

```

```

constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;

```

```

[...]
```

```

// [time.cal.wdidx], class weekday_indexed
class weekday_indexed;

constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;

[...]
```

```

// [time.cal.wdlast], class weekday_last
class weekday_last;

constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;

[...]
```

```

// [time.cal.md], class month_day
class month_day;

constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator<(const month_day& x, const month_day& y) noexcept;
constexpr bool operator>(const month_day& x, const month_day& y) noexcept;
constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;

[...]
```

```

// [time.cal.mdlast], class month_day_last
class month_day_last;

constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator<(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator>(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;

[...]
```

```

// [time.cal.mwd], class month_weekday
class month_weekday;

constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;

[...]
```

```

// [time.cal.mwdlast], class month_weekday_last
class month_weekday_last;

constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;

[...]
```

```

// [time.cal.ym], class year_month
class year_month;

constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator<(const year_month& x, const year_month& y) noexcept;
constexpr bool operator>(const year_month& x, const year_month& y) noexcept;
constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;

[...]

// [time.cal.ymd], class year_month_day
class year_month_day;

constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator<(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator>(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;

[...]

// [time.cal.ymdlast], class year_month_day_last
class year_month_day_last;

constexpr bool operator==(const year_month_day_last& x,
const year_month_day_last& y) noexcept;
constexpr bool operator!=(const year_month_day_last& x,
const year_month_day_last& y) noexcept;
constexpr bool operator<(const year_month_day_last& x,
const year_month_day_last& y) noexcept;
constexpr bool operator>(const year_month_day_last& x,
const year_month_day_last& y) noexcept;
constexpr bool operator<=(const year_month_day_last& x,
const year_month_day_last& y) noexcept;
constexpr bool operator>=(const year_month_day_last& x,
const year_month_day_last& y) noexcept;

[...]

// [time.cal.ymwd], class year_month_weekday
class year_month_weekday;

constexpr bool operator==(const year_month_weekday& x,
const year_month_weekday& y) noexcept;
constexpr bool operator!=(const year_month_weekday& x,
const year_month_weekday& y) noexcept;

[...]

// [time.cal.ymwdlast], class year_month_weekday_last
class year_month_weekday_last;

constexpr bool operator==(const year_month_weekday_last& x,

```

```

const year_month_weekday_last& y) noexcept;
constexpr bool operator!=(const year_month_weekday_last& x,
const year_month_weekday_last& y) noexcept;

[...]

// [time.zone.timezone], class time_zone
enum class choose {earliest, latest};
class time_zone;

bool operator==(const time_zone& x, const time_zone& y) noexcept;
bool operator!=(const time_zone& x, const time_zone& y) noexcept;

bool operator<(const time_zone& x, const time_zone& y) noexcept;
bool operator>(const time_zone& x, const time_zone& y) noexcept;
bool operator<=(const time_zone& x, const time_zone& y) noexcept;
bool operator>=(const time_zone& x, const time_zone& y) noexcept;
[...]

// [time.zone.zonedtime], class template zoned_time
template<class Duration, class TimeZonePtr = const time_zone*> class zoned_time;

using zoned_seconds = zoned_time;

template<class Duration1, class Duration2, class TimeZonePtr>
bool operator==(const zoned_time<Duration1, TimeZonePtr>& x,
const zoned_time<Duration2, TimeZonePtr>& y);

template<class Duration1, class Duration2, class TimeZonePtr>
bool operator!=(const zoned_time<Duration1, TimeZonePtr>& x,
const zoned_time<Duration2, TimeZonePtr>& y);

[...]

// [time.zone.leap], leap second support
class leap;

bool operator==(const leap& x, const leap& y);
bool operator!=(const leap& x, const leap& y);
bool operator<(const leap& x, const leap& y);
bool operator>(const leap& x, const leap& y);
bool operator<=(const leap& x, const leap& y);
bool operator>=(const leap& x, const leap& y);

template<class Duration>
bool operator==(const leap& x, const sys_time<Duration>& y);
template<class Duration>
bool operator==(const sys_time<Duration>& x, const leap& y);
template<class Duration>
bool operator!=(const leap& x, const sys_time<Duration>& y);
template<class Duration>
bool operator!=(const sys_time<Duration>& x, const leap& y);
template<class Duration>
bool operator<(const leap& x, const sys_time<Duration>& y);
template<class Duration>
bool operator<(const sys_time<Duration>& x, const leap& y);
template<class Duration>

```

```

bool operator> (const leap& x, const sys_time<Duration>& y);
template<class Duration>
bool operator> (const sys_time<Duration>& x, const leap& y);
template<class Duration>
bool operator<=(const leap& x, const sys_time<Duration>& y);
template<class Duration>
bool operator<=(const sys_time<Duration>& x, const leap& y);
template<class Duration>
bool operator>=(const leap& x, const sys_time<Duration>& y);
template<class Duration>
bool operator>=(const sys_time<Duration>& x, const leap& y);

// [time.zone.link], class link
class link;

bool operator==(const link& x, const link& y);
bool operator!=(const link& x, const link& y);
bool operator< (const link& x, const link& y);
bool operator> (const link& x, const link& y);
bool operator<=(const link& x, const link& y);
bool operator>=(const link& x, const link& y);
}
}

```

Change 27.5 [time.duration]:

```

namespace std::chrono {
    template<class Rep, class Period = ratio<1>>
    class duration {
    public:
        using rep    = Rep;
        using period = typename Period::type;

    private:
        rep rep_;          // exposition only

    public:
        [...]
        // [time.duration.comparisons], duration comparisons
        template<class Rep2, class Period2>
        friend constexpr bool operator==(const duration& lhs, const duration<Rep2, Period2>& rhs);
        template<class Rep2, class Period2>
        friend constexpr bool operator<(const duration& lhs, const duration<Rep2, Period2>& rhs);
        template<class Rep2, class Period2>
        friend constexpr bool operator>(const duration& lhs, const duration<Rep2, Period2>& rhs);
        template<class Rep2, class Period2>
        friend constexpr bool operator<=(const duration& lhs, const duration<Rep2, Period2>& rhs);
        template<class Rep2, class Period2>
        friend constexpr bool operator>=(const duration& lhs, const duration<Rep2, Period2>& rhs);
        template<class Rep2, class Period2> requires see below
        friend constexpr see below operator<=>(const duration& lhs, const duration<Rep2, Period2>& rhs);
    };
}

```

Change 27.5.6 [time.duration.comparisons]:

In the function descriptions that follow, `CT` represents `common_type_t<A, B>`, where `A` and `B` are the types of the two arguments to the function.

```
template<class Rep1, class Period1, class Rep2, class Period2>
    friend constexpr bool operator==(const duration<Rep1, Period1>& lhs,
                                     const duration<Rep2, Period2>& rhs);
```

Returns: `CT(lhs).count() == CT(rhs).count()`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
    constexpr bool operator!=(const duration<Rep1, Period1>& lhs,
                             const duration<Rep2, Period2>& rhs);
```

Returns: ~~`!(lhs == rhs)`.~~

```
template<class Rep1, class Period1, class Rep2, class Period2>
    friend constexpr bool operator<(const duration<Rep1, Period1>& lhs,
                                   const duration<Rep2, Period2>& rhs);
```

Returns: `CT(lhs).count() < CT(rhs).count()`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
    friend constexpr bool operator>(const duration<Rep1, Period1>& lhs,
                                   const duration<Rep2, Period2>& rhs);
```

Returns: `rhs < lhs`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
    friend constexpr bool operator<=(const duration<Rep1, Period1>& lhs,
                                    const duration<Rep2, Period2>& rhs);
```

Returns: `!(rhs < lhs)`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
    friend constexpr bool operator>=(const duration<Rep1, Period1>& lhs,
                                    const duration<Rep2, Period2>& rhs);
```

Returns: `!(lhs < rhs)`.

```
template<class Rep2, class Period2> requires ThreeWayComparable<typename CT::rep>
    friend constexpr compare_three_way_result_t<typename CT::rep> operator<=>(const duration<Rep1, Period1>& lhs,
                                         const duration<Rep2, Period2>& rhs);
```

Returns: `CT(lhs).count() <=> CT(rhs).count()`.

Change 27.6 [time.point]:

```
namespace std::chrono {
    template<class Clock, class Duration = typename Clock::duration>
    class time_point {
    public:
        [...]
```

```

    // [time.point.comparisons], time_point comparisons
    template<class Duration2>
    friend constexpr bool operator==(const time_point& lhs, const time_point<Clock, Duration2>& rhs);
    template<class Duration2>
    friend constexpr bool operator<(const time_point& lhs, const time_point<Clock, Duration2>& rhs);
    template<class Duration2>
    friend constexpr bool operator>(const time_point& lhs, const time_point<Clock, Duration2>& rhs);
    template<class Duration2>
    friend constexpr bool operator<=(const time_point& lhs, const time_point<Clock, Duration2>& rhs);
    template<class Duration2>
    friend constexpr bool operator>=(const time_point& lhs, const time_point<Clock, Duration2>& rhs);
    template<ThreeWayComparableWith<Duration> Duration2>
    friend constexpr compare_three_way_result_t<Duration, Duration2>
    operator<=>(const time_point& lhs, const time_point<Clock, Duration2>& rhs);
};
}

```

Change 27.6.6 [time.point.comparisons]:

```

template<class Clock, class Duration1, class Duration2>
    friend constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
                                     const time_point<Clock, Duration2>& rhs);

```

Returns: `lhs.time_since_epoch() == rhs.time_since_epoch()`.

```

template<class Clock, class Duration1, class Duration2>
    constexpr bool operator!=(const time_point<Clock, Duration1>& lhs,
                              const time_point<Clock, Duration2>& rhs);

```

Returns: ~~`!(lhs == rhs)`~~.

```

template<class Clock, class Duration1, class Duration2>
    friend constexpr bool operator<(const time_point<Clock, Duration1>& lhs,
                                    const time_point<Clock, Duration2>& rhs);

```

Returns: `lhs.time_since_epoch() < rhs.time_since_epoch()`.

```

template<class Clock, class Duration1, class Duration2>
    friend constexpr bool operator>(const time_point<Clock, Duration1>& lhs,
                                    const time_point<Clock, Duration2>& rhs);

```

Returns: `rhs < lhs`.

```

template<class Clock, class Duration1, class Duration2>
    friend constexpr bool operator<=(const time_point<Clock, Duration1>& lhs,
                                     const time_point<Clock, Duration2>& rhs);

```

Returns: `!(rhs < lhs)`.

```

template<class Clock, class Duration1, class Duration2>
    friend constexpr bool operator>=(const time_point<Clock, Duration1>& lhs,
                                     const time_point<Clock, Duration2>& rhs);

```

Returns: `!(lhs < rhs)`.

```
template<ThreeWayComparableWith<Duration> Duration2>  
friend constexpr compare_three_way_result_t<Duration, Duration2>  
operator<=>(const time_point& lhs, const time_point<Clock, Duration2>& rhs);
```

Returns: `lhs.time_since_epoch() <=> rhs.time_since_epoch()`.

Change 27.8.3.1 [time.cal.day.overview]:

```
namespace std::chrono {  
    class day {  
        unsigned char d_;           // exposition only  
    public:  
        [...]  
  
        friend constexpr bool operator==(const day& x, const day& y) noexcept;  
        friend constexpr strong_ordering operator<=>(const day& x, const day& y) noexcept;  
    };  
}
```

Change 27.8.3.3 [time.cal.day.nonmembers]:

```
friend constexpr bool operator==(const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

```
constexpr bool operator<(const day& x, const day& y) noexcept;
```

Returns: ~~`unsigned{x} < unsigned{y}`~~.

```
friend constexpr strong_ordering operator<=>(const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} <=> unsigned{y}`.

Change 27.8.4.1 [time.cal.month.overview]:

```
namespace std::chrono {  
    class month {  
        unsigned char m_;           // exposition only  
    public:  
        [...]  
  
        friend constexpr bool operator==(const month& x, const month& y) noexcept;  
        friend constexpr strong_ordering operator<=>(const month& x, const month& y) noexcept;  
    };  
}
```

Change 27.8.4.3 [time.cal.month.nonmembers]:

```
friend constexpr bool operator==(const month& x, const month& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.


```
constexpr bool operator<(const month& x, const month& y) noexcept;
```

~~Returns: `unsigned{x} < unsigned{y}`.~~

```
friend constexpr strong_ordering operator<==(const month& x, const month& y) noexcept;
```

~~Returns: `unsigned{x} <== unsigned{y}`.~~

Change 27.8.5.1 [time.cal.year.overview]:

```
namespace std::chrono {
    class year {
        short y_;          // exposition only
    public:
        [...]

        friend constexpr bool operator==(const year& x, const year& y) noexcept;
        friend constexpr strong_ordering operator<==(const year& x, const year& y) noexcept;
    };
}
```

Change 27.8.5.3 [time.cal.year.nonmembers]:

```
friend constexpr bool operator==(const year& x, const year& y) noexcept;
```

Returns: `int{x} == int{y}`.

```
constexpr bool operator<(const year& x, const year& y) noexcept;
```

~~Returns: `int{x} < int{y}`.~~

```
friend constexpr strong_ordering operator<==(const year& x, const year& y) noexcept;
```

~~Returns: `int{x} <== int{y}`.~~

Change 27.8.6.1 [time.cal.wd.overview]:

```
namespace std::chrono {
    class weekday {
        unsigned char wd_;          // exposition only
    public:
        [...]

        friend constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
    };
}
```

Change 27.8.6.3 [time.cal.wd.nonmembers]:

```
friend constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

Change 27.8.7.1 [time.cal.wdidx.overview]:

```
namespace std::chrono {
    class weekday_indexed {
        chrono::weekday wd_;           // exposition only
        unsigned char    index_;       // exposition only
    public:
        [...]

        friend constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
    };
}
```

Change 27.8.7.3 [time.cal.wdidx.nonmembers]:

```
friend constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
```

Returns: `x.weekday() == y.weekday() && x.index() == y.index()`.

Change 27.8.8.1 [time.cal.wdlast.overview]:

```
namespace std::chrono {
    class weekday_last {
        chrono::weekday wd_;           // exposition only
    public:
        [...]

        friend constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
    };
}
```

Change 27.8.8.3 [time.cal.wdlast.nonmembers]:

```
friend constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
```

Returns: `x.weekday() == y.weekday()`.

Change 27.8.9.1 [time.cal.md.overview]:

```
namespace std::chrono {
    class month_day {
        chrono::month m_;             // exposition only
        chrono::day d_;               // exposition only
    public:
        [...]

        friend constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
        friend constexpr strong_ordering operator<=>(const month_day& x, const month_day& y) noexcept;
    };
}
```

Change 27.8.9.3 [time.cal.md.nonmembers]:

```
friend constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
```

Returns: `x.month() == y.month() && x.day() == y.day()`.

~~constexpr bool operator<(const month_day& x, const month_day& y) noexcept;~~

~~Returns: If `x.month() < y.month()` returns `true`. Otherwise, if `x.month() > y.month()` returns `false`. Otherwise, returns `x.day() < y.day()`.~~

friend constexpr strong_ordering operator<=>(const month_day& x, const month_day& y) noexcept;

Returns: Let `c` be `x.month() <=> y.month()`. If `c != 0` returns `c`. Otherwise, returns `x.day() <=> y.day()`.

Change 27.8.10.1 [time.cal.mdlast.overview]:

```
namespace std::chrono {
    class month_day_last {
        chrono::month m_;          // exposition only
    public:
        [...]

        friend constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
        friend constexpr strong_ordering operator<=>(const month_day_last& x, const month_day_last& y) noexcept;
    };
}
```

Change 27.8.10.3 [time.cal.mdlast.nonmembers]:

friend constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;

Returns: `x.month() == y.month()`.

~~constexpr bool operator<(const month_day_last& x, const month_day_last& y) noexcept;~~

~~Returns: `x.month() < y.month()`.~~

friend constexpr strong_ordering operator<=>(const month_day_last& x, const month_day_last& y) noexcept;

Returns: `x.month() <=> y.month()`.

Change 27.8.11.1 [time.cal.mwd.overview]:

```
namespace std::chrono {
    class month_weekday {
        chrono::month m_;          // exposition only
        chrono::weekday_indexed wdi_; // exposition only
    public:
        [...]

        friend constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
    };
}
```

Change 27.8.11.3 [time.cal.mwd.nonmembers]:

```
friend constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
```

Returns: `x.month() == y.month() && x.weekday_indexed() == y.weekday_indexed()`.

Change 27.8.12.1 [time.cal.mwdlast.overview]:

```
namespace std::chrono {
    class month_weekday_last {
        chrono::month m_; // exposition only
        chrono::weekday_last wdl_; // exposition only
    public:
        [...]

        friend constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
    };
}
```

Change 27.8.12.3 [time.cal.mwdlast.nonmembers]:

```
friend constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
```

Returns: `x.month() == y.month() && x.weekday_last() == y.weekday_last()`.

Change 27.8.13.1 [time.cal.ym.overview]:

```
namespace std::chrono {
    class year_month {
        chrono::year y_; // exposition only
        chrono::month m_; // exposition only
    public:
        [...]

        friend constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
        friend constexpr strong_ordering operator<=>(const year_month& x, const year_month& y) noexcept;
    };
}
```

Change 27.8.13.3 [time.cal.ym.nonmembers]:

```
friend constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month()`.

~~constexpr bool operator<(const year_month& x, const year_month& y) noexcept;~~

~~Returns: If `x.year() < y.year()` returns `true`. Otherwise, if `x.year() > y.year()` returns `false`. Otherwise, returns `x.month() < y.month()`.~~

```
friend constexpr strong_ordering operator<=>(const year_month& x, const year_month& y) noexcept;
```

Returns: Let `c` be `x.year() <=> y.year()`. If `c != 0` returns `c`. Otherwise, returns `x.month() <=> y.month()`.

Change 27.8.14.1 [time.cal.ymd.overview]:

```
namespace std::chrono {
    class year_month_day {
        chrono::year y_;           // exposition only
        chrono::month m_;          // exposition only
        chrono::day d_;            // exposition only
    public:
        [...]

        friend constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
        friend constexpr strong_ordering operator<==(const year_month_day& x, const year_month_day& y) noexcept;
    };
}
```

Change 27.8.14.3 [time.cal.ymd.nonmembers]:

```
friend constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.day() == y.day()`.

```
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
```

~~Returns: If `x.year() < y.year()`, returns true. Otherwise, if `x.year() > y.year()`, returns false. Otherwise, if `x.month() < y.month()`, returns true. Otherwise, if `x.month() > y.month()`, returns false. Otherwise, returns `x.day() < y.day()`.~~

```
friend constexpr strong_ordering operator<==(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: Let `c` be `x.year() <== y.year()`. If `c != 0` returns `c`. Let `c2` be `x.month() <== y.month()`. If `c2 != 0` returns `c2`. Otherwise, returns `x.day() <== y.day()`.

Change 27.8.15.1 [time.cal.ymdlast.overview]:

```
namespace std::chrono {
    class year_month_day_last {
        chrono::year y_;           // exposition only
        chrono::month_day_last mdl_; // exposition only
    public:
        [...]

        friend constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
        friend constexpr strong_ordering operator<==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
    };
}
```

Change 27.8.15.3 [time.cal.ymdlast.nonmembers]:

```
friend constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `x.year() == y.year() && x.month_day_last() == y.month_day_last()`.

```
constexpr bool operator<=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

~~Returns: If `x.year()` < `y.year()` returns `true`. Otherwise, if `x.year()` > `y.year()` returns `false`. Otherwise, returns `x.month_day_last()` < `y.month_day_last()`.~~

```
friend constexpr strong_ordering operator<==(const year_month_day_last& x, const year_month_day_last& y) =
```

~~Returns: Let `c` be `x.year()` <== `y.year()`. If `c != 0` returns `c`. Otherwise, returns `x.month_day_last()` <== `y.month_day_last()`.~~

Change 27.8.16.1 [time.cal.ymwd.overview]:

```
namespace std::chrono {
    class year_month_weekday {
        chrono::year y_;           // exposition only
        chrono::month m_;          // exposition only
        chrono::weekday_indexed wdi_; // exposition only
    public:
        [...]

        friend constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) =
    };
}
```

Change 27.8.16.3 [time.cal.ymwd.nonmembers]:

```
friend constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) =
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.weekday_indexed() == y.weekday_indexed()`.

Change 27.8.17.1 [time.cal.ymwdlast.overview]:

```
namespace std::chrono {
    class year_month_weekday_last {
        chrono::year y_;           // exposition only
        chrono::month m_;          // exposition only
        chrono::weekday_last wdl_; // exposition only
    public:
        [...]

        friend constexpr bool operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) =
    };
}
```

Change 27.8.17.3 [time.cal.ymwdlast.nonmembers]:

```
friend constexpr bool operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) =
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.weekday_last() == y.weekday_last()`.

Change 27.10.5.1 [time.zone.overview]:

```

namespace std::chrono {
    class time_zone {
    public:
        [...]

        template<class Duration>
            local_time<common_type_t<Duration, seconds>>
                to_local(const sys_time<Duration>& tp) const;

        friend bool operator==(const time_zone& x, const time_zone& y) noexcept;
        friend strong_ordering operator<=>(const time_zone& x, const time_zone& y) noexcept;
    };
}

```

Change 27.10.5.3 [time.zone.nonmembers]:

```
friend bool operator==(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `x.name() == y.name()`.

```
bool operator<(const time_zone& x, const time_zone& y) noexcept;
```

~~Returns: `x.name() < y.name()`.~~

```
strong_ordering operator<=>(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `x.name() <=> y.name()`.

Change 27.10.7.1 [time.zone.zonedtime.overview]:

```

namespace std::chrono {
    template
    class zoned_time {
    public:
        [...]
        sys_time    get_sys_time()    const;
        sys_info    get_info()        const;

        template<class Duration2>
        friend bool operator==(const zoned_time& x, const zoned_time<Duration2, TimeZonePtr>
    };
    [...]
}

```

Change 27.10.7.4 [time.zone.zonedtime.nonmembers]:

```

template<class Duration1, class Duration2, class TimeZonePtr>
    friend bool operator==(const zoned_time<Duration1, TimeZonePtr>& x,
                          const zoned_time<Duration2, TimeZonePtr>& y);

```

Returns: `x.zone_ == y.zone_ && x.tp_ == y.tp_`.

```
template<class Duration1, class Duration2, class TimeZonePtr>
bool operator!=(const zoned_time<Duration1, TimeZonePtr>& x,
const zoned_time<Duration2, TimeZonePtr>& y);
```

~~Returns: `!(x == y)`.~~

Change 27.10.8.1 [time.zone.leap.overview]:

```
namespace std::chrono {
    class leap {
    public:
        leap(const leap&) = default;
        leap& operator=(const leap&) = default;

        // unspecified additional constructors

        constexpr sys_seconds date() const noexcept;

        friend constexpr bool operator==(const leap& x, const leap& y) noexcept;
        friend constexpr strong_ordering operator<=>(const leap& x, const leap& y) noexcept;

        template<class Duration>
        friend constexpr bool operator==(const leap& x, const sys_time<Duration>& y) noexcept;
        template<class Duration>
        friend constexpr strong_ordering operator<=>(const leap& x, const sys_time<Duration>& y) noexcept;
    };
}
```

Change 27.10.8.3 [time.zone.leap.nonmembers]:

```
friend constexpr bool operator==(const leap& x, const leap& y) noexcept;
```

Returns: `x.date() == y.date()`.

```
constexpr bool operator<(const leap& x, const leap& y) noexcept;
```

~~Returns: `x.date() < y.date()`.~~

```
friend constexpr strong_ordering operator<=>(const leap& x, const leap& y) noexcept;
```

Returns: `x.date() <=> y.date()`.

```
template<class Duration>
friend constexpr bool operator==(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: `x.date() == y`.

```
template<class Duration>
friend constexpr strong_ordering operator<=>(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: `x.date() <=> y`.


```
template<class Duration>
constexpr bool operator==(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: `y == x` :

```
template<class Duration>
constexpr bool operator!=(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: `!(x == y)` :

```
template<class Duration>
constexpr bool operator!=(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: `!(x == y)` :

```
template<class Duration>
constexpr bool operator<(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: `x.date() < y` :

```
template<class Duration>
constexpr bool operator<(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: `x < y.date()` :

```
template<class Duration>
constexpr bool operator>(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: `y < x` :

```
template<class Duration>
constexpr bool operator>(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: `y < x` :

```
template<class Duration>
constexpr bool operator<=(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: `!(y < x)` :

```
template<class Duration>
constexpr bool operator<=(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: `!(y < x)` :

```
template<class Duration>
constexpr bool operator>=(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: `!(x < y)` :

```
template<class Duration>  
constexpr bool operator==(const sys_time<Duration>& x, const leap& y) noexcept;
```

~~Returns: $!(x < y)$.~~

Change 27.10.9.1 [time.zone.link.overview]:

```
namespace std::chrono {  
    class link {  
    public:  
        link(link&&) = default;  
        link& operator=(link&&) = default;  
  
        // unspecified additional constructors  
  
        string_view name() const noexcept;  
        string_view target() const noexcept;  
  
        friend bool operator==(const link& x, const link& y) noexcept;  
        friend strong_ordering operator<=>(const link& x, const link& y) noexcept;  
    };  
}
```

Change 27.10.9.3 [time.zone.link.nonmembers]:

```
friend bool operator==(const link& x, const link& y) noexcept;
```

Returns: $x.name() == y.name()$.

```
bool operator<(const link& x, const link& y) noexcept;
```

~~Returns: $x.name() < y.name()$.~~

```
strong_ordering operator<=>(const link& x, const link& y) noexcept;
```

Returns: $x.name() <=> y.name()$.

4.13. Clause 28: Localization library

Change 28.3.1 [locale]:

```
namespace std {  
    class locale {  
    public:  
        [...]  
        // locale operations  
        basic_string name() const;  
  
        bool operator==(const locale& other) const;  
bool operator!=(const locale& other) const;  
        [...]  
    };  
}
```

Change 28.3.1.4 [locale.operators]:

```
bool operator==(const locale& other) const;
```

Returns: `true` if both arguments are the same locale, or one is a copy of the other, or each has a name and the names are identical; `false` otherwise.

```
bool operator!=(const locale& other) const;
```

~~Returns: `!(*this == other)`.~~

§ 4.14. Clause 29: Input/output library

Change 29.11.5 [fs.filesystem.syn]:

```
namespace std::filesystem {
    [...]
    // [fs.class.file_status], file status
    class file_status;

    struct space_info {
        uintmax_t capacity;
        uintmax_t free;
        uintmax_t available;

        friend bool operator==(const space_info&, const space_info&) = default;
    };
    [...]
}
```

Change 29.11.7 [fs.class.path]:

```
namespace std::filesystem {
    class path {
    public:
        [...]
        // [fs.path.nonmember], non-member operators
        friend bool operator==(const path& lhs, const path& rhs) noexcept;
friend bool operator!=(const path& lhs, const path& rhs) noexcept;
friend bool operator< (const path& lhs, const path& rhs) noexcept;
friend bool operator<= (const path& lhs, const path& rhs) noexcept;
friend bool operator> (const path& lhs, const path& rhs) noexcept;
friend bool operator>= (const path& lhs, const path& rhs) noexcept;
friend weak_ordering operator<=>(const path& lhs, const path& rhs) noexcept;
        [...]
    };
}
```

Change 29.11.7.7 [fs.path.nonmember], paragraphs 5-10:

```
friend bool operator!=(const path& lhs, const path& rhs) noexcept;
```

~~Returns: `!(lhs == rhs)`.~~

```
friend bool operator< (const path& lhs, const path& rhs) noexcept;
```

~~Returns: `lhs.compare(rhs) < 0` .~~

```
friend bool operator<= (const path& lhs, const path& rhs) noexcept;
```

~~Returns: `!(rhs < lhs)` .~~

```
friend bool operator> (const path& lhs, const path& rhs) noexcept;
```

~~Returns: `rhs < lhs` .~~

```
friend bool operator>= (const path& lhs, const path& rhs) noexcept;
```

~~Returns: `!(lhs < rhs)` .~~

```
friend weak_ordering operator<=>(const path& lhs, const path& rhs) noexcept;
```

Returns: `lhs.compare(rhs) <=> 0` .

```
friend path operator/ (const path& lhs, const path& rhs);
```

Effects: Equivalent to: `return path(lhs) /= rhs;`

Change 29.11.10 [fs.class.file_status]:

```
namespace std::filesystem {
    class file_status {
    public:
        // [fs.file_status.cons], constructors and destructor
        file_status() noexcept : file_status(file_type::none) {}
        explicit file_status(file_type ft,
                             perms prms = perms::unknown) noexcept;
        file_status(const file_status&) noexcept = default;
        file_status(file_status&&) noexcept = default;
        ~file_status();

        // assignments
        file_status& operator=(const file_status&) noexcept = default;
        file_status& operator=(file_status&&) noexcept = default;

        // [fs.file_status.mods], modifiers
        void type(file_type ft) noexcept;
        void permissions(perms prms) noexcept;

        // [fs.file_status.obs], observers
        file_type type() const noexcept;
        perms permissions() const noexcept;

        friend bool operator==(const file_status& lhs, const file_status& rhs) noexcept
        { return lhs.type() == rhs.type() && lhs.permissions() == rhs.permissions(); }
    };
}
```

Change 29.11.11 [fs.class.directory_entry]:

```
namespace std::filesystem {
    class directory_entry {
    public:
        [...]
        bool operator==(const directory_entry& rhs) const noexcept;
bool operator!=(const directory_entry& rhs) const noexcept;
bool operator<(const directory_entry& rhs) const noexcept;
bool operator>(const directory_entry& rhs) const noexcept;
bool operator<=(const directory_entry& rhs) const noexcept;
bool operator>=(const directory_entry& rhs) const noexcept;
weak_ordering_operator<=>(const directory_entry& rhs) const noexcept;

    private:
        filesystem::path pathobject;    // exposition only
        friend class directory_iterator; // exposition only
    };
}
```

Change 29.11.11.3 [fs.dir.entry.obs], paragraphs 31-36:

```
bool operator==(const directory_entry& rhs) const noexcept;
```

Returns: `pathobject == rhs.pathobject`.

```
bool operator!=(const directory_entry& rhs) const noexcept;
```

~~Returns: `pathobject != rhs.pathobject`.~~

```
bool operator<(const directory_entry& rhs) const noexcept;
```

~~Returns: `pathobject < rhs.pathobject`.~~

```
bool operator>(const directory_entry& rhs) const noexcept;
```

~~Returns: `pathobject > rhs.pathobject`.~~

```
bool operator<=(const directory_entry& rhs) const noexcept;
```

~~Returns: `pathobject <= rhs.pathobject`.~~

```
bool operator>=(const directory_entry& rhs) const noexcept;
```

~~Returns: `pathobject >= rhs.pathobject`.~~

```
weak_ordering_operator<=>(const directory_entry& rhs) const noexcept;
```

Returns: `pathobject <=> rhs.pathobject`.

§ 4.15. Clause 30: Regular expressions library

```

#include <initializer_list>

namespace std {
    [...]
    // [re.submatch], class template sub_match
    template<class BidirectionalIterator>
        class sub_match;

    using csub_match = sub_match<const char*>;
    using wsub_match = sub_match<const wchar_t*>;
    using ssub_match = sub_match<string::const_iterator>;
    using wssub_match = sub_match<wstring::const_iterator>;


    // [re.submatch.op], sub_match non-member operators
    template<class BiIter>
    bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
    template<class BiIter>
    bool operator!=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
    template<class BiIter>
    bool operator<(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
    template<class BiIter>
    bool operator>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
    template<class BiIter>
    bool operator<=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
    template<class BiIter>
    bool operator>=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);

    template<class BiIter, class ST, class SA>
    bool operator==(
        const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        const sub_match<BiIter>& rhs);
    template<class BiIter, class ST, class SA>
    bool operator!=(
        const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        const sub_match<BiIter>& rhs);
    template<class BiIter, class ST, class SA>
    bool operator<(
        const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        const sub_match<BiIter>& rhs);
    template<class BiIter, class ST, class SA>
    bool operator>(
        const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        const sub_match<BiIter>& rhs);
    template<class BiIter, class ST, class SA>
    bool operator<=(
        const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        const sub_match<BiIter>& rhs);
    template<class BiIter, class ST, class SA>
    bool operator>=(
        const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        const sub_match<BiIter>& rhs);

    template<class BiIter, class ST, class SA>
    bool operator==(


```

```

const sub_match<BiIter>& lhs,
const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter, class ST, class SA>
bool operator!=(
const sub_match<BiIter>& lhs,
const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter, class ST, class SA>
bool operator<(
const sub_match<BiIter>& lhs,
const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter, class ST, class SA>
bool operator>(
const sub_match<BiIter>& lhs,
const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter, class ST, class SA>
bool operator<=(
const sub_match<BiIter>& lhs,
const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter, class ST, class SA>
bool operator>=(
const sub_match<BiIter>& lhs,
const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);

template<class BiIter>
bool operator==(const typename iterator_traits<BiIter>::value_type* lhs,
const sub_match<BiIter>& rhs);
template<class BiIter>
bool operator!=(const typename iterator_traits<BiIter>::value_type* lhs,
const sub_match<BiIter>& rhs);
template<class BiIter>
bool operator<(const typename iterator_traits<BiIter>::value_type* lhs,
const sub_match<BiIter>& rhs);
template<class BiIter>
bool operator>(const typename iterator_traits<BiIter>::value_type* lhs,
const sub_match<BiIter>& rhs);
template<class BiIter>
bool operator<=(const typename iterator_traits<BiIter>::value_type* lhs,
const sub_match<BiIter>& rhs);
template<class BiIter>
bool operator>=(const typename iterator_traits<BiIter>::value_type* lhs,
const sub_match<BiIter>& rhs);

template<class BiIter>
bool operator==(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type* rhs);
template<class BiIter>
bool operator!=(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type* rhs);
template<class BiIter>
bool operator<(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type* rhs);
template<class BiIter>
bool operator>(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type* rhs);
template<class BiIter>
bool operator<=(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type* rhs);

```

```

template<class BiIter>
bool operator==(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type* rhs);

template<class BiIter>
bool operator==(const typename iterator_traits<BiIter>::value_type& lhs,
const sub_match<BiIter>& rhs);

template<class BiIter>
bool operator!=(const typename iterator_traits<BiIter>::value_type& lhs,
const sub_match<BiIter>& rhs);

template<class BiIter>
bool operator<(const typename iterator_traits<BiIter>::value_type& lhs,
const sub_match<BiIter>& rhs);

template<class BiIter>
bool operator>(const typename iterator_traits<BiIter>::value_type& lhs,
const sub_match<BiIter>& rhs);

template<class BiIter>
bool operator<=(const typename iterator_traits<BiIter>::value_type& lhs,
const sub_match<BiIter>& rhs);

template<class BiIter>
bool operator>=(const typename iterator_traits<BiIter>::value_type& lhs,
const sub_match<BiIter>& rhs);

template<class BiIter>
bool operator==(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type& rhs);

template<class BiIter>
bool operator!=(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type& rhs);

template<class BiIter>
bool operator<(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type& rhs);

template<class BiIter>
bool operator>(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type& rhs);

template<class BiIter>
bool operator<=(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type& rhs);

template<class BiIter>
bool operator>=(const sub_match<BiIter>& lhs,
const typename iterator_traits<BiIter>::value_type& rhs);

template<class charT, class ST, class BiIter>
basic_ostream<charT, ST>&
operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);

// [re.results], class template match_results
template<class BidirectionalIterator,
class Allocator = allocator<sub_match<BidirectionalIterator>>>
class match_results;

using cmatch = match_results<const char*>;
using wcmatch = match_results<const wchar_t*>;
using smatch = match_results<string::const_iterator>;
using wsmatch = match_results<wstring::const_iterator>;

// match_results comparisons

```



```

template<class BidirectionalIterator, class Allocator>
bool operator==(const match_results<BidirectionalIterator, Allocator>& m1,
const match_results<BidirectionalIterator, Allocator>& m2);
template<class BidirectionalIterator, class Allocator>
bool operator!=(const match_results<BidirectionalIterator, Allocator>& m1,
const match_results<BidirectionalIterator, Allocator>& m2);

[...]
```

Change 30.9 [re.submatch]:

```

namespace std {
    template<class BidirectionalIterator>
    class sub_match : public pair<BidirectionalIterator, BidirectionalIterator> {
    public:
        using value_type      =
            typename iterator_traits<BidirectionalIterator>::value_type;
        using difference_type =
            typename iterator_traits<BidirectionalIterator>::difference_type;
        using iterator        = BidirectionalIterator;
        using string_type     = basic_string<value_type>;

        bool matched;

        constexpr sub_match();

        difference_type length() const;
        operator string_type() const;
        string_type str() const;

        int compare(const sub_match& s) const;
        int compare(const string_type& s) const;
        int compare(const value_type* s) const;

        friend bool operator==(const sub_match& lhs, const sub_match& rhs);
        friend compare_three_way_result_t<string_type> operator<=>(const sub_match& lhs, const sub_match& rhs);

        template<class ST, class SA>
        friend bool operator==(const sub_match& lhs, const basic_string<value_type, ST, SA>& rhs);
        template<class ST, class SA>
        friend compare_three_way_result_t<string_type>
        operator<=>(const sub_match& lhs, const basic_string<value_type, ST, SA>& rhs);

        friend bool operator==(const sub_match& lhs, const value_type* rhs);
        friend compare_three_way_result_t<string_type> operator<=>(const sub_match& lhs, const value_type* rhs);

        friend bool operator==(const sub_match& lhs, const value_type& rhs);
        friend compare_three_way_result_t<string_type> operator<=>(const sub_match& lhs, const value_type& rhs);
    };
}
```

Replace the entirety of 30.9.2 [re.submatch.op] (all 42 non-member comparison operators, plus an `operator<<`) with:

```

friend bool operator==(const sub_match& lhs, const sub_match& rhs);
```

Returns: `lhs.compare(rhs) == 0`.

```
friend compare_three_way_result_t<string_type> operator<=>(const sub_match& lhs, const sub_match& rhs);
```

Returns: `lhs.compare(rhs) <=> 0`.

```
template<class ST, class SA>
friend bool operator==(const sub_match& lhs, const basic_string<value_type, ST, SA>& rhs);
```

Returns: `lhs.compare(string_type(rhs.data(), rhs.size())) == 0`

```
template<class ST, class SA>
friend compare_three_way_result_t<string_type>
operator<=>(const sub_match& lhs, const basic_string<value_type, ST, SA>& rhs);
```

Returns: `lhs.compare(string_type(rhs.data(), rhs.size())) <=> 0`

```
friend bool operator==(const sub_match& lhs, const value_type* rhs);
```

Returns: `lhs.compare(rhs) == 0`.

```
friend compare_three_way_result_t<string_type> operator<=>(const sub_match& lhs, const value_type* rhs);
```

Returns: `lhs.compare(rhs) <=> 0`.

```
friend bool operator==(const sub_match& lhs, const value_type& rhs);
```

Returns: `lhs.compare(string_type(1, rhs)) == 0`.

```
friend compare_three_way_result_t<string_type> operator<=>(const sub_match& lhs, const value_type& rhs);
```

Returns: `lhs.compare(string_type(1, rhs)) <=> 0`.

```
template<class charT, class ST, class BiIter>
basic_ostream<charT, ST>&
operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);
```

Returns: `os << m.str()`.

Change 30.10 [re.results]:

```
namespace std {
    template<>
    class match_results {
    public:

        [...]
        // [re.results.swap], swap
        void swap(match_results& that);

        friend bool operator==(const match_results& m1, const match_results& m2);
    };
}
```

```
};
}
```

Change 30.10.8 [re.results.nonmember]:

```
template<class BidirectionalIterator, class Allocator>
friend bool operator==(const match_results<BidirectionalIterator, Allocator>& m1,
                      const match_results<BidirectionalIterator, Allocator>& m2);
```

Returns: `true` if neither match result is ready [...]

```
template<class BidirectionalIterator, class Allocator>
bool operator!=(const match_results<BidirectionalIterator, Allocator>& m1,
               const match_results<BidirectionalIterator, Allocator>& m2);
```

Returns: `!(m1 == m2)`.

4.16. Clause 31: Atomic operations library

No changes necessary.

4.17. Clause 32: Thread support library

Change 32.3.2.1 [thread.thread.id]:

```
namespace std {
    class thread::id {
    public:
        id() noexcept;

        friend bool operator==(thread::id x, thread::id y) noexcept;
        friend strong_ordering operator<==(thread::id x, thread::id y) noexcept;
    };

    bool operator==(thread::id x, thread::id y) noexcept;
    bool operator!=(thread::id x, thread::id y) noexcept;
    bool operator<(thread::id x, thread::id y) noexcept;
    bool operator>(thread::id x, thread::id y) noexcept;
    bool operator<=(thread::id x, thread::id y) noexcept;
    bool operator>=(thread::id x, thread::id y) noexcept;

    template<class charT, class traits>
        basic_ostream<charT, traits>&
            operator<<(basic_ostream<charT, traits>& out, thread::id id);

    // hash support
    template<class T> struct hash;
    template<> struct hash<thread::id>;
}
```

and paragraphs 6-11:

```
friend bool operator==(thread::id x, thread::id y) noexcept;
```

Returns: `true` only if `x` and `y` represent the same thread of execution or neither `x` nor `y` represents a thread of execution.

```
bool operator!=(thread::id x, thread::id y) noexcept;
```

Returns: ~~`!(x == y)`~~

```
bool operator<(thread::id x, thread::id y) noexcept;
```

Returns: A value such that `operator<` is a total ordering as described in [alg.sorting].

```
friend strong_ordering operator<==(thread::id x, thread::id y) noexcept;
```

Returns: A value such that `operator<==` is a total ordering.

```
bool operator>(thread::id x, thread::id y) noexcept;
```

Returns: ~~`y < x`~~ :

```
bool operator<=(thread::id x, thread::id y) noexcept;
```

Returns: ~~`!(y < x)`~~ :

```
bool operator>=(thread::id x, thread::id y) noexcept;
```

Returns: ~~`!(x < y)`~~ :

§ 5. References

- [P0790R2] "Effect of operator<==> on the C++ Standard Library" by David Stone, 2019-01-21
- [P0891R2] "Make strong_order a Customization Point!" by Gašper Ažman, Jeff Snyder, 2019-01-20
- [P1154R1] "Type traits for structural comparison" by Arthur O'Dwyer, Jeff Snyder, 2019-01-19
- [P1185R1] "<==> != ==" by Barry Revzin, 2019-01-22
- [P1186R1] "When do you actually use <==>?" by Barry Revzin, 2019-01-22
- [P1188R0] "Library utilities for <==>" by Barry Revzin, 2019-01-22
- [P1189R0] "Add <==> to Library" by Barry Revzin, 2019-01-22
- [P1191R0] "Adding operator<==> to types that are not currently comparable" by David Stone, 2018-08-22
- [P1295R0] "Spaceship library update" by Tomasz Kamiński, 2018-10-07
- [P1380R1] "Ambiguity and Insecurities with Three-Way Comparison" by Lawrence Cowl, 2019-01-20